

Genomics Data Visualization

with Python · Day 1 · Foundations &
differential expression

Dr. Loukas Theodosiou // Day 1 // June 2026

2026-06-22

Why do we need data visualisations?

WHY WE VISUALIZE · DAY 01

Hello, I'm Loukas

YOUR HOST

Welcome to Genomics Data Visualization with Python

I'm **Loukas Theodosiou**, a data scientist in genomics, machine learning, and data visualization.

Happy to host you. We'll go from matplotlib basics to publication-ready genomics figures, and *when* to use each.

FEEDBACK

I'd love your **feedback as we go**. I'll add small breaks so you can DM me, what's landing, what's not.

Questions welcome throughout.



Loukas Theodosiou, PhD

Senior Bioinformatics Scientist | AI
Researcher | R package Development | N...



Anscombe's Quartet

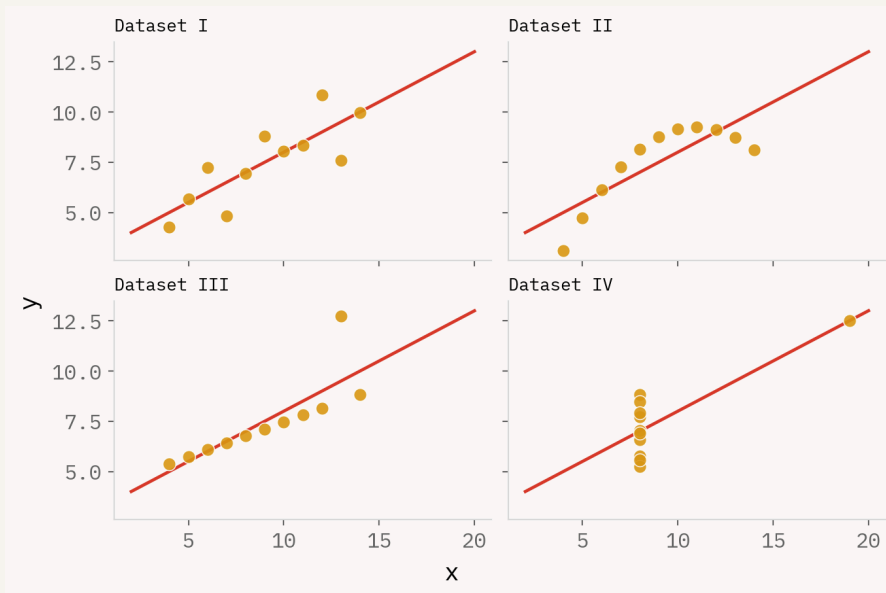
WHY WE VISUALIZE

Same statistics, different shapes

Four datasets with identical means, variance, correlation (0.816), and regression line. Summary statistics alone can hide the shape of your data, so plot first.

PLOT

CODE



The Datasaurus Dozen

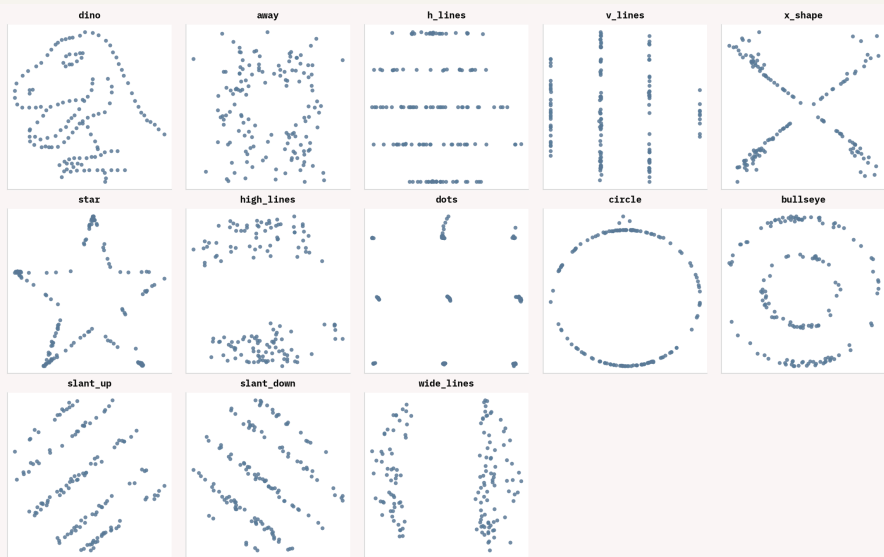
WHY WE VISUALIZE

Identical stats, different shapes

All 13 datasets share almost the same summary statistics: $\bar{x} \approx 54.3$, $\bar{y} \approx 47.8$, $\sigma_x \approx 16.8$, $\sigma_y \approx 26.9$. Same numbers, wildly different pictures.

PLOT

CODE



Big data in biology

WHY WE VISUALIZE · SCALE

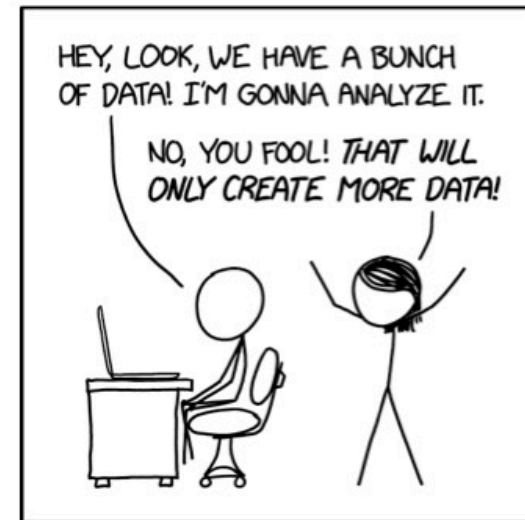
The Genomics Data Explosion

WHY WE VISUALIZE

Scale of the problem

- 1 to 2 exabytes of data produced on YouTube per year
- 220 million genomes sequenced per year
- Equivalent to 40 exabytes per year
- That's more than 20 times more than YouTube produces

For years we struggled to collect biological data, but now our biggest bottleneck is **analyzing and interpreting it**



How are we going to work?

COURSE LOGISTICS · DAY 01

Course Structure

COURSE LOGISTICS

Format and Day-1 timeline

- 1. Presentations.** I present the material; questions welcome throughout or at the end. Slides are in your course materials.
- 2. Live demonstrations.** Observe and take notes; practice time follows. All code is in your course notebook.
- 3. Hands-on practice.** Work through exercises at your own pace, with all tutorials and datasets provided as Quarto/Jupyter notebooks.

GOAL

By the end you can build a **publication-ready** plot, and know **which** plot to reach for.

Day 1 plan (today)

Time	Topic
13:00 – 13:30	Fundamentals of data visualization
13:30 – 14:15	Python and the visualization ecosystem
14:15 – 15:15	Bar, stacked bar, and distribution plots
15:15 – 15:45	<i>Break</i>
15:45 – 16:15	Volcano plots
16:15 – 17:00	Dimensionality reduction: PCA, t-SNE, UMAP
17:00 – 17:30	Heatmaps, waffle, and waterfall plots
17:30 – 18:00	Peak + pathway/annotation plots

Fundamentals of Data Visualization

CHART DESIGN PRINCIPLES · DAY 01

The Lie Factor

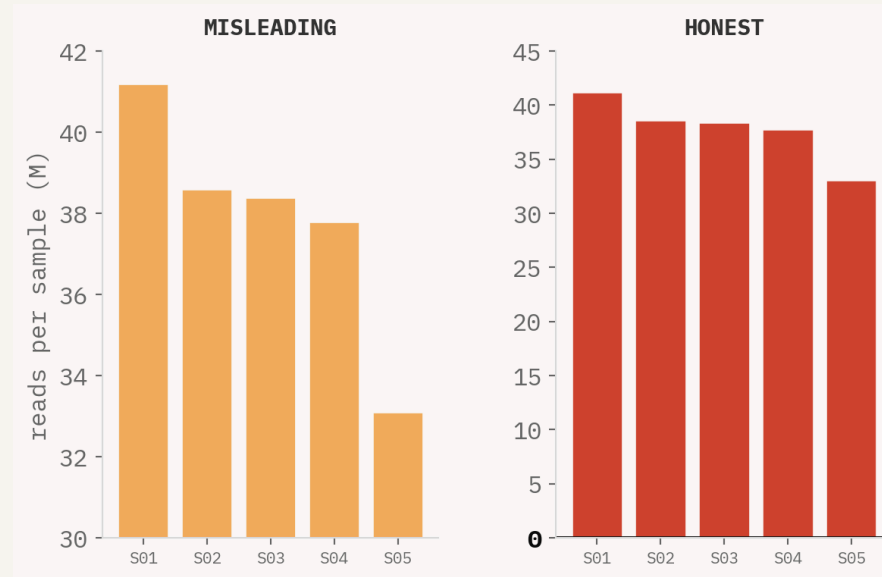
HONEST CHARTS • BASELINES

Set bars to zero

Bars encode by **length**, so the value axis must start at zero. Truncate it and a routine batch looks like one sample failed.

Lines and dots can be zoomed, just label the axis.

PLOT CODE



THE BOTTOM LINE

Bars start at **zero**. Lines and dots can be zoomed, just label the axis.

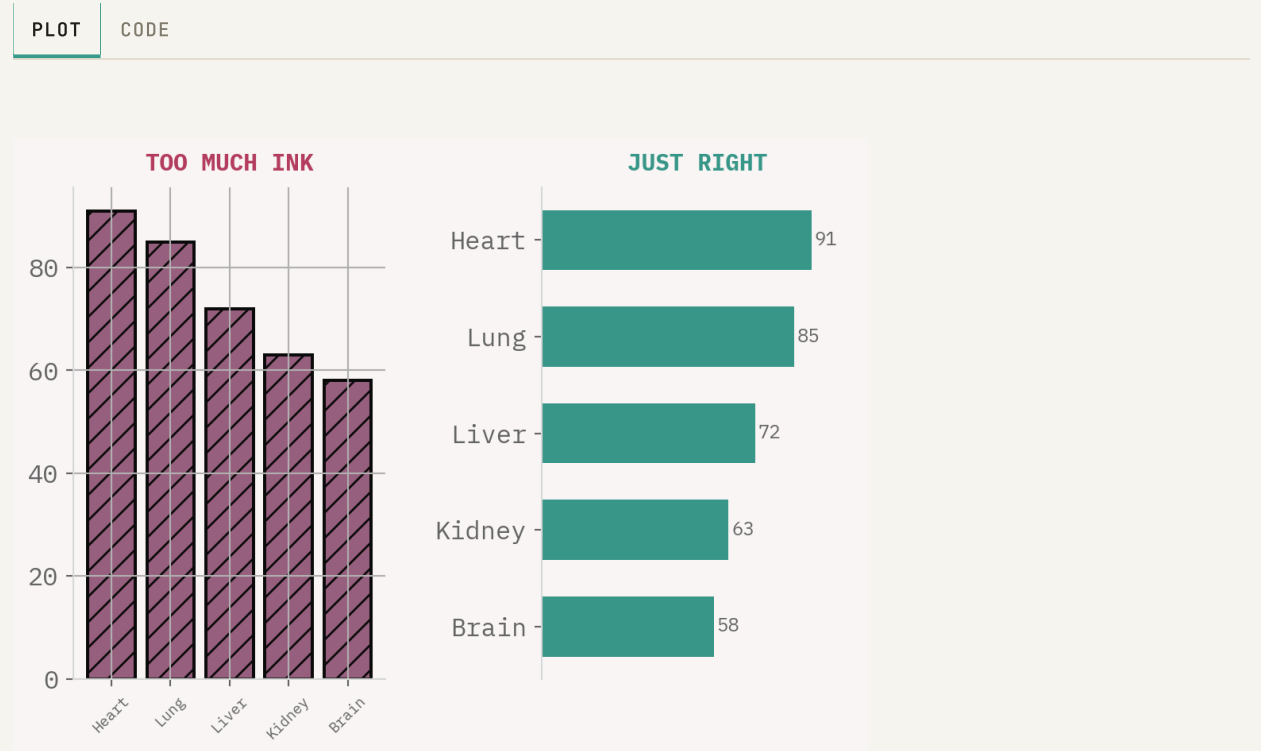
Data-to-Ink Ratio

HONEST CHARTS • SIMPLICITY

Remove what doesn't help, keep what does

Every drop of ink should encode data. Strip the gridlines, hatching, and heavy borders, keep the bars and the numbers.

Like seasoning: too much overwhelms, too little is bland.



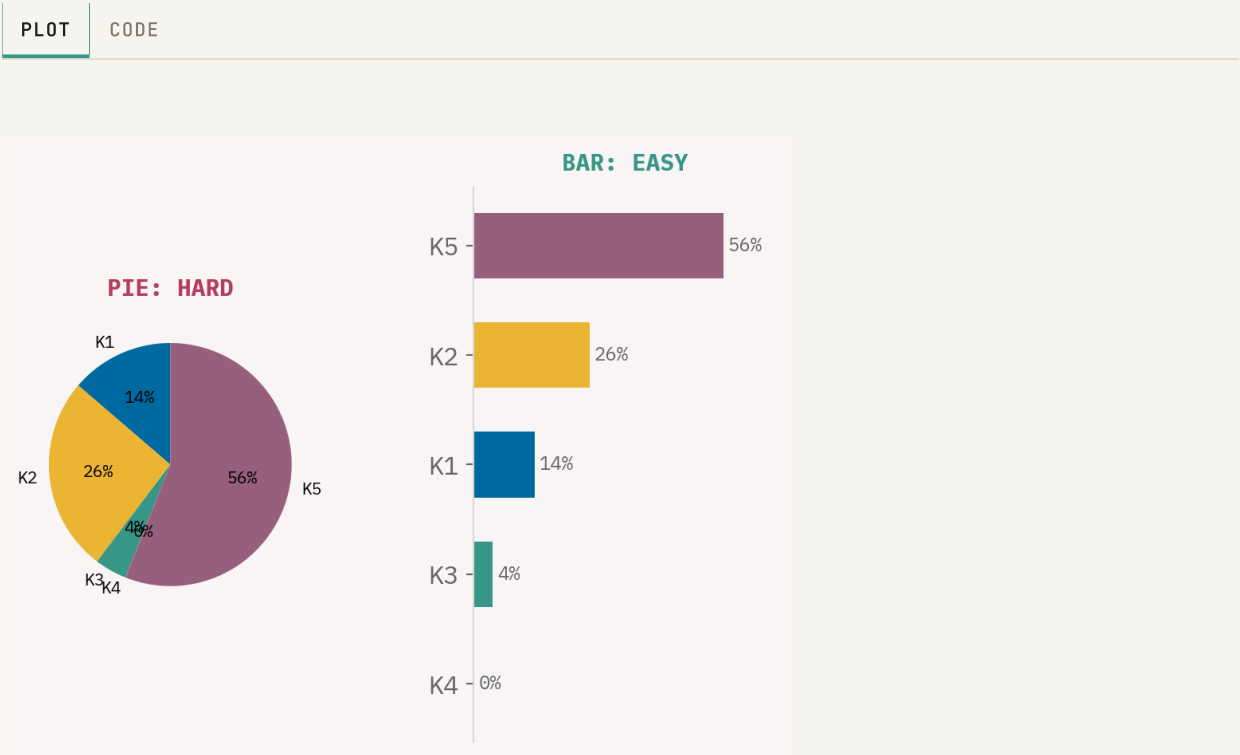
Maximise the share of ink that encodes real data. The right amount lets the data shine.

Length Beats Area

HONEST CHARTS • GEOMETRIC ACCURACY

Compare with bars, not slices

The eye judges **length** well and **area** poorly. A pie asks you to compare wedge areas, and the small ancestry components blur together. The same proportions as bars rank in an instant.



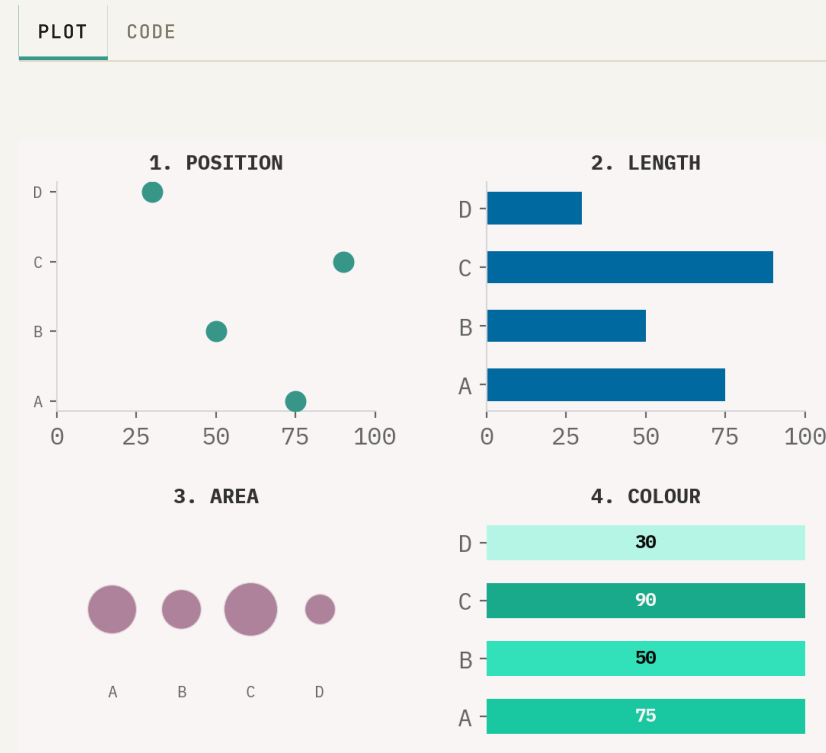
How We See: Perceptual Ranking

HONEST CHARTS • PERCEPTION

Position is read most accurately, colour least

The same four values, four ways. Position is read most accurately, then length, then area, and colour least of all.

Encode your **main message** with position or length, reserve area and colour for secondary variables.



Give your audience the easiest channel. Position and length for the headline, colour and area for the rest.

Pre-attentive Pop-out

HONEST CHARTS • PERCEPTION

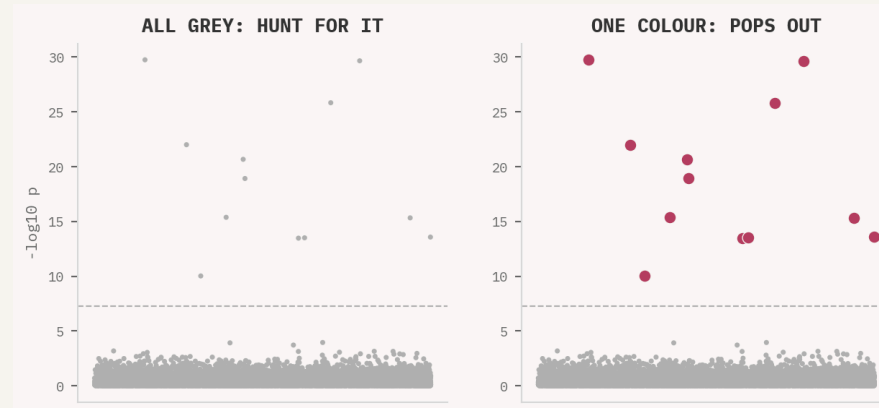
Find the hit: a slow hunt, or an instant pop

Same Manhattan plot, same significant SNP. On the left every point is grey, so you have to **hunt** along the line for the hit. On the right one highlight colour makes it **pop out** before you even look.

Colour, size, and orientation are pre-attentive. Encode the thing you want seen first with one of them.

PLOT

CODE



THE BOTTOM LINE

Colour, size, and orientation pop out pre-attentively. Put your one finding on a channel the eye catches for free.

Pre-attentive: Size

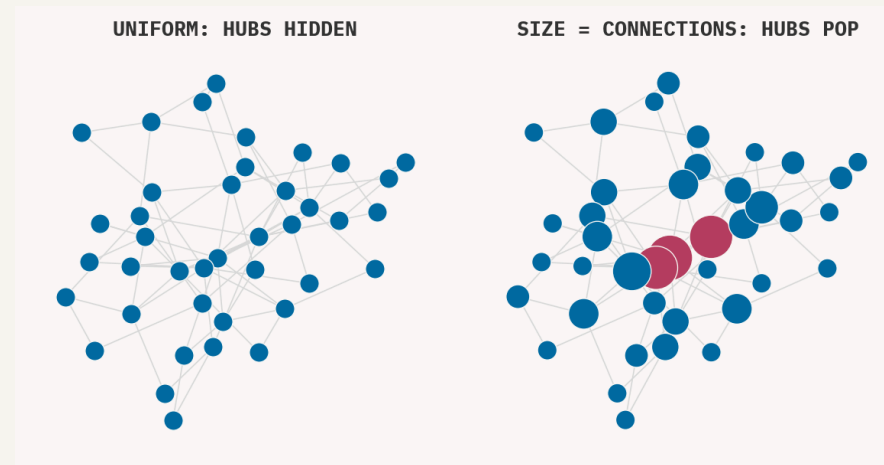
HONEST CHARTS • PERCEPTION

Make the size mean something, hubs jump out

PLOT CODE

In a co-expression network, every gene drawn the same size hides the structure. Map node **size to the number of connections** and the hub genes pop out instantly, no labels, no hunting.

Size is pre-attentive: bigger reads as “more” before you think.



Map size to a real quantity (degree, expression, effect size). Bigger reads as more, instantly.

Pre-attentive: Direction

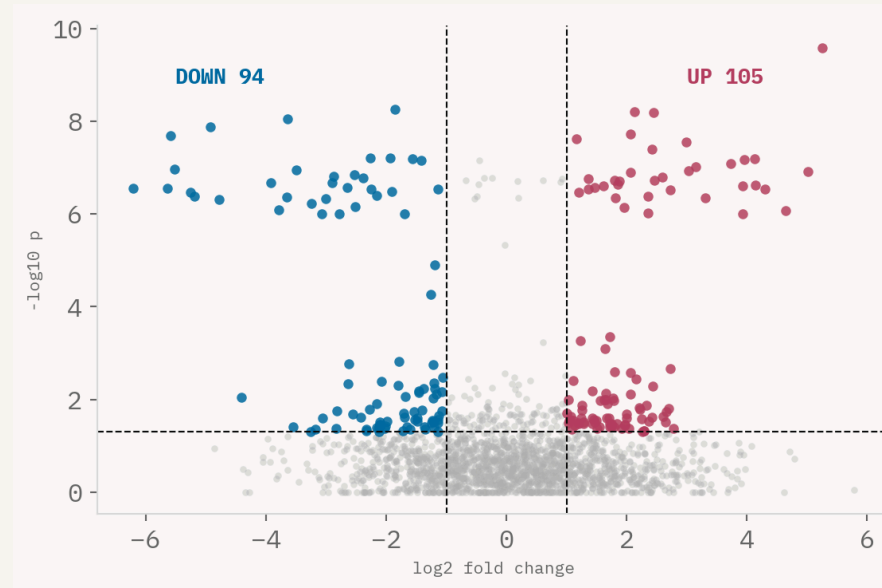
HONEST CHARTS • PERCEPTION

Left vs right reads as down vs up

A volcano splits genes by the **sign** of the fold change: down-regulated to the left, up-regulated to the right. Position relative to zero is the direction, the eye sorts the two groups before reading a single label.

Like an age pyramid (male left, female right), direction carries meaning for free.

PLOT CODE



THE BOTTOM LINE

Position around a zero line encodes direction. Down-left, up-right, the eye groups them before reading.

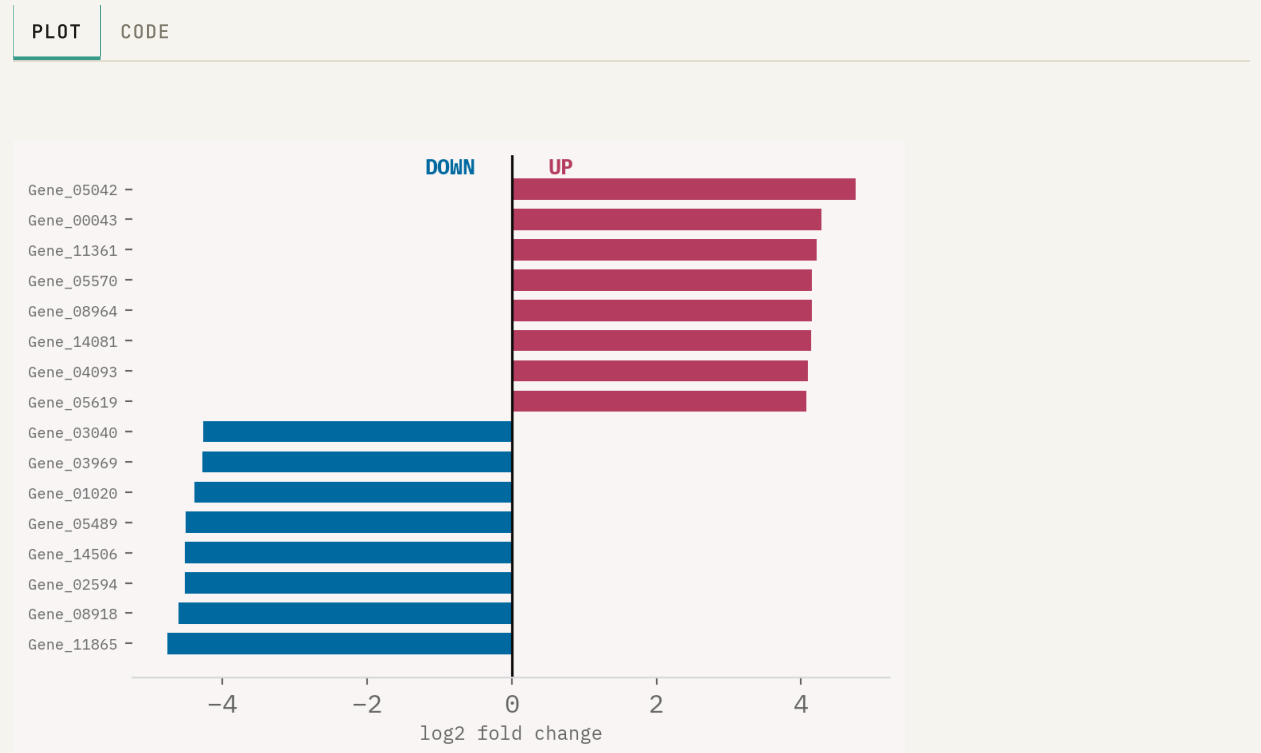
Pre-attentive: Direction with Bars

HONEST CHARTS • PERCEPTION

Anchor at zero, let the bars point up or down

The volcano shows direction by *position*; a diverging bar shows it by *length from zero*. Down-regulated genes grow left, up-regulated grow right, ranked by effect size.

Same pre-attentive idea as an age pyramid: one zero line, two directions, no legend needed.



THE BOTTOM LINE

Anchor bars at zero and let length encode direction. Down-left, up-right, ranked by effect size.

Colour in Data Visualization

COLOUR THEORY FOR CHARTS · DAY 01

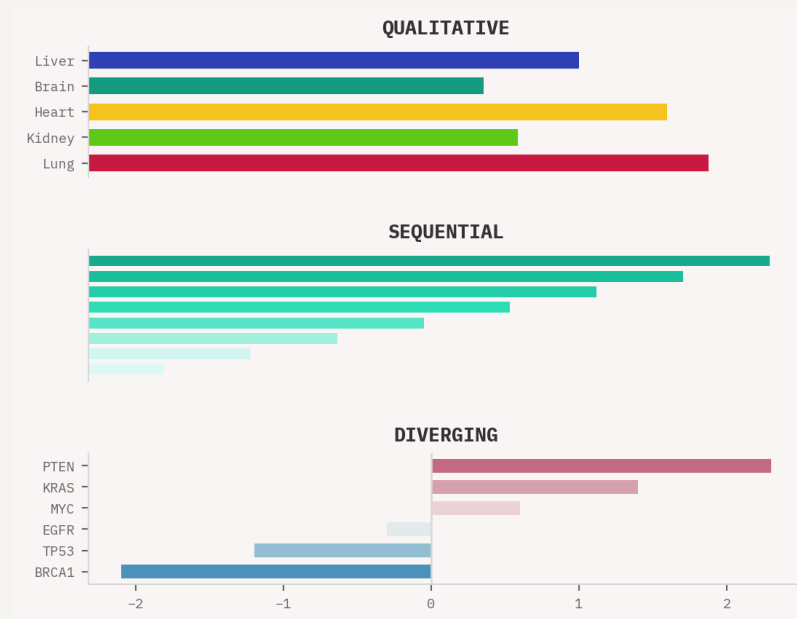
Three Palette Types

COLOUR • PALETTE TYPES

Match the palette to your data's structure

Qualitative for unordered categories, **sequential** for magnitude (one hue, light to dark), **diverging** for signed values around zero (two hues, neutral midpoint).

PLOT CODE



The Rainbow Trap & Accessibility

COLOUR • ACCESSIBILITY

Avoid rainbow scales, design for colour-blindness

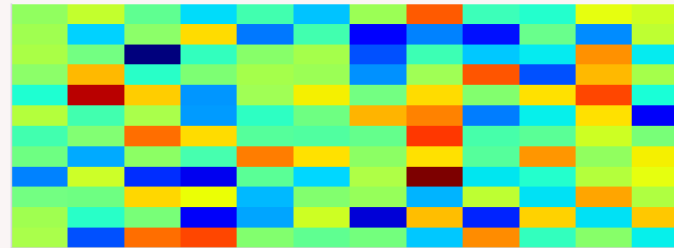
Hue has no natural order, so the brain cannot turn a rainbow back into magnitude. Prefer perceptually-uniform maps, avoid red-green pairs, and use the squint test.

About 5% of your audience is colour-blind.

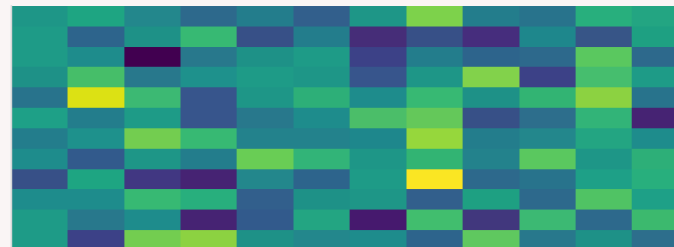
PLOT

CODE

JET (RAINBOW): MISLEADING



VIRIDIS: UNIFORM



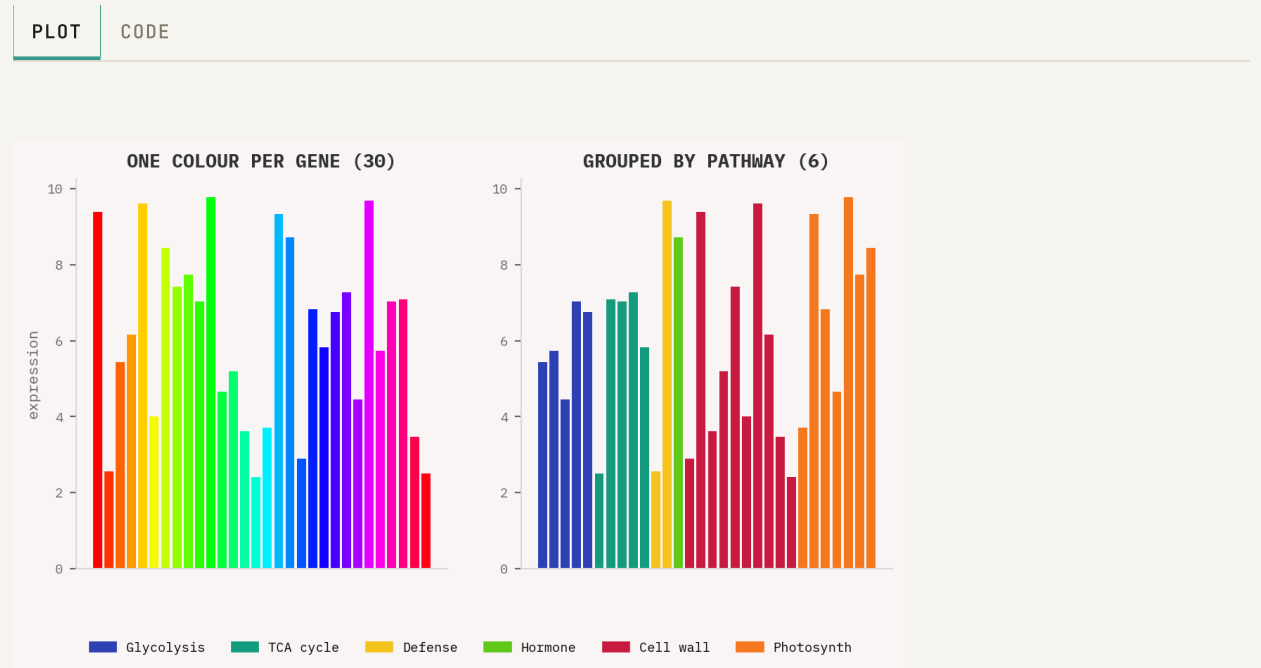
Colour Pitfall: Too Many Colours

COLOUR • PITFALLS

Don't give colour a job too big for it

One colour per gene across 30 genes is a rainbow with no meaning, the hues blur and the legend is useless. Group the genes into a few **pathways** and sort by group: now colour reads as six clean blocks instead of noise.

Keep categorical palettes to about 5 to 7 colours. Beyond that, group, facet, or label.



THE BOTTOM LINE

When you have too many categories, group them. Colour by pathway, and sort so each group reads as a block.

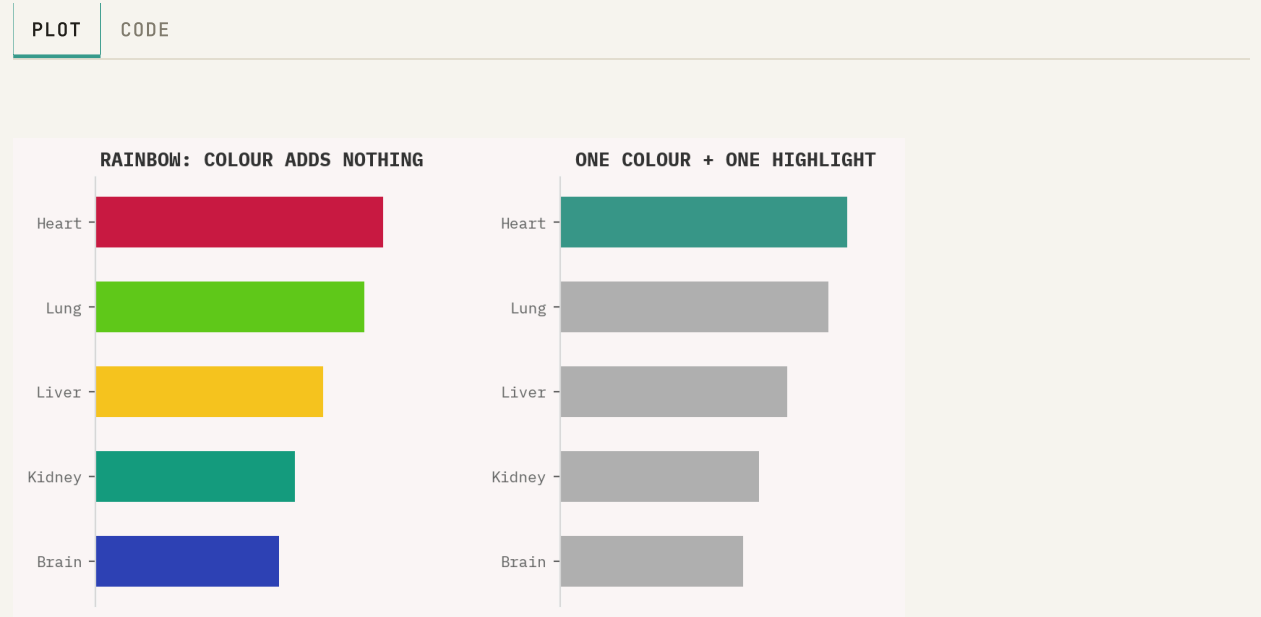
Colour Pitfall: Colour for No Reason

COLOUR • PITFALLS

Don't encode in colour what's already shown

If the x-axis already tells you which tissue is which, colouring each bar a different hue adds nothing, it just adds visual load and a redundant legend. Use a single colour, and save a highlight hue for the one bar that matters.

Colour should add information, not decorate.



If the axis already identifies the category, colour is redundant. Spend it on the one thing you want seen.

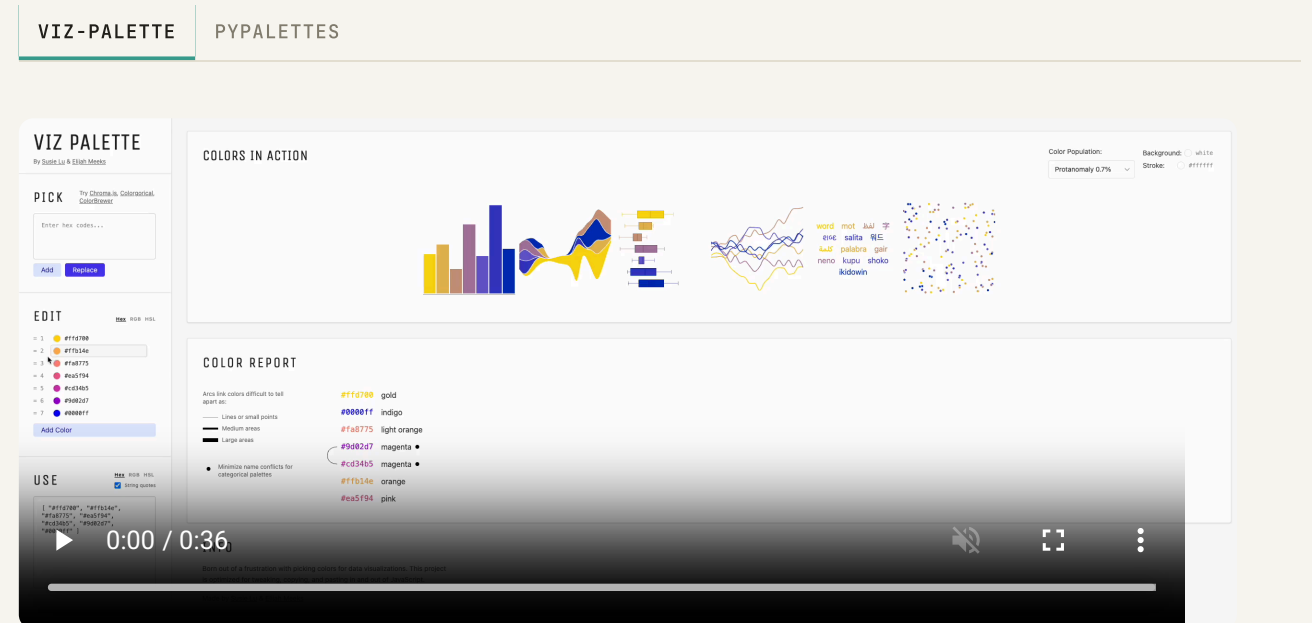
Colour Pitfall: Forgetting Colour-Blindness

COLOUR • PITFALLS

Test your palette the way ~5% will see it

A red/green pair looks fine to you and collapses to one colour for a red-green colour-blind reader. Always check under simulated deficiency, and pair colour with a second cue (shape, position, label).

These tools simulate it live, drop in your hexes and see.



THE BOTTOM LINE

A red/green pair can vanish for ~5% of readers. Simulate it, and never let colour carry the message alone.

Finding Good Palettes

COLOUR • RESOURCES

Don't invent colours, borrow tested ones

Three rules for any palette: enough contrast, colour-blind safe, and only as many colours as you have groups. Rather than guessing hexes, pull from a curated source:

- **viz-palette** (`projects.susielu.com/viz-palette`), paste your hex list and simulate every type of colour-blindness.
- **R color palettes**, Emil Hvitfeldt (`emilhvitfeldt.github.io/r-color-palettes`), curated discrete sets (Wes Anderson, scientific) with copyable hex codes.
- **R Graph Gallery palette finder** (`r-graph-gallery.com/color-palette-finder`), browse and filter by palette type.

IN PYTHON

`pypalettes` brings all these curated palettes into matplotlib: `load_cmap("Darjeeling1")` and you're done.

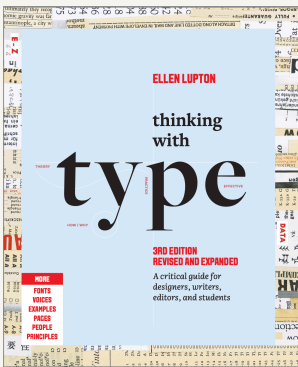
Typography for Scientific Figures

DESIGN • TYPOGRAPHY

Pick a font for the job, then stop choosing

Five font categories, each with a job. For figures and slides you mostly need two: a clean **sans serif** for labels and a **monospace** for code and aligned numbers. A **display** face for titles is optional.

Pick one of each and reuse them everywhere, consistency reads as polish.



READ Ellen Lupton, *Thinking with Type*

Category	Characteristics	Best used for	Examples
Sans Serif	Clean, modern	Screen, slides	Asap, Helvetica
Serif	Traditional, formal	Papers, reports	Georgia, Times
Slab Serif	Bold serifs, strong	Headings, emphasis	Rockwell, Courier
Monospace	Fixed-width	Code, alignment	JetBrains Mono
Display	Decorative, distinct	Titles, branding	Space Grotesk

Grey Is Your Best Friend

COLOUR • STRATEGIC USE

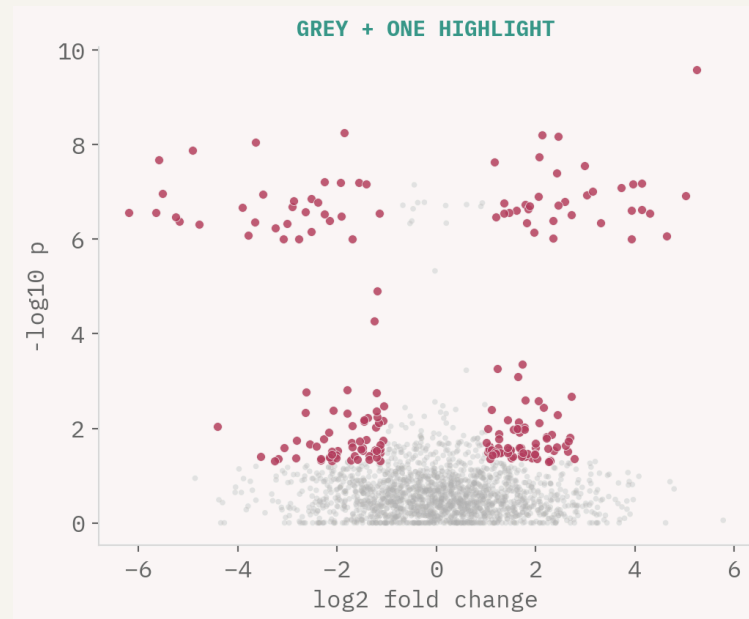
Highlight by de-emphasising everything else

Grey the 95% that is context, colour the 5% that matters.
In a volcano of 2,000 genes, the significant hits should be the only thing in colour.

Like silence in music, it makes the melody stand out.

PLOT

CODE



Colour the 5% that matters, grey the 95% that is context.

From Chart to Story

NARRATIVE DATA VISUALIZATION • DAY 01

The Story Arc

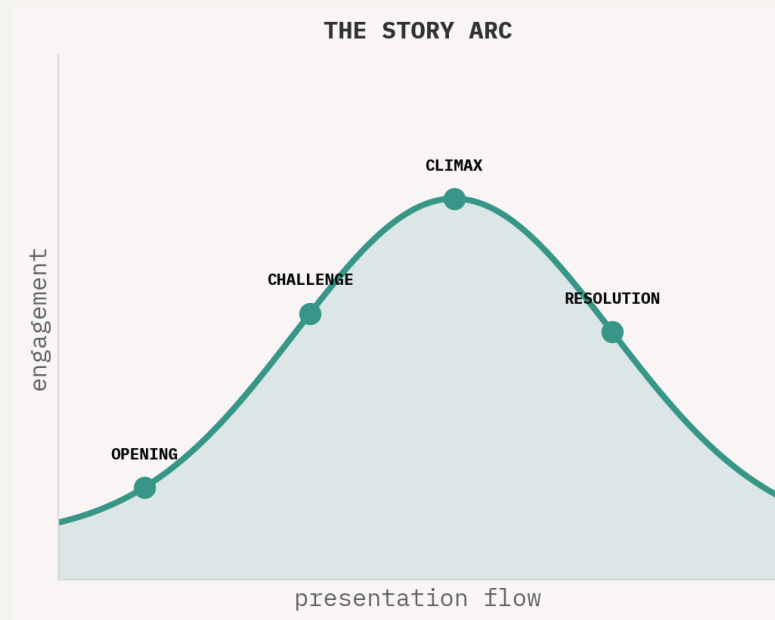
STORYTELLING · ARC

Every data presentation needs tension and resolution

A single figure rarely tells the whole story. Open with context, raise a challenge (which loci?), show the action (fine-mapping), then resolve it with the causal variant. Chapters, not one slide.

PLOT

CODE



Build Up Towards Complexity

STORYTELLING • PROGRESSIVE DISCLOSURE

Show a simple version first, then reveal the full picture

Teach the reader to read **one** panel first, then show the full grid with shared scales. Like playing one bar on the piano before handing over the orchestral score.



Editorial Thinking: Angle, Frame, Focus

STORYTELLING • EDITORIAL

Put the finding in the title, the highlight on one bar

“Expression varies across tissues” is a topic. “Heart shows the highest BRCA1” is an **angle**, so put it in the title. Frame to the tissues that matter, focus with one highlight colour.

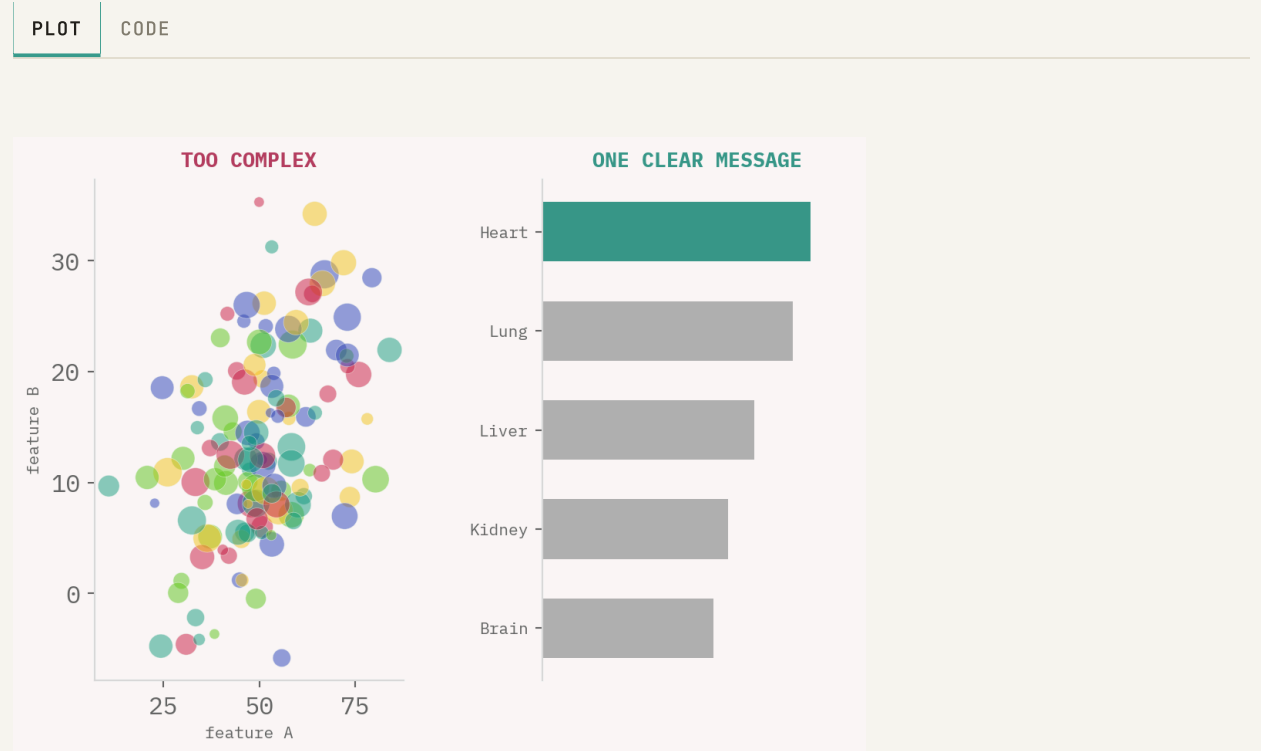


Make a Figure for the Generals

STORYTELLING • SIMPLICITY

If a busy reader can't get the point in ten seconds, simplify

The generals aren't slow, they're busy. If five data dimensions are tangential to your point, strip them down to the one comparison that answers the question.
(after Claus Wilke)



If five dimensions are tangential to your point, remove them.

Memorability vs Minimalism

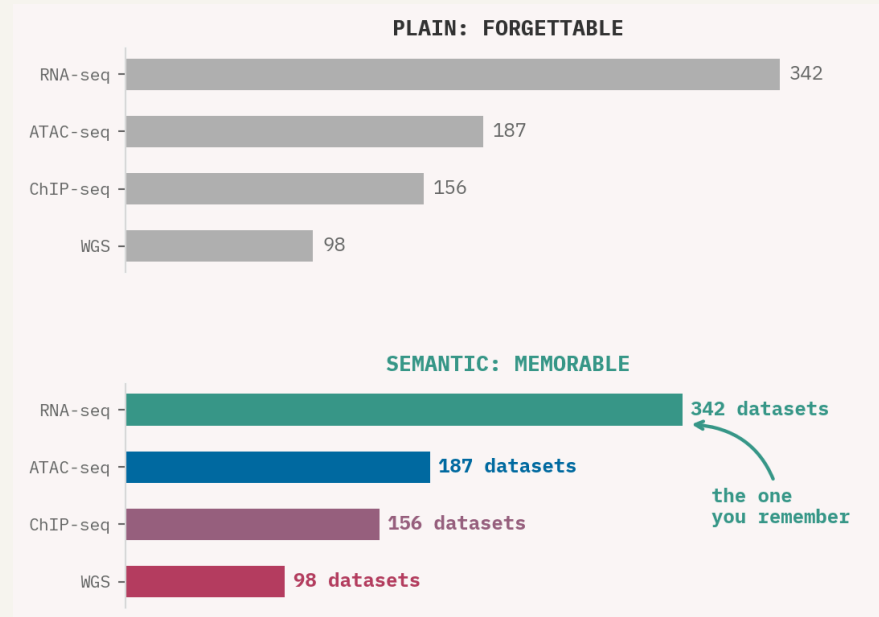
STORYTELLING • MEMORABILITY

Simple is clear, distinctive is remembered

Tufte says strip everything, research says distinctive figures stick. Both are right: add elements that **carry meaning**, one consistent colour per assay and the key number, never decoration.

PLOT

CODE



Data Transformations

HONEST CHARTS • SCALES

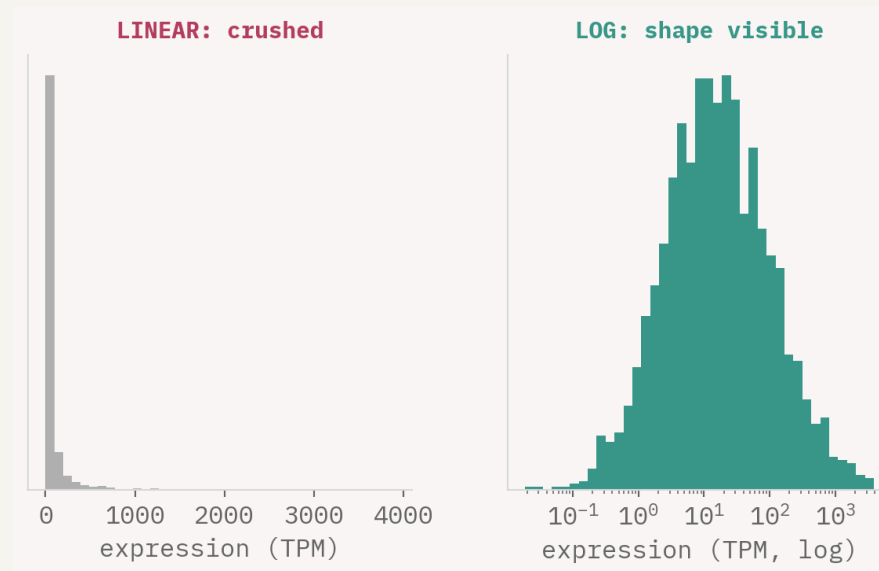
Log the axis when values span orders of magnitude

Expression, counts, and p-values are **right-skewed**: most values are small, a few are huge. On a linear axis everything crushes left. A log axis spreads them out and the real shape appears.

Log for orders of magnitude, sqrt for counts, linear for a narrow range.

PLOT

CODE



Log for data spanning orders of magnitude, sqrt for counts, linear for a narrow range. Always label the axis as transformed.

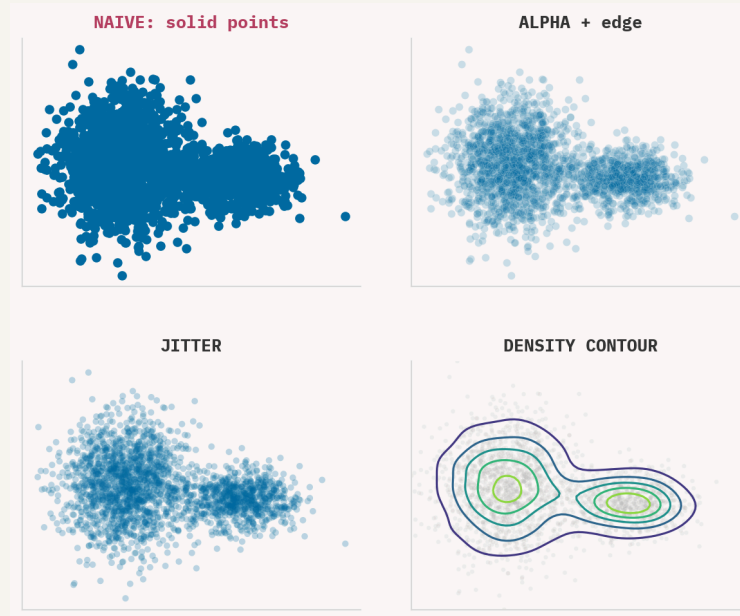
Seeing Through Overplotting

HONEST CHARTS • DENSITY

When points pile up, reveal the density

PLOT CODE

Thousands of solid points become one blob. Four fixes: **transparency** (alpha) plus a thin edge, **jitter** to separate stacked points, and a **density contour** to show where the mass actually is.



THE BOTTOM LINE

Solid points hide structure. Add transparency and a thin edge, jitter to unstack, or draw a density contour to show where points concentrate.

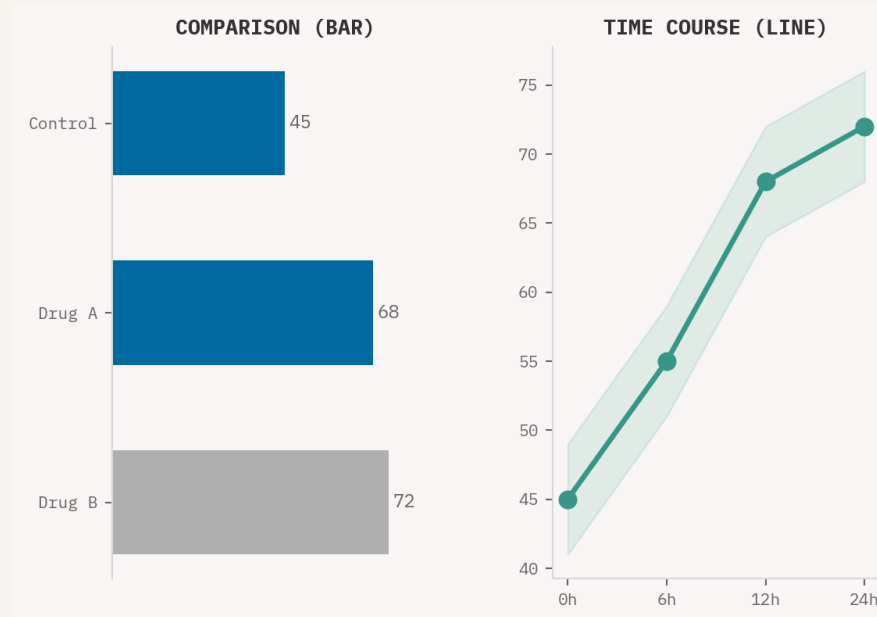
Don't Repeat Yourself: Vary Chart Types

STORYTELLING • VARIETY

Use a different visualization for each distinct analysis

PLOT CODE

Two bar charts blur together. Use a **bar** for the discrete comparison and a **line** for the time course, and each panel reads as its own idea.

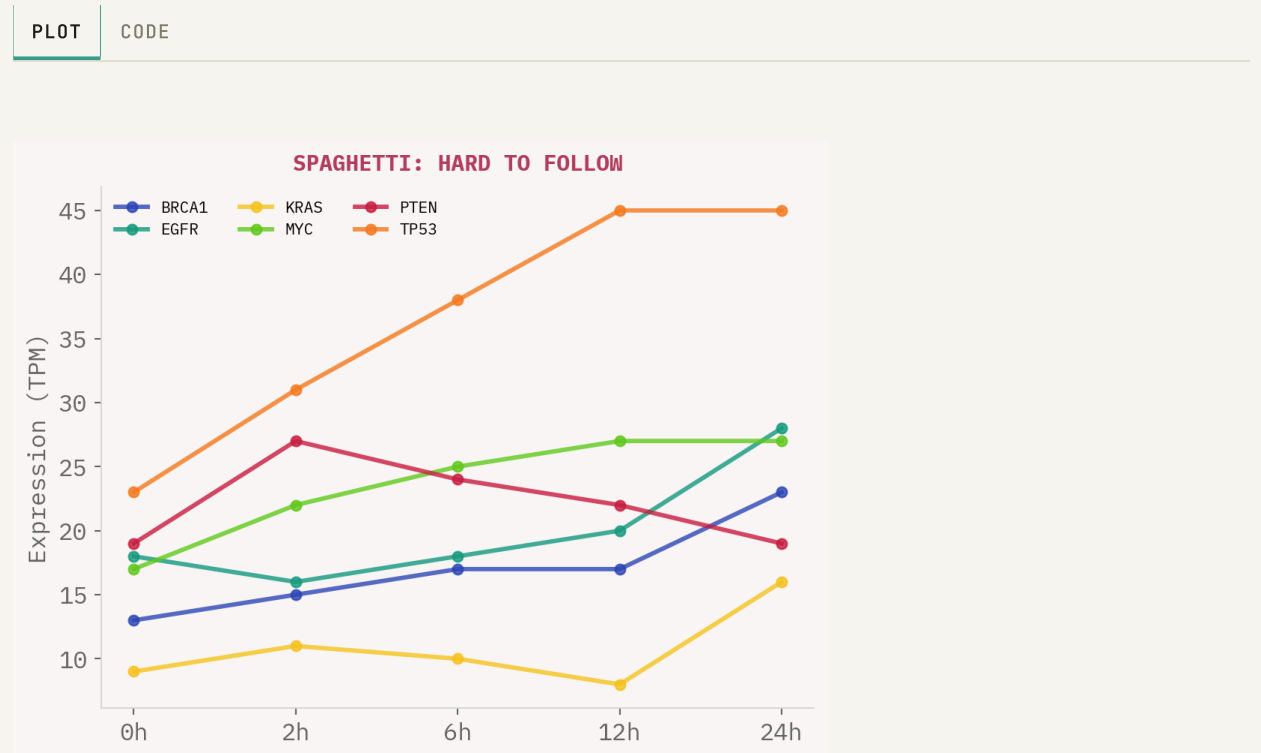


Small Multiples · Step 1: the spaghetti problem

STORYTELLING · FACETING

Six genes on one axis overlap into an unreadable tangle

Plotting many series on a single axis quickly becomes spaghetti: the lines cross, the legend grows, and no one trend is followable.



Small Multiples · Step 2: the faceted fix

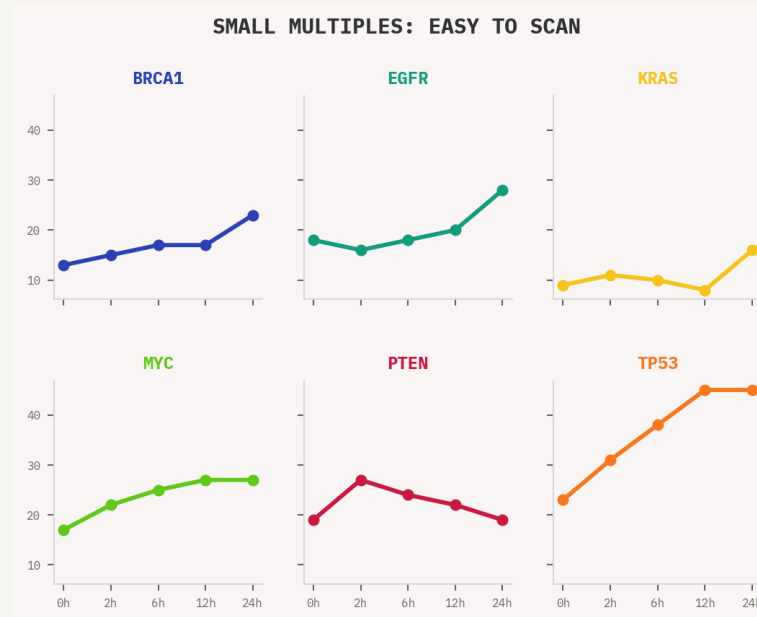
STORYTELLING · FACETING

One small panel per gene, with shared scales, scans at a glance

PLOT

CODE

Give each series its own panel and **share the axes** so the panels are comparable. The eye scans a grid far faster than it untangles overlapping lines.



What We've Covered, What's Next

FUNDAMENTALS · RECAP

The rules so far, and the ones still ahead

COVERED TODAY

HONEST CHARTS

Bars start at zero · maximise data-ink · length beats area · see through overplotting

PERCEPTION

Perceptual ranking, position first · pre-attentive pop-out · Gestalt grouping

COLOUR

Three palette types · no rainbow · design for colour-blindness · grey is your friend

STORYTELLING

Angle in the title · story arc · figure for the generals · small multiples

COMING LATER

DISTRIBUTIONS

Histogram bin width · KDE bandwidth · boxplots hide shape

PROPORTIONS

Pie vs stacked vs side-by-side · stacked only for ~2 categories

SCALES

Log bars anchor at 1, not 0 · amounts or ratios?

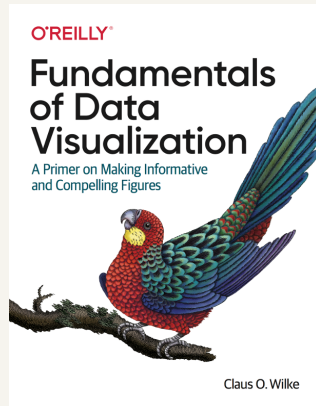
DIMENSIONALITY

Why we avoid 3D · map the third variable to colour or facets

Read These Next

DAY 01 • FURTHER READING

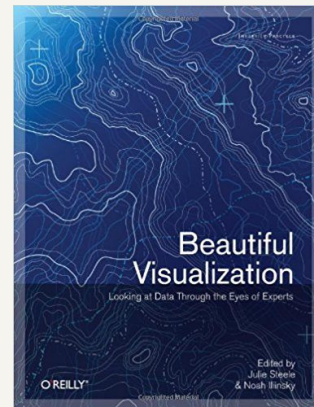
Four books that shaped this course



**Fundamentals of Data
Visualization** Claus Wilke

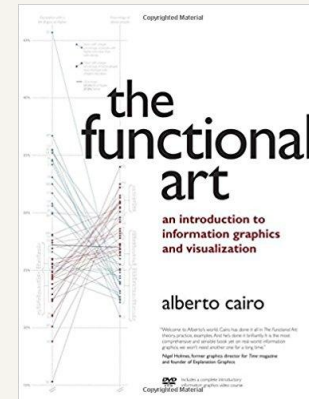
Free online

clauswilke.com/dataviz



Beautiful Visualization Steele
& Iliinsky

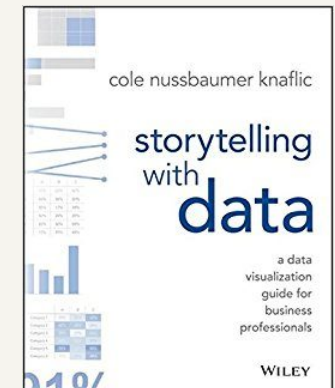
O'Reilly oreilly.com



The Functional Art
Alberto Cairo

Author site

thefunctionalart.com



Storytelling with Data Cole
Nussbaumer Knaflic

Author site

storytellingwithdata.com

START HERE

Wilke is free online and closest to this course. The other three deepen storytelling, colour, and craft.

Why Python for Genomics Visualization?

WHY WE VISUALIZE · SCALE

Why Python for Genomics Visualization?

THE PYTHON VIZ STACK

One language, full pipeline

Python's strengths in bioinformatics:

Ecosystem breadth

- One language for the full pipeline: parsing → analysis → visualization
- Direct integration with `scikit-learn`, `scanpy`, `biopython`
- Reproducible notebooks with Jupyter / Quarto

Interactivity out of the box

- Plotly, Bokeh, Panel for web-ready figures
- DashBio for genomics-specific widgets

Publication quality

- Matplotlib + Seaborn: journal-grade static figures
- Fine-grained control over every visual element

```
1 # The whole pipeline in one language
2 import pandas as pd          # data wrangling
3 import numpy as np           # numerics
4 import matplotlib.pyplot as plt # static plots
5 import seaborn as sns        # statistical plots
6 import plotly.express as px   # interactive plots
7
8 # Genomics-specific
9 import scanpy as sc           # single-cell
10 from Bio import SeqIO         # sequence parsing
11 import pyCircize              # circos plots
12 import toytree                # phylogenetics
```

Where to Find More

EXPLORE FURTHER

A gallery of genome-visualization tools

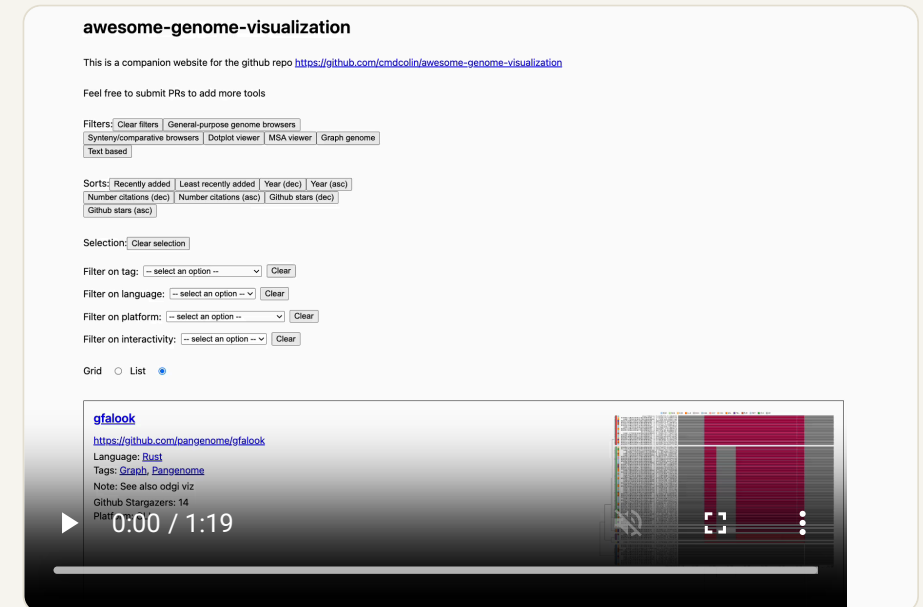
Before we start building, a resource worth bookmarking: [awesome-genome-visualization](https://github.com/cmdcolln/awesome-genome-visualization).

It's a curated, searchable catalogue of genome-visualization tools, each with a screenshot and a link to the paper and the code. When you need a plot for a specific kind of genomic data, chances are someone has built it, and you can see the repo behind it.

A great place to find inspiration, and the study that produced each figure.

⋮

⋮ {column width="58%"}



From ggplot2 to Python — The Mental Map

FROM GGLOT2 TO PYTHON

Same grammar, different syntax

ggplot2 (R) Built on the Grammar of Graphics, the same ideas power Python's ecosystem.

ggplot2	Python equivalent
<code>aes(color=group)</code>	color kwarg / palette dict
<code>geom_point()</code>	<code>ax.scatter()</code>
<code>geom_line()</code>	<code>ax.plot()</code>
<code>facet_wrap(~group)</code>	<code>fig, axes = plt.subplots()</code>
<code>theme()</code>	<code>rcParams</code> + <code>ax.set_*</code>
<code>scale_color_manual()</code>	custom <code>palette</code> dict
<code>labs()</code>	<code>ax.set_title/xlabel/ylabel()</code>
<code>ggsave()</code>	<code>fig.savefig(..., dpi=300)</code>

The key shift in mindset:

- ggplot2 maps data aesthetics **declaratively** — you describe *what* to show
- matplotlib/seaborn are **imperative** — you describe *how* to draw it
- seaborn bridges the gap with a high-level API closer to ggplot2
- For exploratory work → seaborn; for full control → matplotlib

Every ggplot2 figure you've made has a matplotlib equivalent. The grammar is the same, only the syntax changes.

Same Plot, Three Syntaxes

FROM GGLOT2 TO PYTHON

First, meet the data

Plotting packages ship example datasets. Load iris straight from seaborn, then peek at it:

```
1 import seaborn as sns
2
3 iris = sns.load_dataset("iris") # bundled with the package
4 print(iris.head())             # raw text, not a rendered table
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Five columns: four measurements plus a `species` label, the column we will map to colour.

Same Plot, Three Syntaxes

FROM GGLOT2 TO PYTHON

One scatterplot of iris, three ways

matplotlib · imperative

```
1 for sp, g in iris.groupby("species"):
2     plt.scatter(g.sepal_length,
3                 g.sepal_width, label=sp)
4 plt.legend()
```

You loop the species and add the legend yourself.

seaborn · in between

```
1 sns.scatterplot(data=iris,
2                 x="sepal_length",
3                 y="sepal_width",
4                 hue="species")
```

One call. `hue` maps colour for you.

ggplot2 (R) · declarative

```
1 ggplot(iris, aes(sepal_length,
2                 sepal_width,
3                 colour = species)) +
4     geom_point()
```

You declare the mapping; R draws it.

matplotlib makes you loop and build the legend. seaborn and ggplot2 just declare the colour mapping. Same grammar, different syntax.

One Plot, Three Libraries

THE PYTHON ECOSYSTEM

Same scatter, three tools, three trade-offs

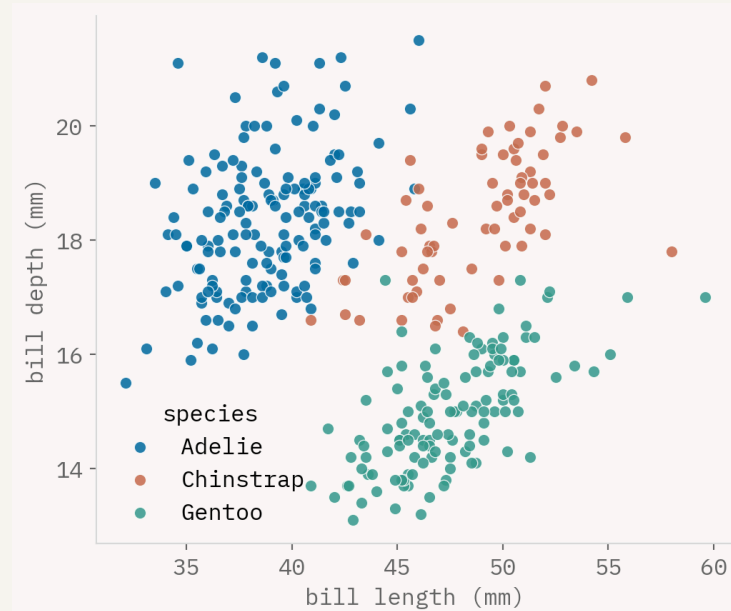
matplotlib gives control, seaborn gives speed, plotly gives interactivity. Pick by the job, not by habit. Setup once, then plot three ways.

```
1 import seaborn as sns
2
3 peng = sns.load_dataset("penguins").dropna(
4     subset=["bill_length_mm", "bill_depth_mm", "species"])
5 PAL = {"Adelie": BLUE, "Chinstrap": "#C56B4E", "Gentoo": GREEN}
```

MATPLOTLIB SEABORN PLOTLY

Full control. `plt.subplots()` hands you the figure (the frame) and `ax` (the canvas you draw on); you loop the groups and build the legend yourself.

```
1 fig, ax = plt.subplots()
2 for sp, g in peng.groupby("species"):
3     ax.scatter(g.bill_length_mm,
4               g.bill_depth_mm,
5               color=PAL[sp],
6               label=sp, s=38)
7 ax.set_xlabel("bill length (mm)")
8 ax.set_ylabel("bill depth (mm)")
9 ax.legend(frameon=False)
```



Install the Course Package

SETUP • THE ONE-LINER

Run this in any Python environment

Colab / Jupyter / any notebook

```
1 # Installs the repo's package directly
2 # (pip clones it internally, pins the branch)
3 !pip install -q "git+https://github.com/loukesio/dataviz-genomicsdata.git@Python_202
```

After install:

```
1 from genomics_course.theme import *
2 from genomics_course.data import load_admixture
```

Local clone (for the .qmd source too)

```
1 git clone https://github.com/loukesio/dataviz-genomicsdata.git
2 cd dataviz-genomicsdata
3 pip install -e .
```

The parquet datasets are bundled with the install — no separate build step needed.

ONE LINE, EVERY ENVIRONMENT

`pip install git+...@Python_2026` brings in **every dependency** the deck uses — no per-package installs. If you see `google-colab 1.0.0 requires pandas==2.2.2` after install on Colab, you can ignore it: the package works fine.

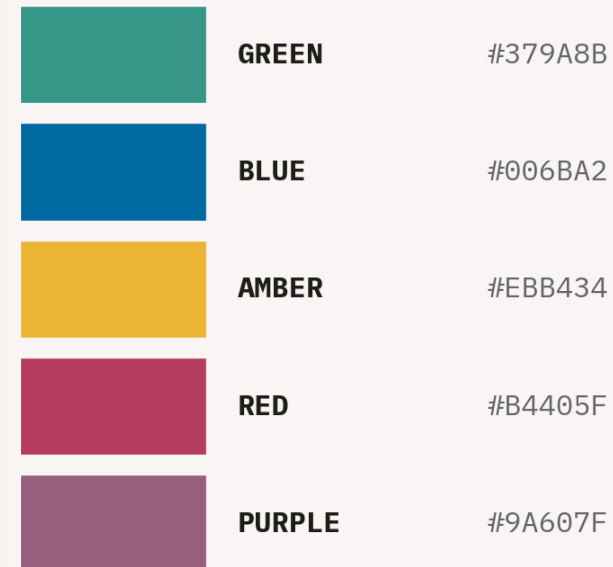
Build Your Own Theme • Step 1 — The Palette

THEME BUILDER • TOKENS

Name your colours once

Pick a few story colours and some quiet neutrals, and bind them to named constants. These are your design tokens. Nothing is styled yet, you're just deciding the vocabulary every later figure will speak.

```
1 # five story colours + neutrals — your design tokens
2 GREEN, BLUE, AMBER = "#379A8B", "#006BA2", "#EBB434"
3 RED, PURPLE       = "#B4405F", "#9A607F"
4 INK, MUTED, GREY  = "#211f1a", "#666666", "#B3B3B3"
5 LINE, CREAM       = "#D9D9D9", "#FAF9F7"
6
7 swatch = [("GREEN", GREEN), ("BLUE", BLUE), ("AMBER", AMBER), ("RED", RED), ("PURPLE", PURPLE)]
8 fig, ax = plt.subplots(figsize=(5.6, 4.4))
9 for i, (name, hexv) in enumerate(swatch):
10     y = len(swatch) - 1 - i
11     ax.add_patch(plt.Rectangle((0, y), 1.1, 0.8, color=hexv))
12     ax.text(1.3, y + 0.4, name, va="center", fontweight="bold", color=INK)
13     ax.text(2.8, y + 0.4, hexv, va="center", color=MUTED)
14 ax.set_xlim(0, 4.5); ax.set_ylim(0, len(swatch)); ax.axis("off")
15 plt.show()
```



Build Your Own Theme • Step 2 — Named Ramps

THEME BUILDER • RAMPS

Dark-to-light gradients for many categories

Five colours run out fast with many-category or continuous data. Add named **ramps**, each a dark-to-light gradient you can pick by name. The full theme ships nine; three are shown here.

```
1 # nine named dark-light ramps (three shown) for category & sequential data
2 ECON = {
3     "chicago": ["#141F52", "#3C4A8C", "#7480C4", "#B7BEEB", "#EBEDFA"],
4     "hongkong": ["#169C7F", "#4FC2A8", "#8FD9C9", "#C7ECE4", "#E9FCF8"],
5     "newyork": ["#F9C31F", "#FAD25A", "#FBE08C", "#FDEFC2", "#FEF8E6"],
6 }
7 fig, ax = plt.subplots(figsize=(6.0, 4.0))
8 for j, (name, ramp) in enumerate(ECON.items()):      # one row per ramp
9     for i, hexv in enumerate(ramp):
10         ax.add_patch(plt.Rectangle((i, -j), 0.95, 0.9, color=hexv, ec=CREAM, lw=0.6))
11     ax.text(-0.3, -j + 0.45, name, ha="right", va="center", family="IBM Plex Mono",
12 ax.set_xlim(-2.5, 5); ax.set_ylim(-len(ECON) + 0.5, 1); ax.axis("off")
13 plt.show()
```

chicago



hongkong



newyork



Build Your Own Theme • Step 3 — Continuous Colormaps

THEME BUILDER • CMAPS

Turn the ramps into matplotlib colormaps

Heatmaps and stacked areas need a *continuous* gradient. `LinearSegmentedColormap.from_list` interpolates any list of hexes into a colormap: one sequential, one diverging, one full-spectrum.

```
1 from matplotlib.colors import LinearSegmentedColormap
2
3 # interpolate the ramps into continuous colormaps
4 econ_cmap = LinearSegmentedColormap.from_list("econ_chicago", ECON["chicago"])
5 econ_diverging = LinearSegmentedColormap.from_list("econ_div", [BLUE, CREAM, RED])
6 econ_spectrum = LinearSegmentedColormap.from_list(
7     "econ_spectrum", [ECON["chicago"][1], ECON["hongkong"][1], ECON["newyork"][1], R
8
9 grad = np.linspace(0, 1, 256).reshape(1, -1)
10 fig, axes = plt.subplots(3, 1, figsize=(6.0, 3.4))
11 demos = [(econ_cmap, "econ_cmap • sequential"),
12          (econ_diverging, "econ_diverging • blue ↔ red"),
13          (econ_spectrum, "econ_spectrum • blue → red")]
14 for a, (cm, label) in zip(axes, demos):
15     a.imshow(grad, aspect="auto", cmap=cm); a.set_xticks([]); a.set_yticks([])
16     a.set_title(label, loc="left", family="IBM Plex Mono", fontsize=10, color=INK, p
17 plt.tight_layout(); plt.show()
```

econ_cmap • sequential



econ_diverging • blue ↔ red



econ_spectrum • blue → red



Build Your Own Theme • Step 4 — Register the Font

THEME BUILDER • FONT

Tell matplotlib about the .ttf files

The repo ships the IBM Plex Mono `.ttf` files. Walk the folder, hand each file to matplotlib's font manager, and remember the family name. A fallback keeps the render alive if the folder ever moves.

```
1 from pathlib import Path
2 import matplotlib.font_manager as fm
3
4 # register the bundled .ttf files so matplotlib knows the family by name
5 font_dir = Path("fonts/IBM_Plex_Mono")
6 if font_dir.exists():
7     for ttf in font_dir.glob("*.ttf"):
8         fm.fontManager.addfont(str(ttf))
9     MONO = "IBM Plex Mono"
10 else:
11     MONO = "DejaVu Sans Mono"      # safe fallback
12
13 fig, ax = plt.subplots(figsize=(6.0, 4.0)); ax.axis("off")
14 ax.text(0.5, 0.70, "IBM Plex Mono", ha="center", fontsize=30, fontweight="bold", col
15 ax.text(0.5, 0.45, 'ax.set_title("BRCA1 expression")', ha="center", fontsize=12, col
16 ax.text(0.5, 0.25, "now available everywhere by name", ha="center", fontsize=11, col
17 plt.show()
```

IBM Plex Mono

`ax.set_title("BRCA1 expression")`

now available everywhere by name

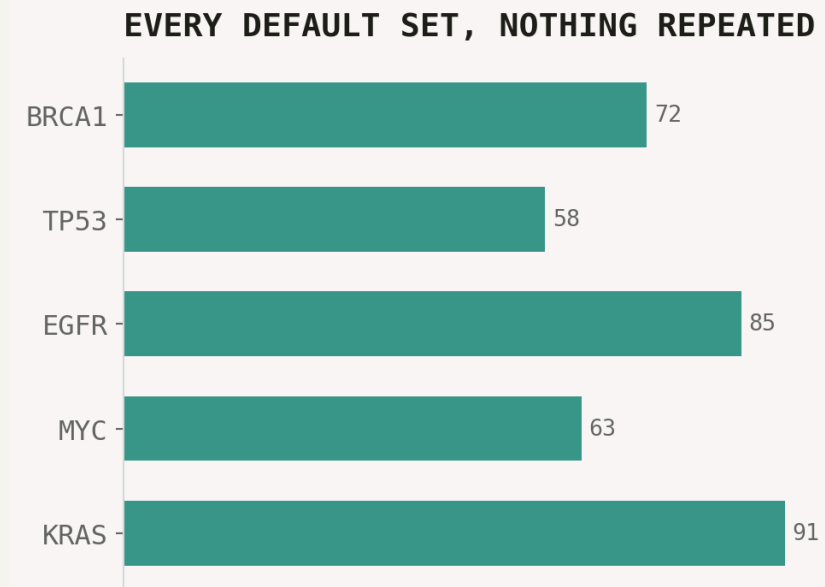
Build Your Own Theme • Step 5 — Set Defaults Once

THEME BUILDER • RCPARAMS

One rcParams call, inherited forever

A single `rcParams.update` makes the cream canvas, muted axes, dropped spines, and IBM Plex Mono the matplotlib default. Every figure after this line starts styled, with no per-chunk boilerplate.

```
1 import matplotlib as mpl
2
3 # set every default once — later figures inherit it, no per-plot styling
4 mpl.rcParams.update({
5     "figure.facecolor": CREAM, "axes.facecolor": CREAM, "savefig.facecolor": CREAM,
6     "font.family": MONO,
7     "axes.edgecolor": LINE, "axes.labelcolor": MUTED,
8     "xtick.color": MUTED, "ytick.color": MUTED,
9     "axes.spines.top": False, "axes.spines.right": False,
10 })
11
12 genes = ["BRCA1", "TP53", "EGFR", "MYC", "KRAS"]; vals = [72, 58, 85, 63, 91]
13 fig, ax = plt.subplots(figsize=(6.0, 4.2))
14 ax.barh(genes[::-1], vals[::-1], color=GREEN, height=0.62)
15 for i, v in enumerate(vals[::-1]):
16     ax.text(v + 1, i, str(v), va="center", fontsize=10, color=MUTED)
17 ax.set_xlim(0, 105); ax.xaxis.set_visible(False)
18 for s in ["top", "right", "bottom"]: ax.spines[s].set_visible(False)
19 ax.set_title("EVERY DEFAULT SET, NOTHING REPEATED", fontweight="bold", color=INK, lo
20 plt.show()
```



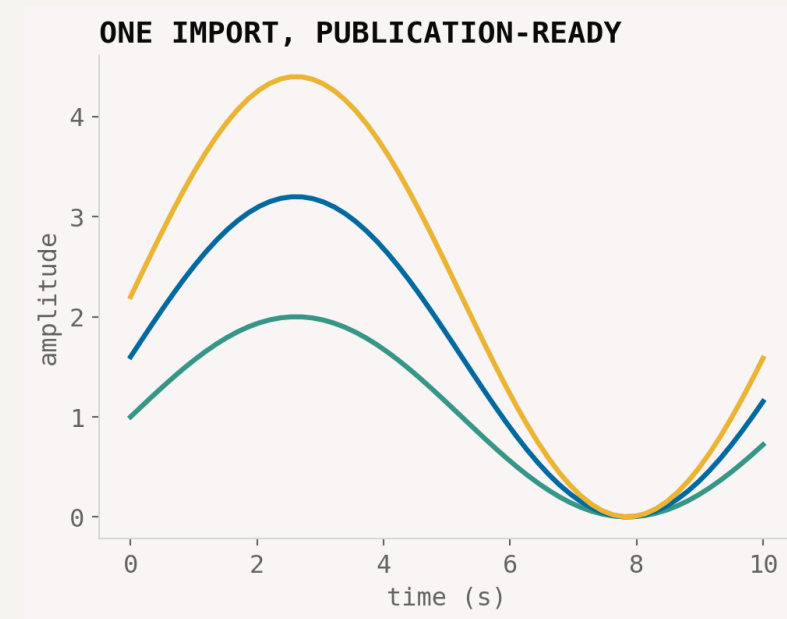
The Whole Theme in One Line

THEME BUILDER • THE PAYOFF

```
from genomics_course.theme import *
```

You've just seen the inside of `theme.py`: `palette`, `ramps`, `colormaps`, `font`, `rcParams`. The deck never re-declares any of it. It imports the same module, and every figure inherits the look.

```
1 from genomics_course.theme import * # palette + ramps + cmaps + font + rcParams
2
3 # every figure now starts publication-ready - no styling needed
4 x = np.linspace(0, 10, 60)
5 fig, ax = plt.subplots(figsize=(6.0, 4.2))
6 for k, c in zip((1, 1.6, 2.2), (GREEN, BLUE, AMBER)):
7     ax.plot(x, np.sin(x * 0.6) * k + k, lw=2.4, color=c)
8 ax.set_title("ONE IMPORT, PUBLICATION-READY", fontweight="bold", color=INK, loc="lef
9 ax.set_xlabel("time (s)"); ax.set_ylabel("amplitude")
10 plt.show()
```



THE BOTTOM LINE

The recipe is **palette → ramps → colormaps → font → rcParams → one import line**. Define it once; every figure in Day 1 through Day 3, and in your own work, shares the same identity.

Does the Theme Cover Seaborn & Plotly?

FIRST PLOTS · SETUP

One rule: matplotlib and anything built on it

Your `rcParams` are matplotlib's defaults, so **seaborn inherits them for free**, it draws on matplotlib. The one trap: `sns.set_theme()` overwrites your rcParams with seaborn's grey style, so don't call it (or re-import your theme after).

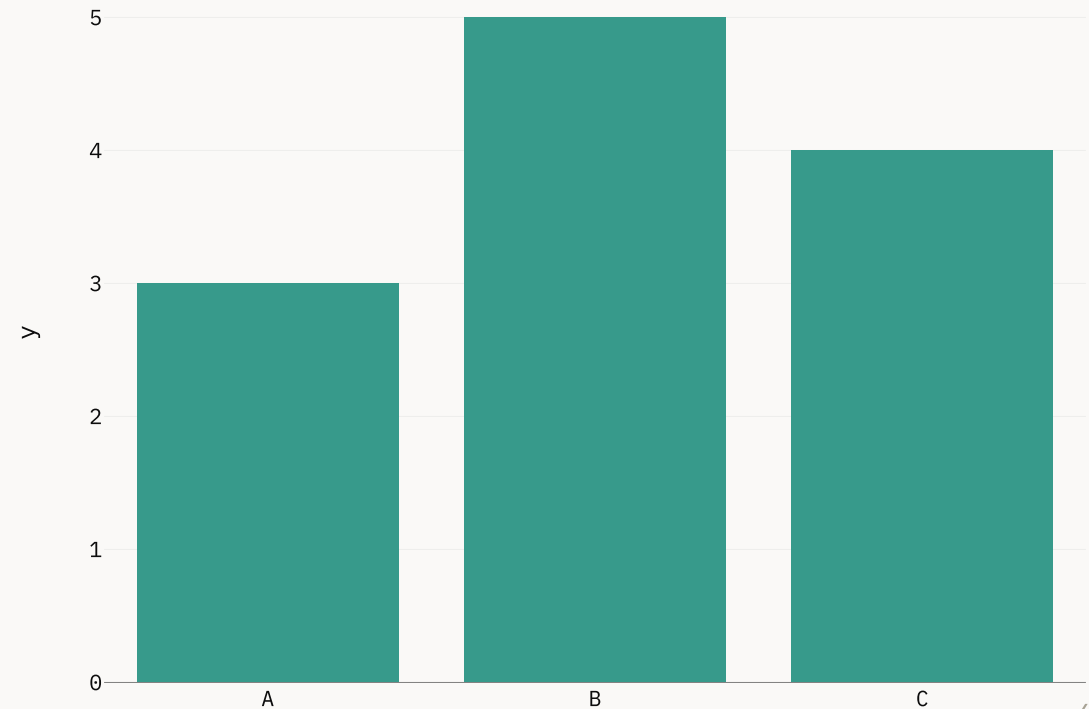
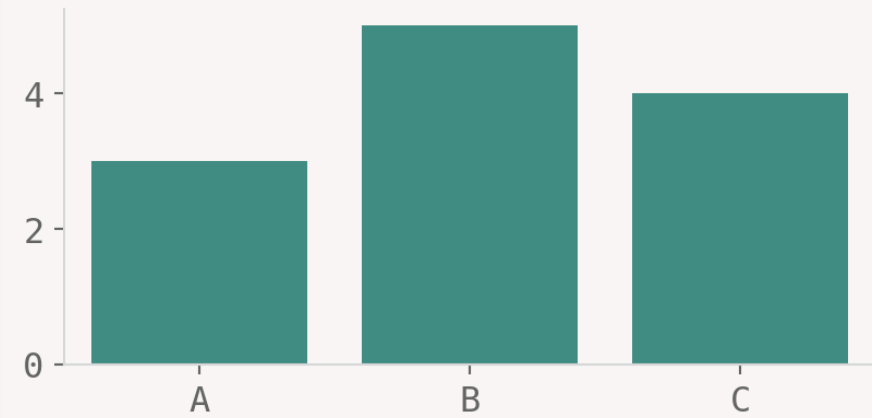
Plotly is a different engine and ignores rcParams completely. You theme it once with a *template*, which `theme.py` registers, so the single import covers all three.

```

1 import seaborn as sns, matplotlib.pyplot as plt
2 import plotly.express as px
3
4 # matplotlib + seaborn → inherit rcParams (cream canvas), no styling needed
5 fig, ax = plt.subplots(figsize=(5, 2.2))
6 sns.barplot(x=["A", "B", "C"], y=[3, 5, 4], ax=ax, color=GREEN) # seaborn rides on
7 ax.set_title("seaborn inherits the theme"); plt.show()
8
9 # plotly → needs the template (registered in theme.py)
10 px.bar(x=["A", "B", "C"], y=[3, 5, 4], template="econ").show() # matches via temp

```

seaborn inherits the theme



Anatomy of a Plot

FIRST PLOTS · THE SKELETON

Every matplotlib plot is the same five lines

Before three libraries, learn the one skeleton they all echo: **import** → **get data** → **make a figure** → **draw** → **label** → **show**. Master this and every later plot is a variation on it.

```
1 import matplotlib.pyplot as plt          # 1. the plotting library
2 from genomics_course.data import load_timecourse
3
4 tc = load_timecourse()                   # 2. get your data (gene, time, tpm, sem)
5
6 fig, ax = plt.subplots()                 # 3. fig = whole image, ax = the plot area inside
7 ax.plot(tc.time, tc.tpm)                 # 4. draw x vs y onto the axes
8 ax.set_xlabel("time"); ax.set_ylabel("TPM") # 5. label, then...
9 plt.show()                               # ...render it
```

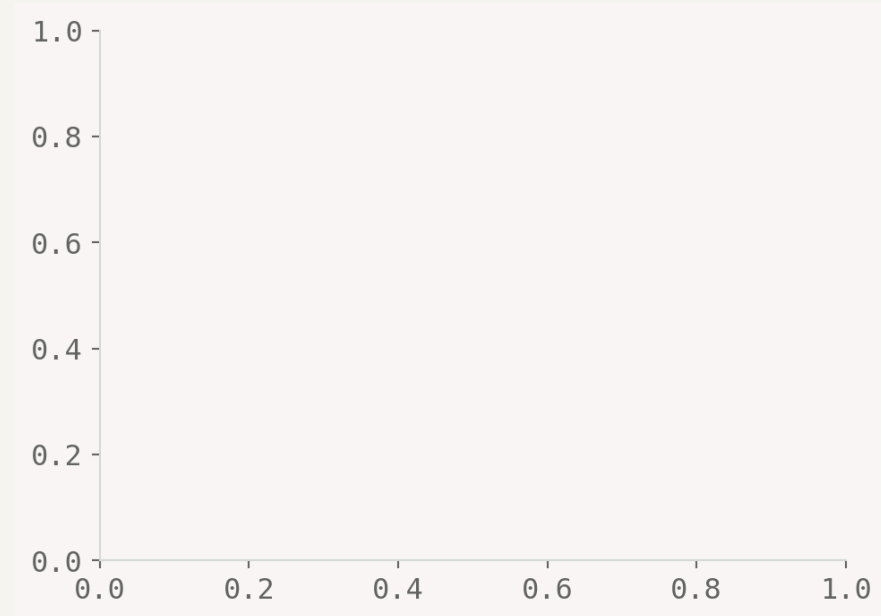
Anatomy · Step 1 — The Canvas

FIRST PLOTS · BUILD

1 · A figure and an axes

`plt.subplots()` hands back two things: the **figure** (the whole image) and the **axes** (the rectangle you draw inside). Nothing's plotted yet, it's a blank canvas.

```
1 import matplotlib.pyplot as plt
2
3 fig, ax = plt.subplots(figsize=(5.6, 4.0)) # fig = whole image, ax = the area you
4 plt.show()                               # empty for now — just the axes
```



Anatomy · Step 2 — Draw the Data

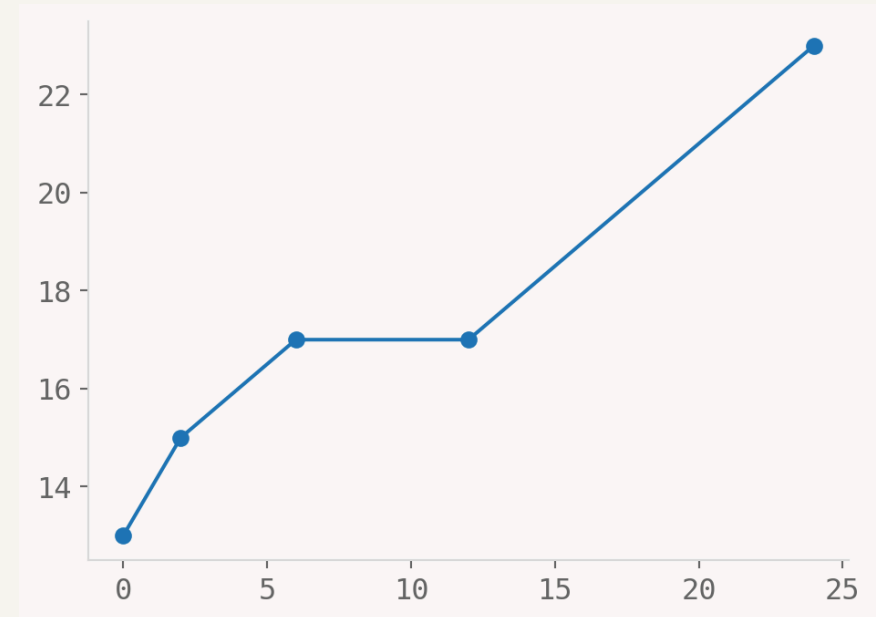
FIRST PLOTS · BUILD

2 · Put x and y on the axes

Load the data, take one gene to keep it simple, and

`ax.plot(x, y)` draws it. `marker="o"` puts a dot at each timepoint so you see the measurements.

```
1 import matplotlib.pyplot as plt
2 from genomics_course.data import load_timecourse
3
4 tc = load_timecourse()           # gene, time, tpm, sem
5 one = tc[tc.gene == tc.gene.iloc[0]] # just the first gene, to start simple
6
7 fig, ax = plt.subplots(figsize=(5.6, 4.0))
8 ax.plot(one.time, one.tpm, marker="o") # draw time (x) against expression (y)
9 plt.show()
```



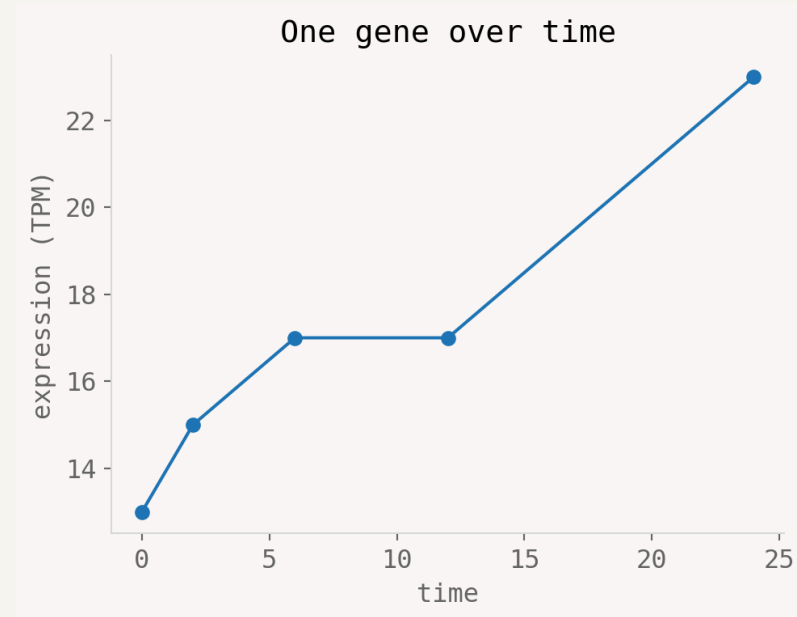
Anatomy · Step 3 — Label & Show

FIRST PLOTS · BUILD

3 · Always label; then render

An unlabelled plot is a puzzle. Name the axes, add a title that says what it shows, then `plt.show()`. That's a complete plot, and the template for everything after.

```
1 import matplotlib.pyplot as plt
2 from genomics_course.data import load_timecourse
3
4 tc = load_timecourse()
5 one = tc[tc.gene == tc.gene.iloc[0]]
6
7 fig, ax = plt.subplots(figsize=(5.6, 4.0))
8 ax.plot(one.time, one.tpm, marker="o")
9 ax.set_xlabel("time")           # what the x-axis is
10 ax.set_ylabel("expression (TPM)") # what the y-axis is
11 ax.set_title("One gene over time") # what the whole plot shows
12 plt.show()                     # render it to the slide
```



THE BOTTOM LINE

Every plot is **figure** → **axes** → **draw** → **label** → **show**. The next three slides make this exact plot in matplotlib, seaborn, and Plotly, so you'll recognise the same skeleton under each.

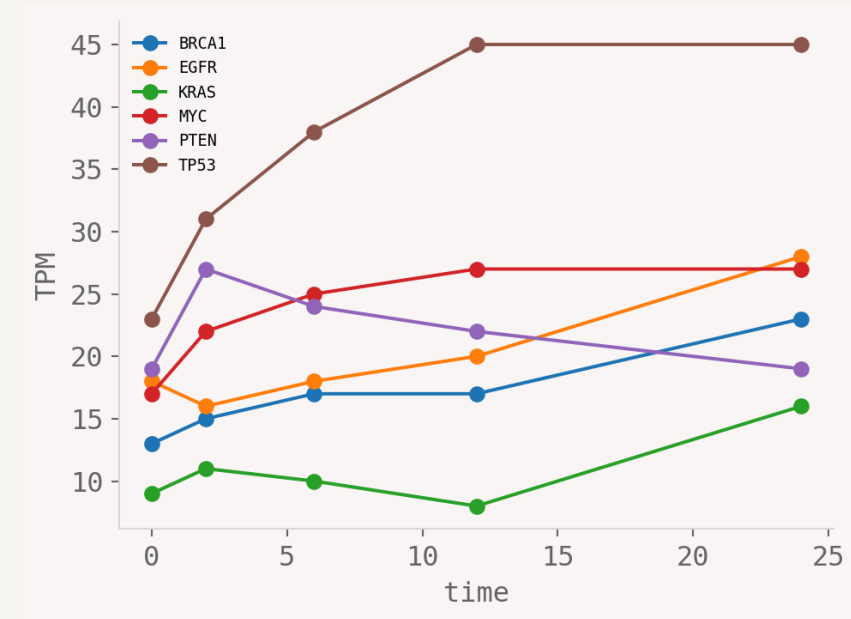
First Plots • Time Series

FIRST PLOTS • LINE

A gene's expression over time, three ways

MATPLOTLIB SEABORN PLOTLY

```
1 from genomics_course.data import load_timecourse
2
3 tc = load_timecourse()                # gene, time, tpm, sem
4 fig, ax = plt.subplots(figsize=(5.6, 4.0))
5 for g, d in tc.groupby("gene"):        # one line per gene
6     ax.plot(d.time, d.tpm, marker="o", label=g) # connect timepoints
7 ax.set_xlabel("time"); ax.set_ylabel("TPM")
8 ax.legend(fontsize=7, frameon=False)
9 plt.show()
```



Exercise • Your First Plot

FIRST PLOTS • EXERCISE

Build the skeleton yourself

CODING EXERCISE

Use the **figure → draw → label → show** skeleton on a new dataset. `load_reads_per_sample()` gives sequencing depth per sample; make a bar chart of it in whichever library you like best.

```
1 from genomics_course.data import load_reads_per_sample
2
3 rps = load_reads_per_sample()      # sample, reads
4
5 # 1. make a figure:    fig, ax = plt.subplots()
6 # 2. draw bars:       ax.bar(rps.sample, rps.reads)    (or sns.barplot / px.bar)
7 # 3. label the y-axis: "reads (millions)"
8 # 4. add a title that says what the chart shows
9 # 5. plt.show()      (or fig.show() for plotly)
```

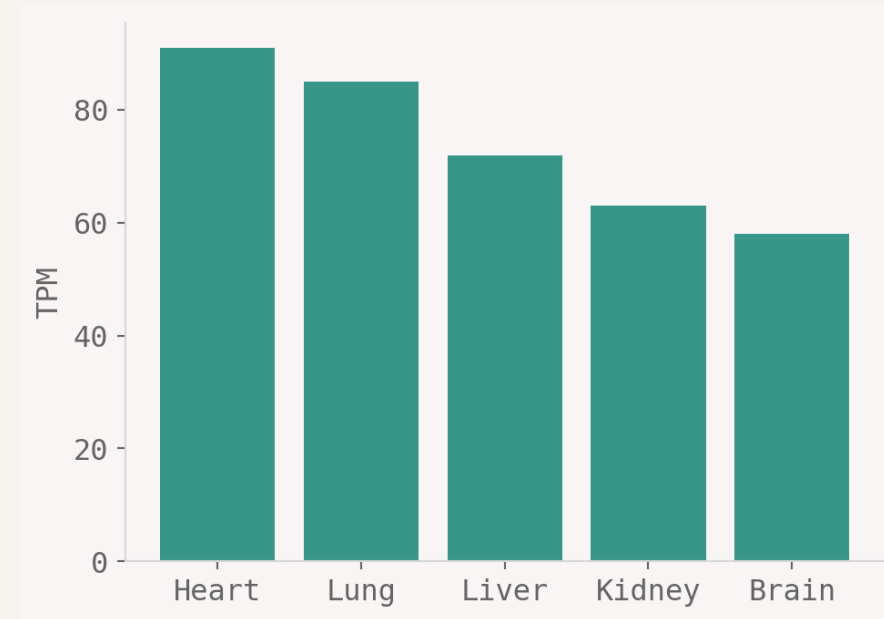
First Plots • Bar Chart

FIRST PLOTS • BAR

Expression per tissue, three ways

MATPLOTLIB SEABORN PLOTLY

```
1 from genomics_course.data import load_tissue_expression
2
3 te = load_tissue_expression()           # tissue, tpm
4 fig, ax = plt.subplots(figsize=(5.6, 4.0))
5 ax.bar(te.tissue, te.tpm, color=GREEN)  # one bar per tissue
6 ax.set_ylabel("TPM")
7 plt.show()
```



First Plots • Scatter

FIRST PLOTS • SCATTER

Co-expression of two genes, three ways

MATPLOTLIB SEABORN PLOTLY

```
1 from genomics_course.data import load_coexpression
2
3 co = load_coexpression()                # tissue, gene_x, gene_y
4 fig, ax = plt.subplots(figsize=(5.6, 4.0))
5 for t, d in co.groupby("tissue"):        # colour by tissue (loop)
6     ax.scatter(d.gene_x, d.gene_y, label=t, alpha=.7)
7 ax.set_xlabel("gene X"); ax.set_ylabel("gene Y")
8 ax.legend(fontsize=7, frameon=False)
9 plt.show()
```



Exercise • Pick Your Library

FIRST PLOTS • EXERCISE

Same data, your choice of tool

CODING EXERCISE

You've now seen all three libraries. Plot the `load_coexpression()` scatter again in the **one library you'd actually reach for**, and add a title that names the relationship. If you pick Plotly, hover a point to read its values.

```
1 from genomics_course.data import load_coexpression
2
3 co = load_coexpression()          # tissue, gene_x, gene_y
4
5 # 1. scatter gene_x vs gene_y, coloured by tissue
6 #   matplotlib (loop) • seaborn (hue=) • or plotly (color=, template="econ")
7 # 2. title it, e.g. "gene X and gene Y are co-expressed"
8 # 3. which library felt fastest for this? that's your default from now on
```

THE BOTTOM LINE

Same data, three libraries, one identity. **Matplotlib** for control, **seaborn** for one-line statistical plots (both inherit your rcParams), **Plotly** for interactivity (themed by the `econ` template). Reach for the one that fits the job; the look stays constant.

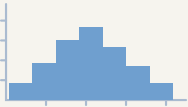
Showing Distributions

BOXPLOTS, VIOLINS & RIDGELINES · DAY 01

Choosing a Distribution Plot

SHOWING DISTRIBUTIONS · WHAT TO PLOT WHEN

Six lenses on a single variable



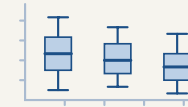
Histogram

Counts in bins. Raw shape; sensitive to bin width.



Density (KDE)

Smoothed shape. Best for overlaying a few groups.



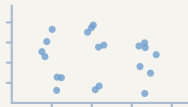
Boxplot

Five-number summary. Compact across many groups.



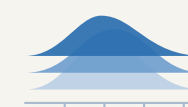
Violin

Density mirrored. Shape + summary in one footprint.



Strip / sina

Every point shown. Honest when n is small.



Ridgeline

Distributions stacked across groups or time.

RULE OF THUMB

Small n → strip · few groups → density or histogram · many groups → boxplot or ridgeline · need shape and summary → violin.

When & Why a Boxplot

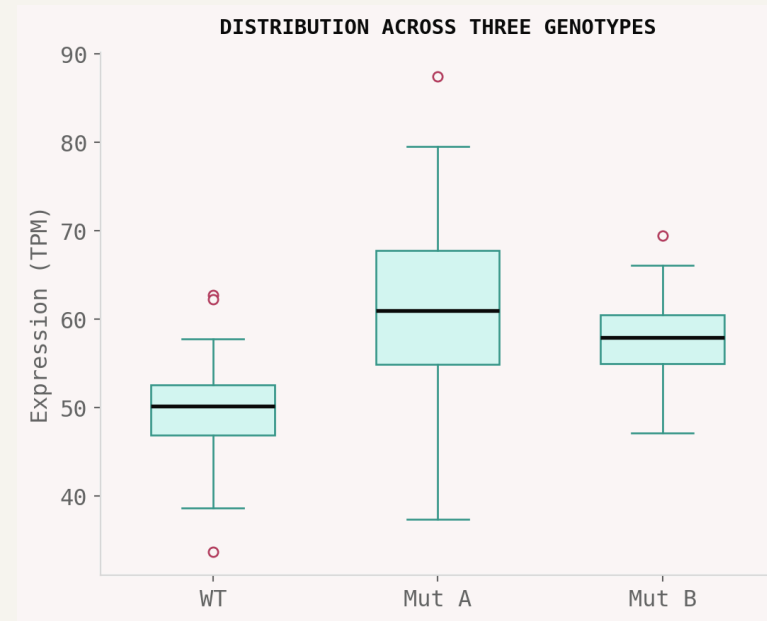
DISTRIBUTIONS • WHEN TO USE

A boxplot summarises spread, pick the chart that fits your question

Use a boxplot to compare the **distribution** of a continuous value across a few groups — its centre, spread, and outliers — without plotting every point.

Spread is not precision. SD describes how the data scatter around the mean; SE describes how precise your estimate of the mean is. A boxplot answers the first question; error bars on a mean answer the second.

```
1 r = np.random.default_rng(1)
2 groups = ["WT", "Mut A", "Mut B"]
3 data = [r.normal(50, 6, 70),           # WT
4         r.normal(62, 10, 70),         # Mut A - bigger spread
5         r.normal(58, 4, 70)]          # Mut B - tighter
```



SD vs SE: spread of the **data** vs precision of the **estimate**. Same numbers, different question, different chart. (Wilke 2019, Ch. 9)

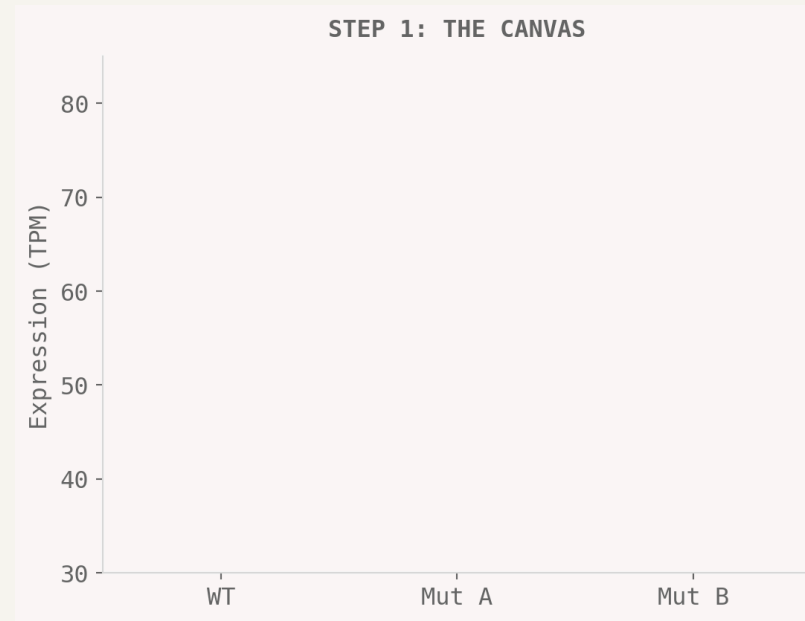
Building a Boxplot • Step 1: the canvas

DISTRIBUTIONS • BOXPLOT BUILD

Set up the figure, labels, and scale before any data

Start with an empty, labelled canvas: axes, group positions, y-scale. Decide the frame first, fill it after.

```
1 groups = ["WT", "Mut A", "Mut B"]
2 fig, ax = plt.subplots(figsize=(6.4, 4.7))
3 ax.set_ylabel("Expression (TPM)")
4 ax.set_xticks([1, 2, 3])
5 ax.set_xticklabels(groups)
6 ax.set_xlim(0.5, 3.5)
7 ax.set_ylim(30, 85)
```



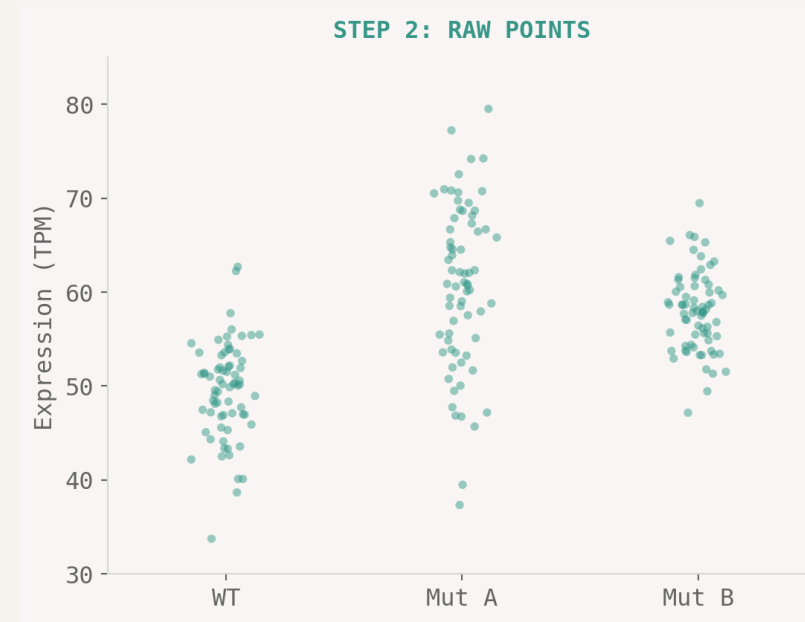
Building a Boxplot • Step 2: the raw points

DISTRIBUTIONS • BOXPLOT BUILD

Look at the data first, with a little horizontal jitter

Plot every observation, jittering x-position so points don't stack on one line. This shows the sample size and shape that a summary will later hide.

```
1 # data: list of three np.ndarrays, one per group
2 rj = np.random.default_rng(2)
3 for i, d in enumerate(data, start=1):
4     x = rj.normal(i, 0.06, len(d))          # gentle jitter
5     ax.scatter(x, d,
6               color=GREEN, s=18,
7               alpha=0.5, edgecolors="none")
```



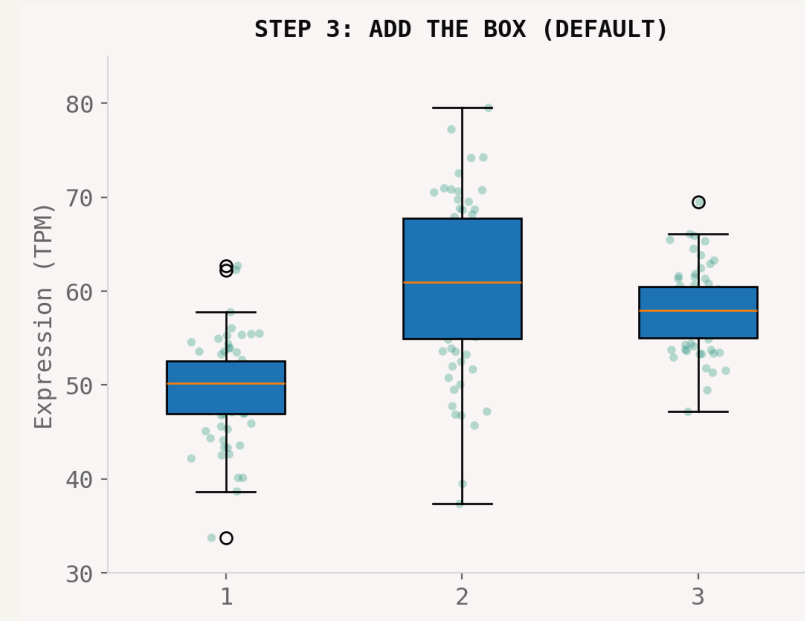
Building a Boxplot • Step 3: add the box

DISTRIBUTIONS • BOXPLOT BUILD

Overlay the five-number summary on top of the points

The box draws Q1 → Q3, the median line, and whiskers to the most extreme points within $1.5 \times \text{IQR}$. Matplotlib computes all of it for you.

```
1 bp = ax.boxplot(  
2     data,  
3     positions=[1, 2, 3],  
4     widths=0.5,  
5     patch_artist=True,      # so we can colour it  
6 )
```



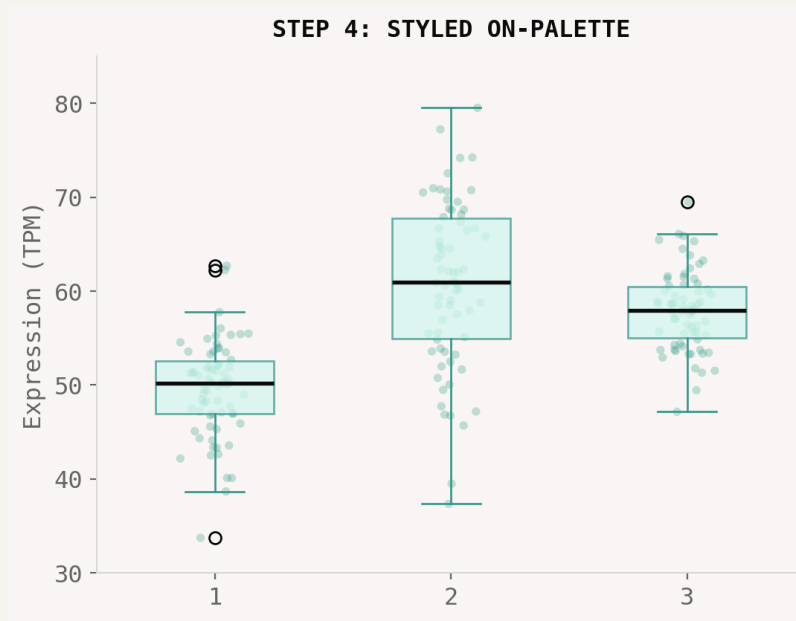
Building a Boxplot • Step 4: style on-palette

DISTRIBUTIONS • BOXPLOT BUILD

Economist green box, ink median, fade the points behind

Style every element: a soft teal fill, a green edge, a bold ink median, green whiskers. Drop the alpha on the points so the box reads on top.

```
1 bp = ax.boxplot(data, positions=[1,2,3], widths=0.5, patch_artist=True)
2 for p in bp["boxes"]:
3     p.set_facecolor(ECON["hongkong"][3])
4     p.set_edgecolor(GREEN); p.set_alpha(0.75)
5 for m in bp["medians"]:
6     m.set_color(INK); m.set_linewidth(2)
7 for w in bp["whiskers"] + bp["caps"]:
8     w.set_color(GREEN)
```



Building a Boxplot • Step 5: the message

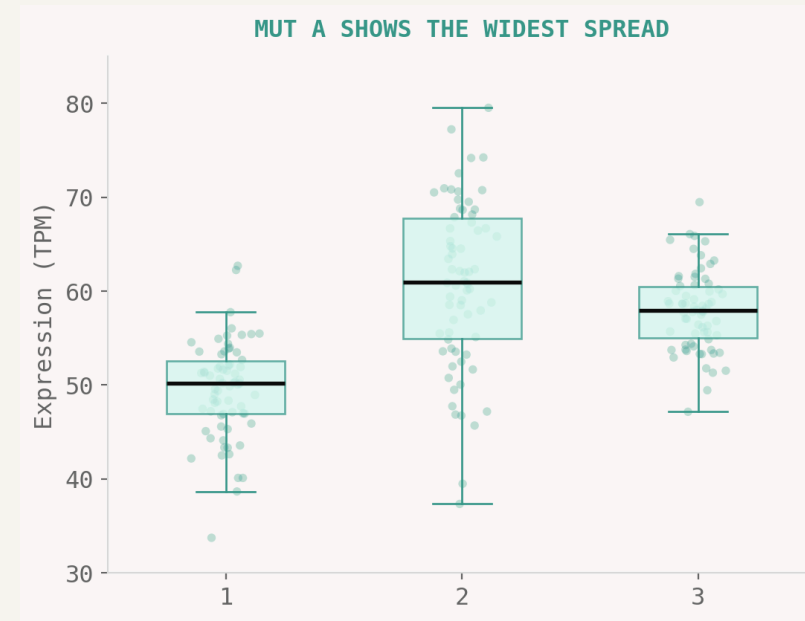
DISTRIBUTIONS • BOXPLOT BUILD

Hide duplicate outliers, title the finding

The points already show every value, so turn off the default flier markers. Finish with a title that says what the reader should take away.

```
1 bp = ax.boxplot(data, showfliers=False, ...)  
2 ax.set_title("MUT A SHOWS THE WIDEST SPREAD")
```

That's the whole recipe: **canvas** → **points** → **box** → **style**
→ **message**.



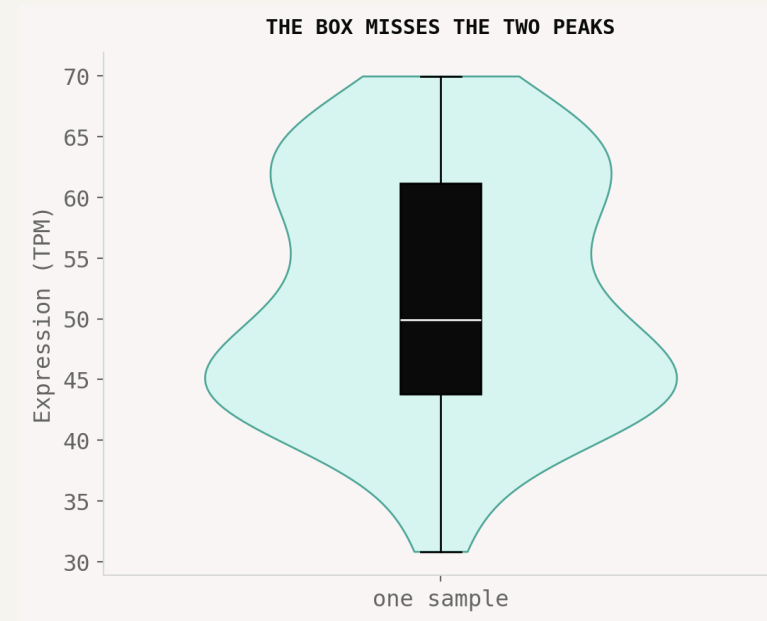
When & Why a Violin

DISTRIBUTIONS • WHEN TO USE

Reach for a violin when the shape matters, not just the summary

A boxplot hides shape — two very different distributions can share the same five-number summary. A violin draws a **mirrored density**, so a second hump or a long skew is visible at a glance.

Use it when the distribution might be **bimodal** or strongly skewed. Use a plain boxplot when a clean five-number summary is all you need.



The boxplot shows the summary. The violin shows the **shape**. Bimodality is the story the box can't tell. (Wilke 2019, Ch. 7)

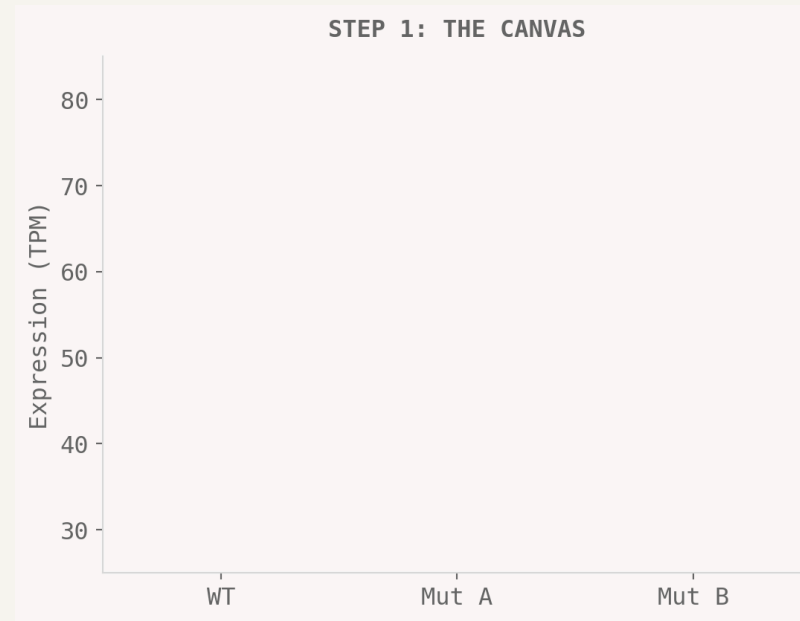
Building a Violin • Step 1: the canvas

DISTRIBUTIONS • VIOLIN BUILD

Same canvas as the boxplot, three groups, one deliberately bimodal

The recipe starts identically: an empty, labelled frame.
One group is deliberately **bimodal** so the violin has something a box would hide.

```
1 r = np.random.default_rng(3)
2 groups = ["WT", "Mut A", "Mut B"]
3 data = [np.concatenate([r.normal(45, 5, 45),
4                          r.normal(62, 5, 35)]), # WT is bimodal
5         r.normal(62, 10, 80),
6         r.normal(58, 4, 80)]
7 fig, ax = plt.subplots(figsize=(6.4, 4.7))
```



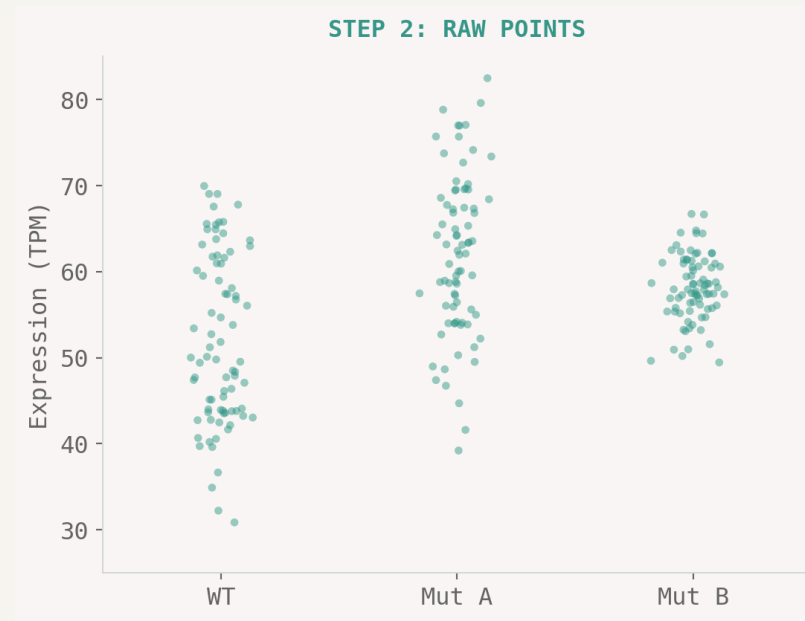
Building a Violin • Step 2: the raw points

DISTRIBUTIONS • VIOLIN BUILD

Jittered points already hint at the bimodality

The WT group sits in two clouds, not one. The violin will make that explicit.

```
1  rj = np.random.default_rng(4)
2  for i, d in enumerate(data, start=1):
3      ax.scatter(rj.normal(i, 0.06, len(d)), d,
4                 color=GREEN, s=16, alpha=0.5,
5                 edgecolors="none")
```



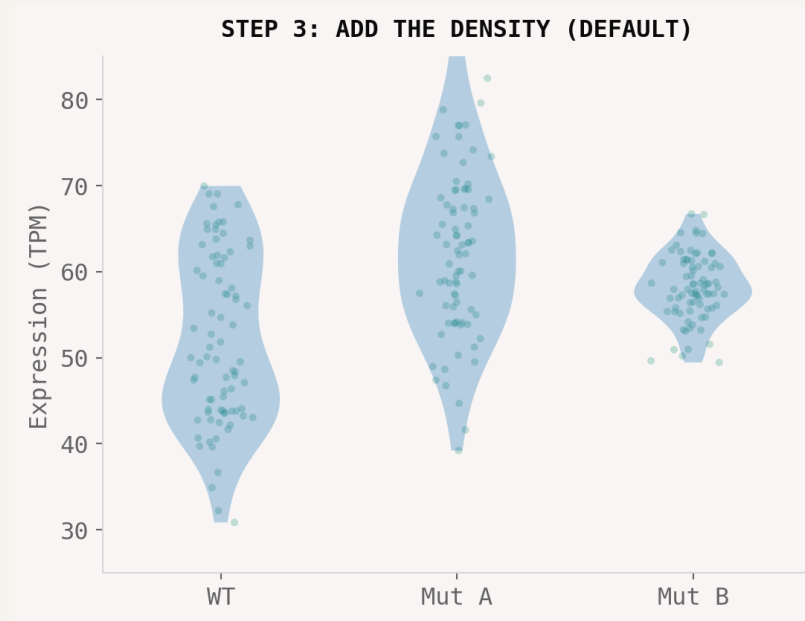
Building a Violin · Step 3: add the density

DISTRIBUTIONS · VIOLIN BUILD

violinplot draws a kernel density and mirrors it about each axis

The body width at any height is the point density there.
The WT violin now shows two bulges, the bimodality a box would flatten away.

```
1 vp = ax.violinplot(  
2     data,  
3     positions=[1, 2, 3],  
4     showextrema=False,  
5 )
```



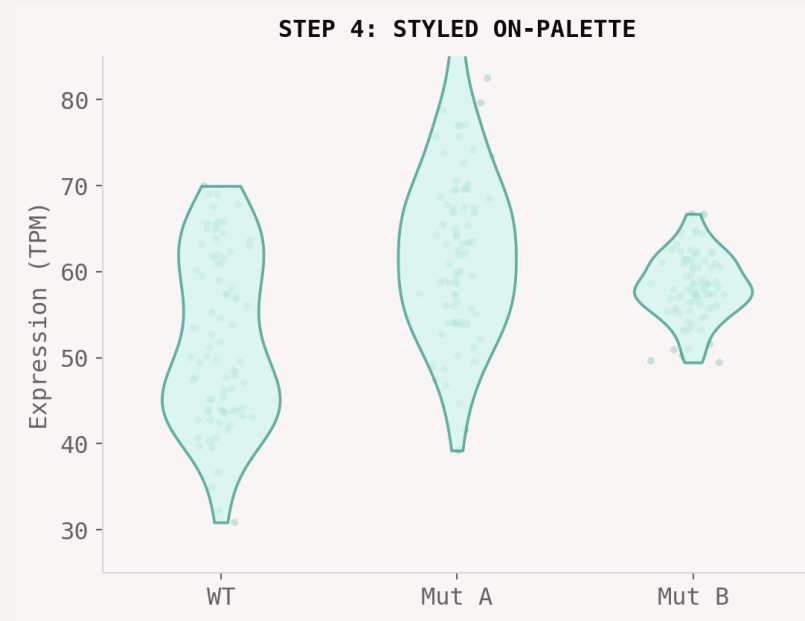
Building a Violin • Step 4: style on-palette

DISTRIBUTIONS • VIOLIN BUILD

Teal body, green edge, fade the points behind

Loop over the violin bodies and colour them on-brand, then drop the points' alpha so the density reads on top.

```
1 vp = ax.violinplot(data, positions=[1,2,3], showextrema=False)
2 for b in vp["bodies"]:
3     b.set_facecolor(ECON["hongkong"][3])
4     b.set_edgecolor(GREEN)
5     b.set_alpha(0.75)
6     b.set_linewidth(1.4)
```



Building a Violin · Step 5: the message

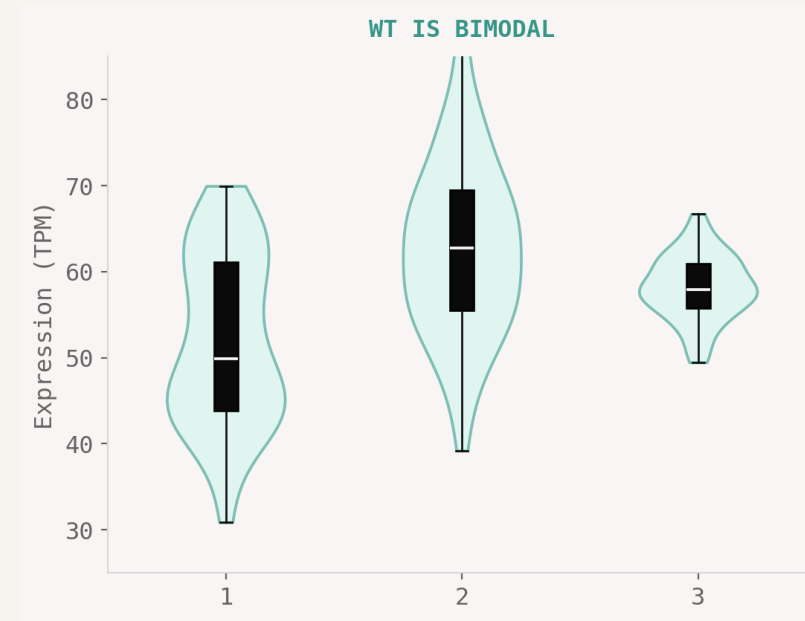
DISTRIBUTIONS · VIOLIN BUILD

Slim inner boxplot for the summary, title the finding

A thin boxplot inside each violin gives the five-number summary and the shape together. Finish with the takeaway in the title.

```
1 for i, d in enumerate(data, start=1):
2     bx = ax.boxplot(d, positions=[i],
3                     widths=0.1, patch_artist=True,
4                     showfliers=False)
5     bx["boxes"][0].set_facecolor(INK)
6     bx["medians"][0].set_color(CREAM)
7 ax.set_title("WT IS BIMODAL")
```

Same recipe as the box: **canvas** → **points** → **density** → **style** → **message**.



Box vs violin: the box gives the **summary**, the violin reveals the **shape**. Show the violin when bimodality or skew is the story.

Ridgeline • The Data

DISTRIBUTIONS • RIDGELINE

2,000 single cells, ordered along a trajectory

`load_pseudotime()` is single-cell data in tidy long form:
one row per cell. Each cell has a pseudotime `stage` (1 = earliest, 8 = latest along the differentiation path) and the marker gene's `expression`. With 250 cells per stage, every stage is a proper distribution, not a single number.

(2000, 3)

```
1 from genomics_course.data import load_pseudotime
2
3 pt = load_pseudotime()      # cell, stage, expression
4 print(pt.shape)             # 2000 cells × 3 columns
5 pt.head(4)
```

	cell	stage	expression
0	S1_000	1	1.607084
1	S1_001	1	1.846248
2	S1_002	1	1.994292
3	S1_003	1	1.623597

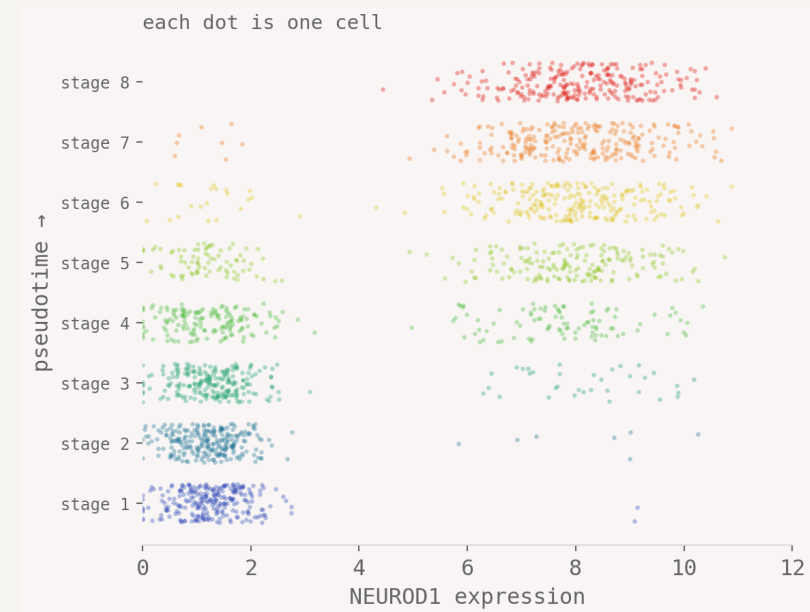
Ridgeline • See the Cells First

DISTRIBUTIONS • RIDGELINE

Before any smoothing: one dot per cell

Plot every cell as a dot, expression on the x-axis, stage on the y-axis with a little jitter. Early stages sit in a low cloud (gene OFF); late stages sit in a high cloud (gene ON); the middle stages show **both clouds at once**, that's the bimodal hand-off. The ridgeline next just replaces each row of dots with a smooth density.

```
1 from genomics_course.data import load_pseudotime
2
3 pt = load_pseudotime()           # 2000 cells: cell, stage
4 stages = sorted(pt.stage.unique()) # ordered pseudotime stag
5 n = len(stages)
6 rng = np.random.default_rng(0)   # reproducible jitter
7
8 fig, ax = plt.subplots(figsize=(6.4, 5.0))
9 for i, s in enumerate(stages):
10     vals = pt.loc[pt.stage == s, "expression"].values # one row of dots per sta
11     y = i + rng.uniform(-0.32, 0.32, len(vals))      # this stage's cells
12     ax.scatter(vals, y, s=6, color=econ_spectrum(i / (n - 1)), # jitter so dots don't pi
13               alpha=.35, linewidths=0)               # x = expression, on
14 ax.set_yticks(range(n)); ax.set_yticklabels([f"stage {s}" for s in stages], fontsize
15 ax.set_xlabel("NEUROD1 expression"); ax.set_ylabel("pseudotime →")
16 ax.set_xlim(0, 12)
17 for sp in ["top", "right", "left"]: # bare frame, same look a
18     ax.spines[sp].set_visible(False)
19 ax.set_title("each dot is one cell", color=MUTED, fontsize=11, loc="left")
20 plt.show()
```



Ridgelines

MANY DISTRIBUTIONS, ONE PANEL • DAY 01

When a Ridgeline

DISTRIBUTIONS • THEORY

Watch a distribution change across an ordered factor

A ridgeline stacks one density per group, slightly overlapping, so you compare many distributions in a compact panel. It earns its place when the groups have a **meaningful order** and the distribution *shifts* across it: expression along pseudotime, a signal across tissues, a metric over time. The **shape** is the story, a mode appearing, a peak sliding right.

Two requirements keep it honest: enough points per group for a smooth density (a KDE on twenty values is noise), and a colour ramp **keyed to the group order** so the eye follows the progression instead of decoding a rainbow. Here: a neural marker switching on as cells differentiate.

Ridgeline • The Data

DISTRIBUTIONS • RIDGELINE

A marker gene across single cells, ordered by pseudotime

`load_pseudotime()` is tidy long: one row per cell, its pseudotime `stage` (1 = earliest), and the marker's `expression`. Each stage is a distribution we'll turn into one ridge.

```
1 from genomics_course.data import load_pseudotime
2
3 pt = load_pseudotime() # cell, stage, expression
4 pt.head(4)
```

	cell	stage	expression
0	S1_000	1	1.607084
1	S1_001	1	1.846248
2	S1_002	1	1.994292
3	S1_003	1	1.623597

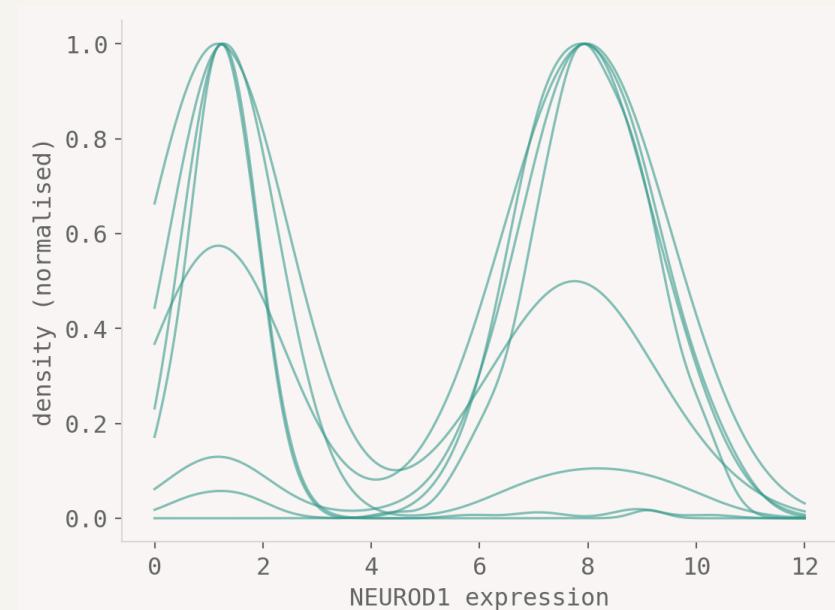
Ridgeline · Step 1 — A Density Per Stage

DISTRIBUTIONS · BUILD

1 • KDE each group

Smooth each stage's expression into a density with `gaussian_kde`, on a shared x-grid, and normalise to the same height so shapes are comparable. Overlaid, you can already see a second hump appear, but they collide.

```
1 from scipy.stats import gaussian_kde          # kernel density estimator
2 from genomics_course.data import load_pseudotime
3
4 pt = load_pseudotime()                       # cell, stage, expression
5 stages = sorted(pt.stage.unique())           # ordered pseudotime stages
6 xs = np.linspace(0, 12, 300)                 # shared grid for every d
7
8 fig, ax = plt.subplots(figsize=(6.4, 4.7))    # one panel
9 for s in stages:                              # one KDE per stage
10     vals = pt.loc[pt.stage == s, "expression"].values # this stage's cells
11     k = gaussian_kde(vals)(xs)                # smooth density over xs
12     ax.plot(xs, k / k.max(), color=GREEN, lw=1.2, alpha=.6) # normalise height; all
13 ax.set_xlabel("NEUROD1 expression"); ax.set_ylabel("density (normalised)")
14 plt.show()
```



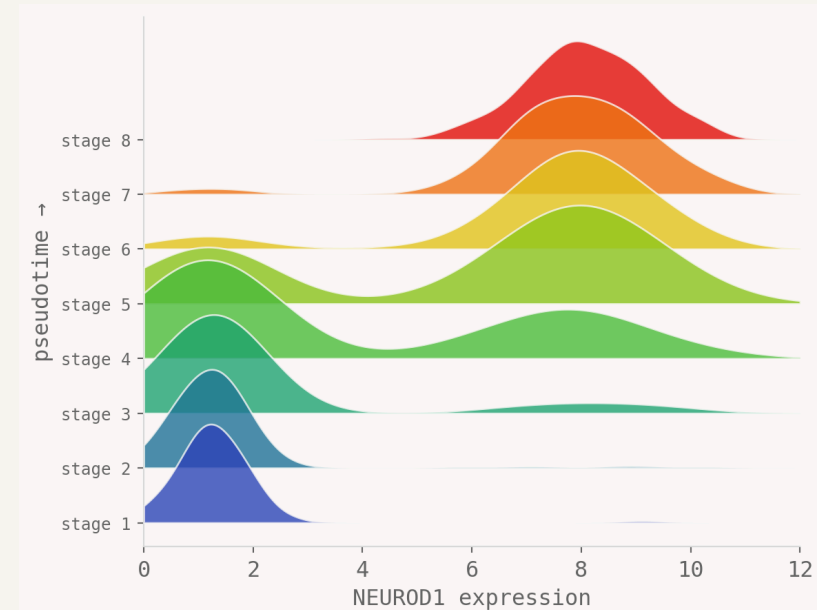
Ridgeline · Step 2 — Stack the Ridges

DISTRIBUTIONS · BUILD

2 · Offset and fill

Give each density its own baseline `y = i` and `fill_between` it, scaled by `overlap` (>1 so neighbours touch). Colour by stage along `econ_spectrum`, and the `zorder` puts lower ridges in front. Now they read as a stack.

```
1 from scipy.stats import gaussian_kde
2 from genomics_course.data import load_pseudotime
3
4 pt = load_pseudotime()           # cell, stage, expression
5 stages = sorted(pt.stage.unique()) # ordered stages
6 xs = np.linspace(0, 12, 300)     # shared grid
7 n = len(stages); overlap = 1.8   # ridge height (>1 → ridg
8
9 fig, ax = plt.subplots(figsize=(6.4, 5.2))
10 for i, s in enumerate(stages):    # stage 1 at the bottom,
11     vals = pt.loc[pt.stage == s, "expression"].values # this stage's cells
12     k = gaussian_kde(vals)(xs); k = k / k.max() * overlap # normalise, then scale t
13     col = econ_spectrum(i / (n - 1)) # colour keyed to pseudot
14     ax.fill_between(xs, i + k, i, color=col, alpha=.8, # ridge sits on baseline
15                     lw=1, edgecolor=CREAM, zorder=n - i) # lower ridges drawn in
16 ax.set_yticks(range(n)); ax.set_yticklabels([f"stage {s}" for s in stages], fontsize
17 ax.set_xlabel("NEUROD1 expression"); ax.set_ylabel("pseudotime →")
18 ax.set_xlim(0, 12)
19 plt.show()
```



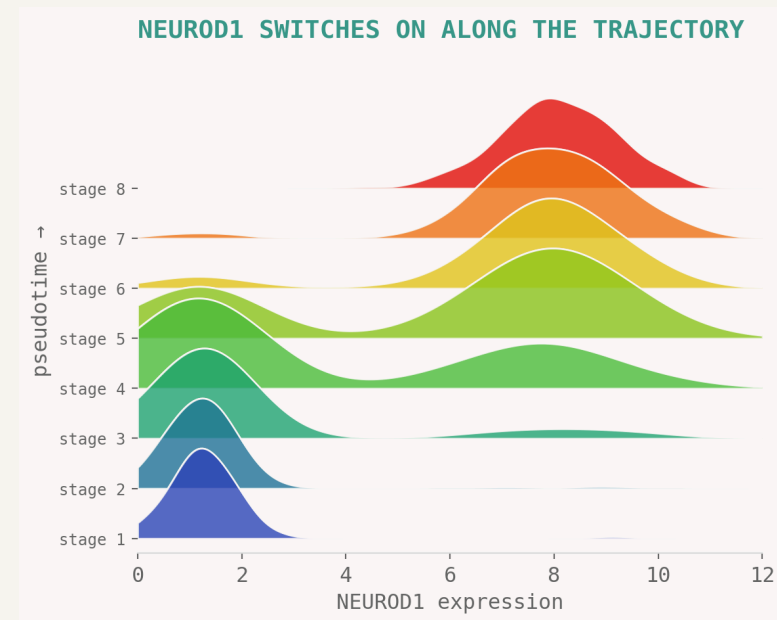
Ridgeline · Step 3 — The Message

DISTRIBUTIONS · BUILD

3 · Clean edge, no frame, name the finding

Trace each ridge's top in cream so overlaps stay crisp, drop the frame (ridgelines read better bare), and title with the result. The OFF→bimodal→ON march is the whole point.

```
1 from scipy.stats import gaussian_kde
2 from genomics_course.data import load_pseudotime
3
4 pt = load_pseudotime()           # cell, stage, expression
5 stages = sorted(pt.stage.unique()) # ordered stages
6 xs = np.linspace(0, 12, 300)     # shared grid
7 n = len(stages); overlap = 1.8   # ridge height
8
9 fig, ax = plt.subplots(figsize=(6.4, 5.2))
10 for i, s in enumerate(stages):    # bottom → top
11     vals = pt.loc[pt.stage == s, "expression"].values # this stage's cells
12     k = gaussian_kde(vals)(xs); k = k / k.max() * overlap # normalised, scaled density
13     col = econ_spectrum(i / (n - 1)) # spectrum keyed to order
14     ax.fill_between(xs, i + k, i, color=col, alpha=.8, lw=1, edgecolor=CREAM, zorder=n - i)
15     ax.plot(xs, i + k, color=CREAM, lw=.8, zorder=n - i) # crisp cream top edge
16 ax.set_yticks(range(n)); ax.set_yticklabels([f"stage {s}" for s in stages], fontsize=10)
17 ax.set_xlabel("NEUROD1 expression"); ax.set_ylabel("pseudotime ↑")
18 ax.set_xlim(0, 12); ax.set_ylim(-0.3, n + overlap) # headroom for the top
19 for sp in ["top", "right", "left"]: # ridgelines read better
20     ax.spines[sp].set_visible(False)
21 ax.set_title("NEUROD1 SWITCHES ON ALONG THE TRAJECTORY", # title states the finding
22             color=GREEN, fontweight="bold", loc="left")
```



THE BOTTOM LINE

A ridgeline is many densities stacked along an ordered factor: **KDE each group → offset and fill → ramp keyed to order**. Watch the marker switch on, a single OFF hump, a bimodal transition where some cells flip, then a single ON hump. The shift is the result.

Exercise • Library QC

DISTRIBUTIONS • EXERCISE

Are the libraries comparable?

REAL SCENARIO

Before differential expression, you check that every sample's expression distribution looks alike, a ridge sitting clearly left or right of the rest flags a library that needs normalising. Melt `load_expression()` to long and build one ridge per sample, coloured by condition.

```
1 from scipy.stats import gaussian_kde
2 from genomics_course.data import load_expression
3
4 expr = load_expression()           # genes × samples
5 long = expr.melt(var_name="sample", value_name="expr")  # one row per gene-sample
6
7 # 1. xs over the expression range; samples = expr.columns
8 # 2. for each sample i: k = gaussian_kde(values); fill_between(xs, i + k/k.max(), i)
9 # 3. colour by condition: BLUE if name starts with "C", AMBER if "D"
10 # 4. read it: does any sample's ridge sit offset from the rest? → normalise before DE
```

Building a Plot

A SCATTER, STEP BY STEP · DAY 01

Building a Plot • The Data

THE PYTHON VIZ STACK • WHAT WE'RE PLOTTING

How the data loads

`load_coexpression()` returns one row per sample: two gene expression values plus the tissue it came from. Three tissues, Leaf / Root / Seed.

```
1 from genomics_course.data import load_coexpression
2
3 co = load_coexpression()      # sample, tissue, gene_x, gene_y
4 co.head(3)
```

	sample	tissue	gene_x	gene_y
0	Leaf_7110	Leaf	3.715459	4.780508
1	Leaf_9330	Leaf	5.334575	6.522094
2	Leaf_8607	Leaf	8.461562	12.642475

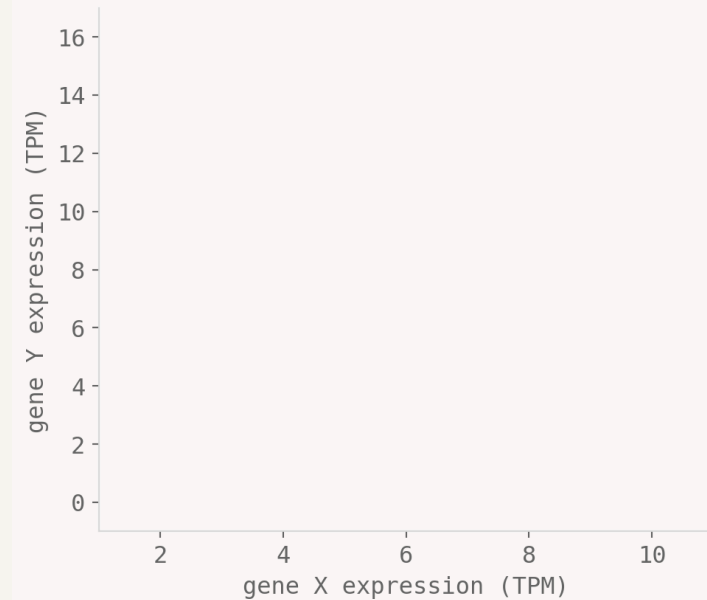
Building a Plot • Step 1 — The Canvas

THE PYTHON VIZ STACK • BUILDING A FIGURE

1 • Canvas

We'll build a **scatter**: gene Y against gene X, one dot per sample. Start with a **figure** and an **axes**, then frame the axes before drawing anything.

```
1 import matplotlib.pyplot as plt
2
3 fig, ax = plt.subplots(figsize=(5.6, 4.8)) # one figure, one axes
4 ax.set_xlabel("gene X expression (TPM)")    # frame the axes before drawing
5 ax.set_ylabel("gene Y expression (TPM)")
6 ax.set_xlim(1, 11); ax.set_ylim(-1, 17)
7 ax.spines[["top", "right"]].set_visible(False)
8 plt.show()
```



Building a Plot • Step 2 — Load & Plot Data

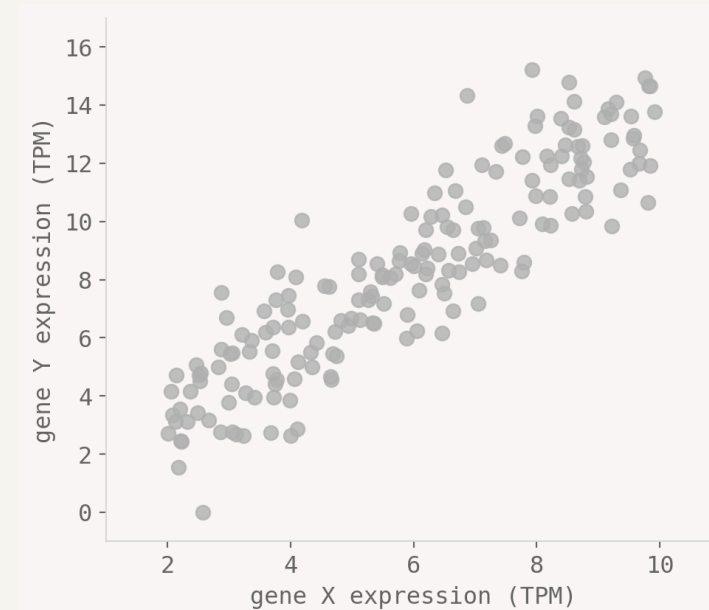
THE PYTHON VIZ STACK • BUILDING A FIGURE

2 • Data

Load the samples and drop them onto the canvas with

`ax.scatter()`. One dot per sample, default colour for now.

```
1 import matplotlib.pyplot as plt
2 from genomics_course.data import load_coexpression
3
4 co = load_coexpression()           # sample, tissue, gene_x, gene_y
5
6 fig, ax = plt.subplots(figsize=(5.6, 4.8))
7 ax.set_xlabel("gene X expression (TPM)")
8 ax.set_ylabel("gene Y expression (TPM)")
9 ax.set_xlim(1, 11); ax.set_ylim(-1, 17)
10 ax.scatter(co.gene_x, co.gene_y, color=GREY, s=50, alpha=.8) # one dot per sample
11 ax.spines[["top", "right"]].set_visible(False)
12 plt.show()
```



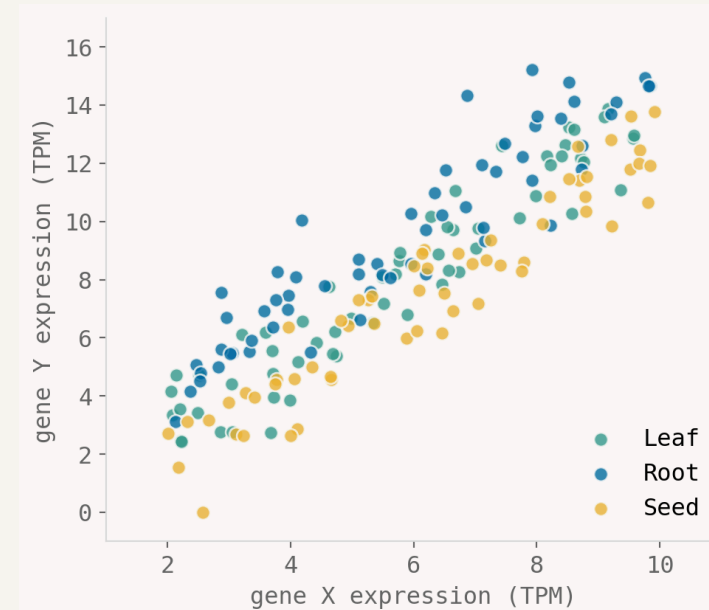
Building a Plot · Step 3 — Colour by Tissue

THE PYTHON VIZ STACK · BUILDING A FIGURE

3 · Colour

Loop over the tissues, one pass each, one palette colour each. Colour now carries real signal, the tissues separate.

```
1 import matplotlib.pyplot as plt
2 from genomics_course.data import load_coexpression
3
4 co = load_coexpression()
5
6 fig, ax = plt.subplots(figsize=(5.6, 4.8))
7 ax.set_xlabel("gene X expression (TPM)")
8 ax.set_ylabel("gene Y expression (TPM)")
9 ax.set_xlim(1, 11); ax.set_ylim(-1, 17)
10 pal = {"Leaf": GREEN, "Root": BLUE, "Seed": AMBER} # one colour per tissue
11 for tissue, c in pal.items():
12     g = co[co.tissue == tissue]
13     ax.scatter(g.gene_x, g.gene_y, color=c, s=50, alpha=.8, edgecolors=CREAM, label=
14 ax.legend(frameon=False, loc="lower right") # name the tissues
15 ax.spines[["top", "right"]].set_visible(False)
16 plt.show()
```



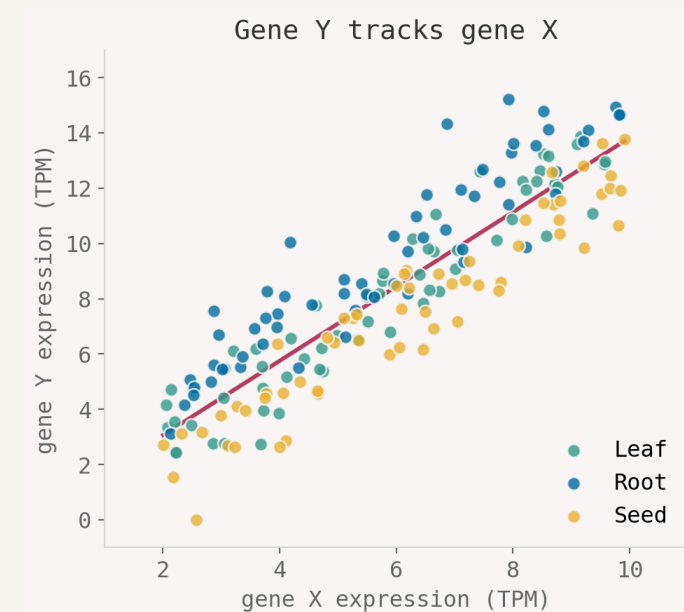
Building a Plot • Step 4 — Polish & Labels

THE PYTHON VIZ STACK • BUILDING A FIGURE

4 • Polish

A single linear fit across all samples summarises the trend. Compute slope and intercept, draw it in red, and let the title state what we found.

```
1 import matplotlib.pyplot as plt
2 from genomics_course.data import load_coexpression
3
4 co = load_coexpression()
5
6 fig, ax = plt.subplots(figsize=(5.6, 4.8))
7 ax.set_xlabel("gene X expression (TPM)")
8 ax.set_ylabel("gene Y expression (TPM)")
9 ax.set_xlim(1, 11); ax.set_ylim(-1, 17)
10 pal = {"Leaf": GREEN, "Root": BLUE, "Seed": AMBER}
11 for tissue, c in pal.items():
12     g = co[co.tissue == tissue]
13     ax.scatter(g.gene_x, g.gene_y, color=c, s=50, alpha=.8, edgecolors=CREAM, label=
14 m, b = np.polyfit(co.gene_x, co.gene_y, 1)          # single linear fit
15 xs = np.array([co.gene_x.min(), co.gene_x.max()])
16 ax.plot(xs, m * xs + b, color=RED, lw=2.2, zorder=1) # draw it in red
17 ax.set_title("Gene Y tracks gene X", color="#333333") # state the finding
18 ax.legend(frameon=False, loc="lower right")
19 ax.spines[["top", "right"]].set_visible(False)
20 plt.show()
```



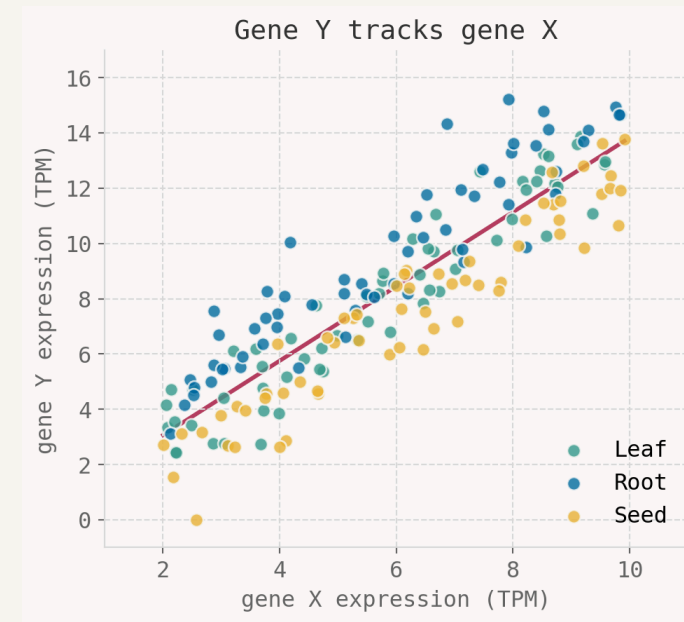
Building a Plot · Step 5 — Style Once, Reuse Everywhere

THE PYTHON VIZ STACK · BUILDING A FIGURE

5 · House style

Bundle your spine and grid choices into one block and apply it. The per-axes spine code disappears, every plot inherits the same look.

```
1 import matplotlib.pyplot as plt
2 from genomics_course.data import load_coexpression
3
4 co = load_coexpression()
5
6 house = {                                # the house style, defined ONCE
7     "axes.spines.top":    False,
8     "axes.spines.right":  False,
9     "axes.grid":          True,
10    "grid.linestyle":      "--",
11    "grid.color":          LINE,
12 }
13 with plt.rc_context(house):              # apply it: no per-axes spine code
14     fig, ax = plt.subplots(figsize=(5.6, 4.8))
15     ax.set_xlabel("gene X expression (TPM)")
16     ax.set_ylabel("gene Y expression (TPM)")
17     ax.set_xlim(1, 11); ax.set_ylim(-1, 17)
18     pal = {"Leaf": GREEN, "Root": BLUE, "Seed": AMBER}
19     for tissue, c in pal.items():
20         g = co[co.tissue == tissue]
21         ax.scatter(g.gene_x, g.gene_y, color=c, s=50, alpha=.8, edgecolors=CREAM,
22                 m_h = np.polyfit(co.gene_x, co.gene_y, 1))
```



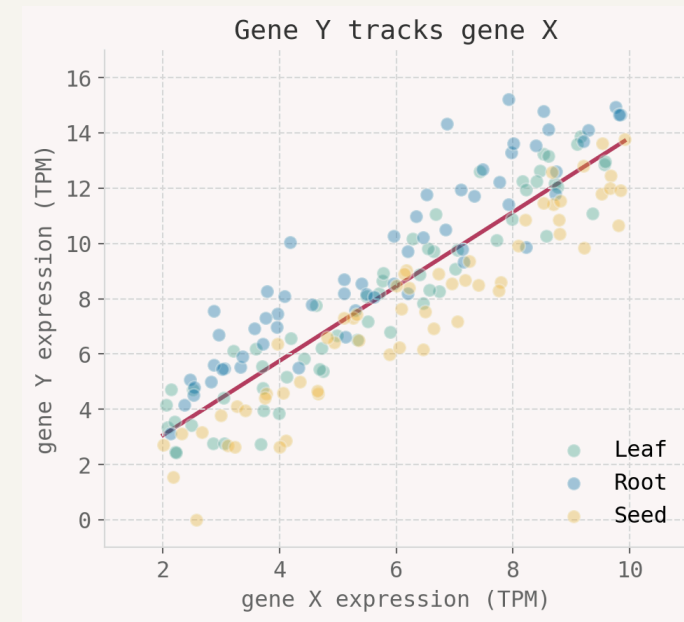
Building a Plot · Step 6 — Transparency

THE PYTHON VIZ STACK · BUILDING A FIGURE

6 · Alpha

Lower the dot opacity. Where samples pile up along the line the colour deepens, so density reads instead of one solid mass.

```
1 import matplotlib.pyplot as plt
2 from genomics_course.data import load_coexpression
3
4 co = load_coexpression()
5
6 house = {
7     "axes.spines.top": False,
8     "axes.spines.right": False,
9     "axes.grid": True,
10    "grid.linestyle": "--",
11    "grid.color": LINE,
12 }
13 with plt.rc_context(house):
14     fig, ax = plt.subplots(figsize=(5.6, 4.8))
15     ax.set_xlabel("gene X expression (TPM)")
16     ax.set_ylabel("gene Y expression (TPM)")
17     ax.set_xlim(1, 11); ax.set_ylim(-1, 17)
18     pal = {"Leaf": GREEN, "Root": BLUE, "Seed": AMBER}
19     for tissue, c in pal.items():
20         g = co[co.tissue == tissue]
21         ax.scatter(g.gene_x, g.gene_y, color=c, s=50, alpha=.35, edgecolors=CREAM,
22                 m_h = np.polyfit(co.gene_x, co.gene_y, 1))
```



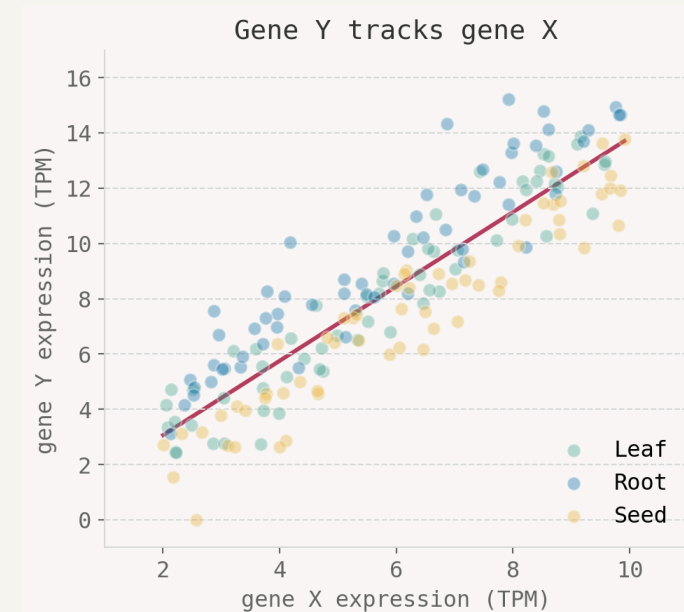
Building a Plot • Step 7 — Drop the Vertical Grid

THE PYTHON VIZ STACK • BUILDING A FIGURE

7 • Grid

Keep the horizontal gridlines as value guides, drop the verticals. Fewer competing lines, the eye reads the y values cleanly.

```
1 import matplotlib.pyplot as plt
2 from genomics_course.data import load_coexpression
3
4 co = load_coexpression()
5
6 house = {
7     "axes.spines.top": False,
8     "axes.spines.right": False,
9     "axes.grid": True,
10    "grid.linestyle": "--",
11    "grid.color": LINE,
12 }
13 with plt.rc_context(house):
14     fig, ax = plt.subplots(figsize=(5.6, 4.8))
15     ax.set_xlabel("gene X expression (TPM)")
16     ax.set_ylabel("gene Y expression (TPM)")
17     ax.set_xlim(1, 11); ax.set_ylim(-1, 17)
18     pal = {"Leaf": GREEN, "Root": BLUE, "Seed": AMBER}
19     for tissue, c in pal.items():
20         g = co[co.tissue == tissue]
21         ax.scatter(g.gene_x, g.gene_y, color=c, s=50, alpha=.35, edgecolors=CREAM,
22                 m_h = np.polyfit(co.gene_x, co.gene_y, 1))
```



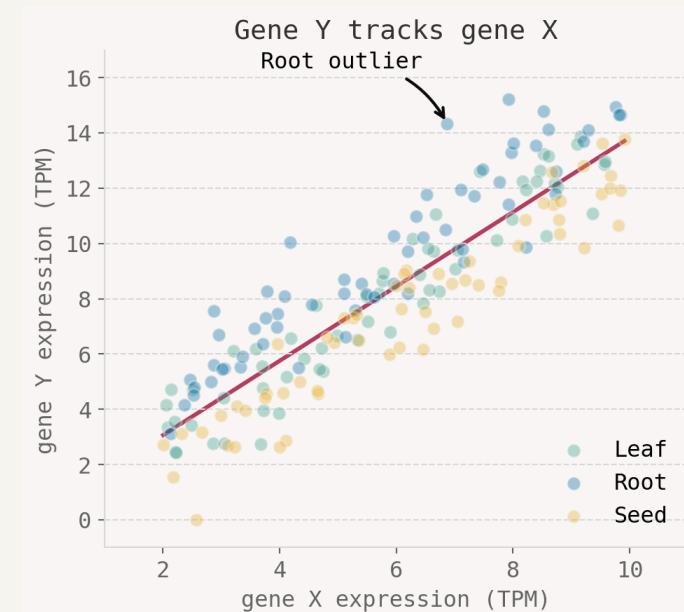
Building a Plot · Step 8 — Point At It

THE PYTHON VIZ STACK · BUILDING A FIGURE

8 · Arrow

Find the sample furthest above the fit, then aim a curved `annotate` arrow at it. The `rad` bends the arrow so it comes in from the side instead of crossing the dot.

```
1 import matplotlib.pyplot as plt
2 from genomics_course.data import load_coexpression
3
4 co = load_coexpression()
5
6 house = {
7     "axes.spines.top": False,
8     "axes.spines.right": False,
9     "axes.grid": True,
10    "grid.linestyle": "--",
11    "grid.color": LINE,
12 }
13 with plt.rc_context(house):
14     fig, ax = plt.subplots(figsize=(5.6, 4.8))
15     ax.set_xlabel("gene X expression (TPM)")
16     ax.set_ylabel("gene Y expression (TPM)")
17     ax.set_xlim(1, 11); ax.set_ylim(-1, 17)
18     pal = {"Leaf": GREEN, "Root": BLUE, "Seed": AMBER}
19     for tissue, c in pal.items():
20         g = co[co.tissue == tissue]
21         ax.scatter(g.gene_x, g.gene_y, color=c, s=50, alpha=.35, edgecolors=CREAM,
22                 m_h = np.polyfit(co.gene_x, co.gene_y, 1))
```



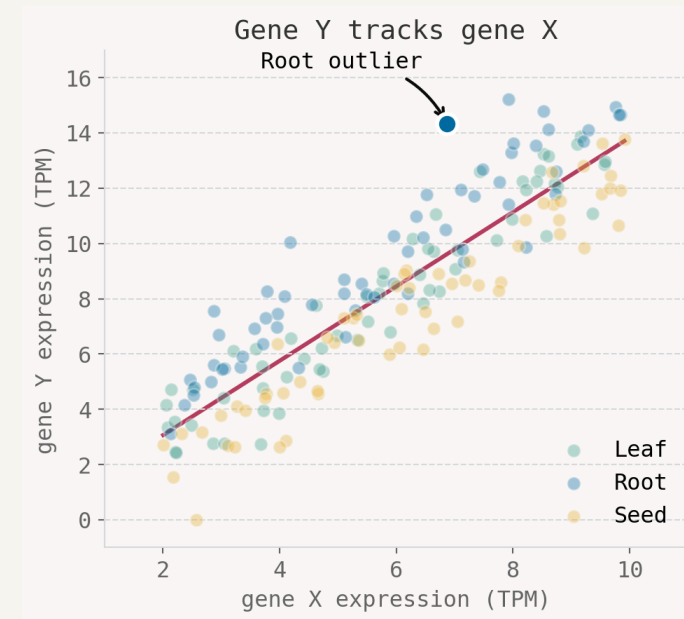
Building a Plot • Step 9 — Highlight the Sample

THE PYTHON VIZ STACK • BUILDING A FIGURE

9 • Edge

Re-draw that one sample with a thick **white edge**. The wall lifts it off the crowd the arrow is pointing at.

```
1 import matplotlib.pyplot as plt
2 from genomics_course.data import load_coexpression
3
4 co = load_coexpression()
5
6 house = {
7     "axes.spines.top": False,
8     "axes.spines.right": False,
9     "axes.grid": True,
10    "grid.linestyle": "--",
11    "grid.color": LINE,
12 }
13 with plt.rc_context(house):
14     fig, ax = plt.subplots(figsize=(5.6, 4.8))
15     ax.set_xlabel("gene X expression (TPM)")
16     ax.set_ylabel("gene Y expression (TPM)")
17     ax.set_xlim(1, 11); ax.set_ylim(-1, 17)
18     pal = {"Leaf": GREEN, "Root": BLUE, "Seed": AMBER}
19     for tissue, c in pal.items():
20         g = co[co.tissue == tissue]
21         ax.scatter(g.gene_x, g.gene_y, color=c, s=50, alpha=.35, edgecolors=CREAM,
22                 m_h = np.polyfit(co.gene_x, co.gene_y, 1))
```



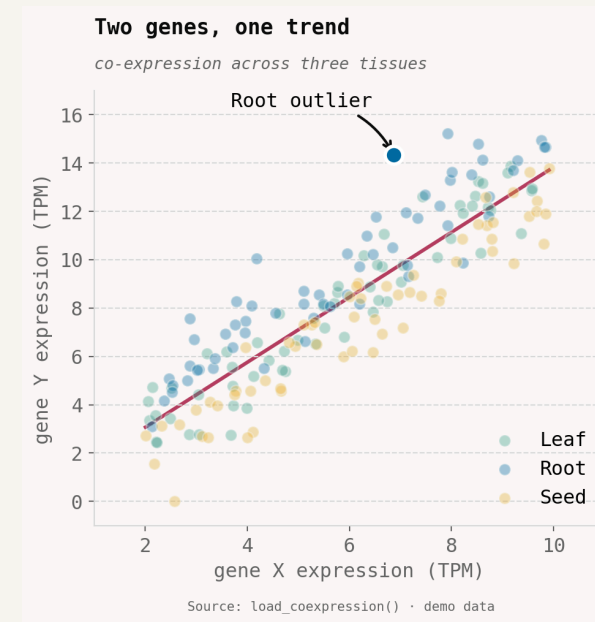
Building a Plot · Step 10 — Title, Subtitle, Caption

THE PYTHON VIZ STACK · BUILDING A FIGURE

10 · Header

Layer the text the way an editorial figure does: a bold **title**, an *italic subtitle*, and a small **caption** crediting the source.

```
1 import matplotlib.pyplot as plt
2 from genomics_course.data import load_coexpression
3
4 co = load_coexpression()
5
6 house = {
7     "axes.spines.top": False,
8     "axes.spines.right": False,
9     "axes.grid": True,
10    "grid.linestyle": "--",
11    "grid.color": LINE,
12 }
13 with plt.rc_context(house):
14     fig, ax = plt.subplots(figsize=(5.6, 4.8))
15     ax.set_xlabel("gene X expression (TPM)")
16     ax.set_ylabel("gene Y expression (TPM)")
17     ax.set_xlim(1, 11); ax.set_ylim(-1, 17)
18     pal = {"Leaf": GREEN, "Root": BLUE, "Seed": AMBER}
19     for tissue, c in pal.items():
20         g = co[co.tissue == tissue]
21         ax.scatter(g.gene_x, g.gene_y, color=c, s=50, alpha=.35, edgecolors=CREAM,
22                 m_h = np.polyfit(co.gene_x, co.gene_y, 1))
```



THE BOTTOM LINE

Build in layers, then refine with **properties**: transparency for density, a lighter grid, a curved arrow plus a white edge to single one point out, and a layered **title / subtitle / caption** to say what it means.

GENOMICS VIZ WITH PYTHON

Exercise — Load, Scatter, Correlate

EXERCISE • YOUR TURN

Your turn: run the whole recipe on a new dataset

`load_scatter_demo()` is a **synthetic practice dataset** (`log2fc`, `expression`, `group`). Load it, scatter `expression` against `log2fc`, then compute Pearson `r` and print it in the corner. Use the house style from Step 5.

```
import matplotlib.pyplot as plt
from genomics_course.data import load_scatter_demo

df = load_scatter_demo()      # columns: log2fc, expression, group

# 1. make a figure + axes
# 2. scatter expression (y) against log2fc (x)
# 3. r = np.corrcoef(...) -> ax.text(...) it in the top-left
# 4. label the axes
# your code here
```

EXERCISE

Get the **mechanics** working first: load → scatter → annotate `r`. We'll look at what the number actually means on the next slide.

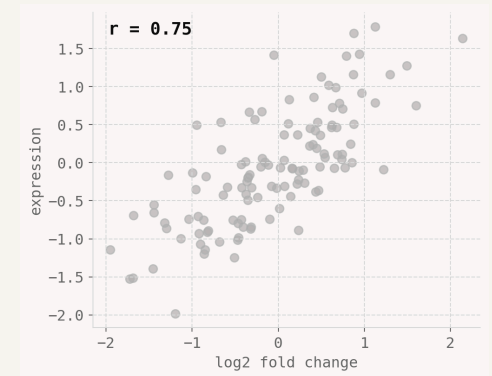
Exercise — Solution & the Catch

EXERCISE • SOLUTION

r is high — but here that is an artifact

The recipe runs and prints a strong-looking r. The catch: in this toy dataset `expression` was **built** as `0.8 * log2fc + noise`, so the correlation is baked in, and `group` is assigned at random. The number is real; the biology is not. Always ask what generated the data before trusting an r.

```
1 import matplotlib.pyplot as plt
2 from genomics_course.data import load_scatter_demo
3
4 df = load_scatter_demo()
5
6 with plt.rc_context(house):
7     fig, ax = plt.subplots(figsize=(10, 10))
8     ax.scatter(df.log2fc, df.expression)
9     r = np.corrcoef(df.log2fc, df.expression)[0, 1]
10    ax.text(0.04, 0.93, f"r = {r:.2f}")
11    ax.set_xlabel("log2 fold change")
12    ax.set_ylabel("expression")
13
14 plt.show()
```



EXERCISE • THE LESSON

A high r only means two columns move together. Here they move together **by construction**. Correlation is a description, not a discovery.

Ternary Plots

COMPOSITIONAL DATA • THREE PARTS THAT SUM TO ONE

Ternary • The Data

GENOMICS • COMPOSITIONAL DATA

Three ancestry fractions per individual, summing to one

Each individual's ancestry splits across three populations and the fractions sum to 1. We simulate that with a Dirichlet draw: mostly-A, mostly-B, mostly-C, plus some evenly admixed.

```
1 import numpy as np, pandas as pd
2
3 rng_adm = np.random.default_rng(7)
4 comp = np.vstack([rng_adm.dirichlet(a, n) for a, n in
5                  [(8,1,1), 30), ([1,8,1], 30), ([1,1,8], 30), ([2,2,2], 20)]]
6 adm = pd.DataFrame(comp, columns=["Pop A", "Pop B", "Pop C"]).round(3)
7 adm.head(4)
```

	Pop A	Pop B	Pop C
0	0.840	0.062	0.098
1	0.697	0.001	0.302
2	0.882	0.075	0.043
3	0.791	0.187	0.022

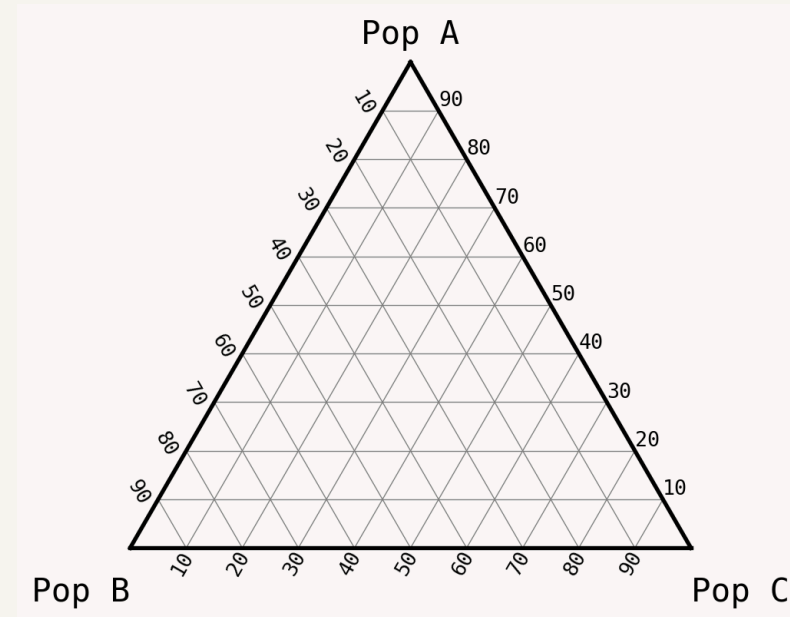
Ternary · Step 1 — The Triangle

GENOMICS · COMPOSITIONAL DATA

1 · Frame

`ternary_diagram` draws the triangle, gridlines, and ticks for you. Three corners, one per population. Near a corner means mostly that population, the centre means evenly admixed.

```
1 from ternary_diagram import TernaryDiagram
2
3 rng_adm = np.random.default_rng(7)          # fixed seed -> same points every step
4 comp = np.vstack([rng_adm.dirichlet(a, n) for a, n in
5                   [([8,1,1], 30), ([1,8,1], 30), ([1,1,8], 30), ([2,2,2], 20)]])
6
7 fig, ax = plt.subplots(figsize=(5.4, 4.6))
8 td = TernaryDiagram(["Pop A", "Pop B", "Pop C"], ax=ax)  # frame, gridlines, ticks
9 plt.show()
```



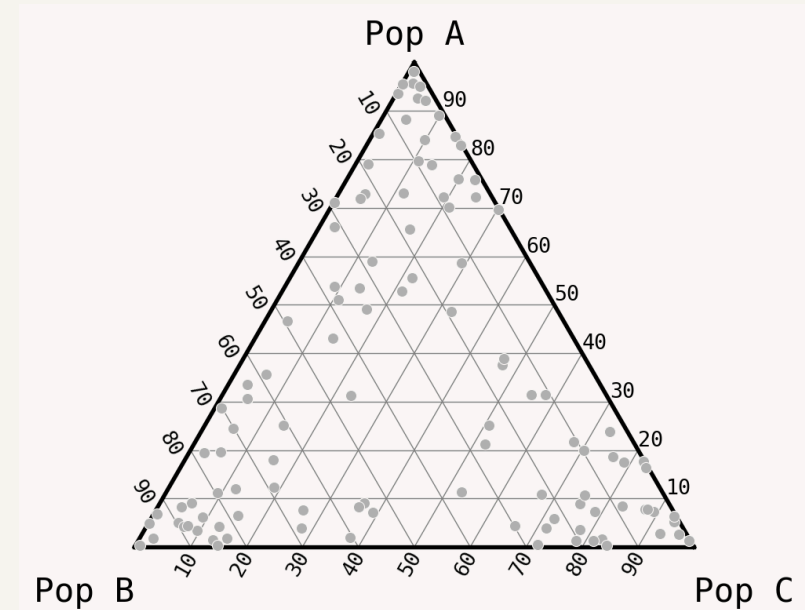
Ternary · Step 2 — The Points

GENOMICS · COMPOSITIONAL DATA

2 · Scatter

`td.scatter` takes the whole `comp` array, one row per individual. It places each point by its three fractions automatically. One colour for now.

```
1 from ternary_diagram import TernaryDiagram
2
3 rng_adm = np.random.default_rng(7)
4 comp = np.vstack([rng_adm.dirichlet(a, n) for a, n in
5                   [[(8,1,1), 30], ([1,8,1], 30), ([1,1,8], 30), ([2,2,2], 20)]]
6
7 fig, ax = plt.subplots(figsize=(5.4, 4.6))
8 td = TernaryDiagram(["Pop A", "Pop B", "Pop C"], ax=ax)
9 td.scatter(vector=comp, c=GREY, s=28, edgecolors=CREAM, linewidths=0.4, zorder=3)
10 plt.show()
```



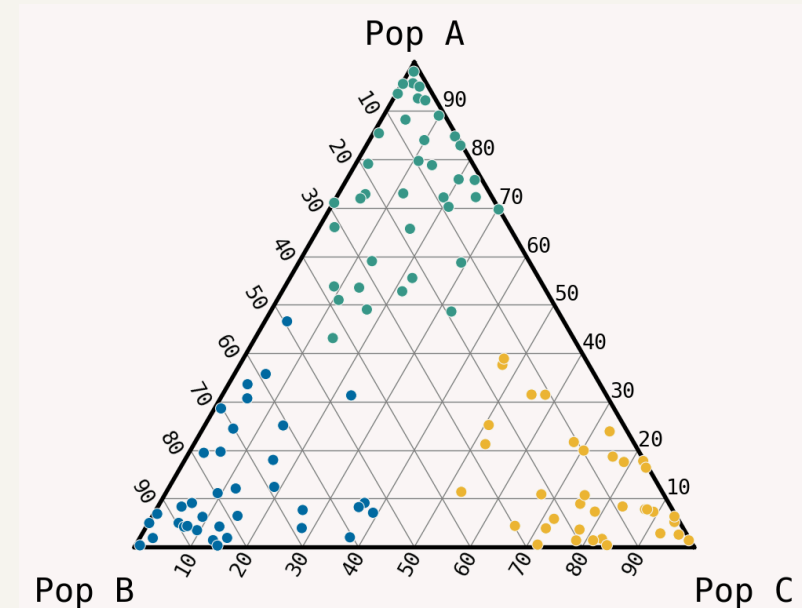
Ternary · Step 3 — Colour by Population

GENOMICS · COMPOSITIONAL DATA

3 · Colour

Colour each point by its **dominant** population (`argmax` of the three fractions). The three corner clusters light up; the admixed individuals stay near the middle.

```
1 from ternary_diagram import TernaryDiagram
2
3 rng_adm = np.random.default_rng(7)
4 comp = np.vstack([rng_adm.dirichlet(a, n) for a, n in
5                   [[8,1,1], 30), ([1,8,1], 30), ([1,1,8], 30), ([2,2,2], 20)]]
6
7 cols = np.array([GREEN, BLUE, AMBER])[comp.argmax(1)] # colour by dominant populat
8 fig, ax = plt.subplots(figsize=(5.4, 4.6))
9 td = TernaryDiagram(["Pop A", "Pop B", "Pop C"], ax=ax)
10 td.scatter(vector=comp, c=cols, s=28, edgecolors=CREAM, linewidths=0.4, zorder=3)
11 plt.show()
```



THE BOTTOM LINE

Three parts that sum to one belong in a **triangle**. Corners are pure ancestry, the centre is admixture, and `argmax` colouring reveals the dominant population at a glance.

Volcano Plots

DIFFERENTIAL EXPRESSION · DAY 01

What Is a Volcano Plot?

DIFFERENTIAL EXPRESSION • THEORY

Effect size meets significance

A volcano plot answers two questions at once for every gene: **how much did it change**, and **how sure are we**.

Each dot is one gene.

The x-axis is **log2 fold change**, the size and direction of the change. The y-axis is **−log10(p-value)**, so higher means more significant. Genes pile up along the bottom (no change, not significant) and the interesting ones rise into the top corners.

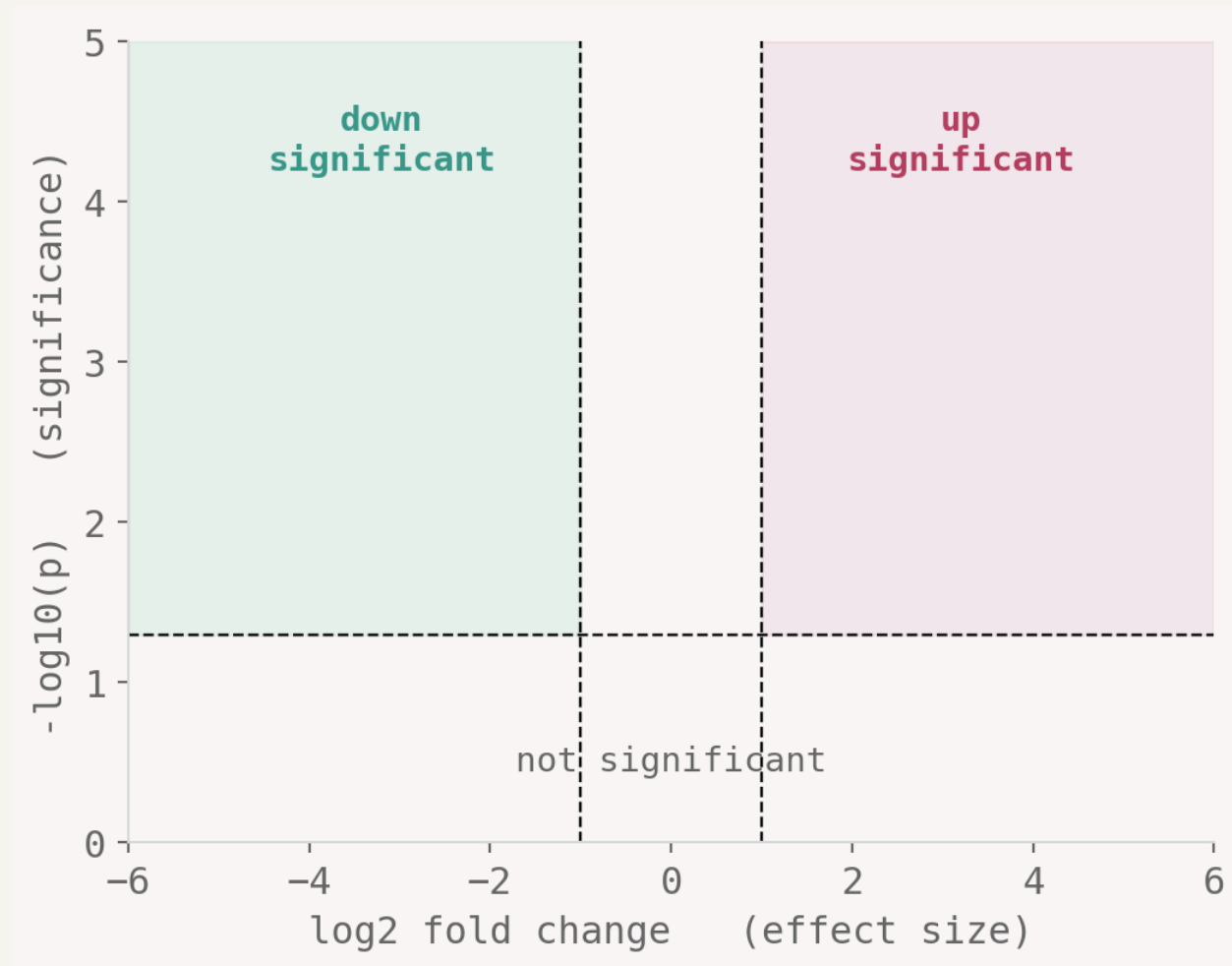
Two thresholds carve out the hits: a fold-change cutoff (typically $|\log_2\text{FC}| > 1$, a two-fold change) and a significance cutoff ($p < 0.05$, i.e. $-\log_{10} p > 1.3$).

Top-right is **up-regulated and significant**, top-left is **down-regulated and significant**, everything else is background.

Anatomy of a Volcano

DIFFERENTIAL EXPRESSION • THEORY

The three regions, before any data



Volcano • The Data

DIFFERENTIAL EXPRESSION

Real DESeq2 output, not simulated

`load_deseq2()` returns one row per gene from a drought-vs-control contrast: effect size (`log2FoldChange`), raw and adjusted p-values (`pvalue`, `padj`), and mean expression (`baseMean`).

```
1 from genomics_course.data import load_deseq2
2
3 de = load_deseq2(contrast="drought") # gene, baseMean, log2FoldChange, pvalue, padj
4 de.head(4)
```

	gene	baseMean	log2FoldChange	pvalue	
0	Gene_00000	2878.659834	0.965552	0.072316	0.99
1	Gene_00001	2637.606836	0.457506	0.395939	0.99
2	Gene_00002	1754.258523	1.002372	0.067253	0.99
3	Gene_00003	11.111826	0.037154	0.973055	0.99

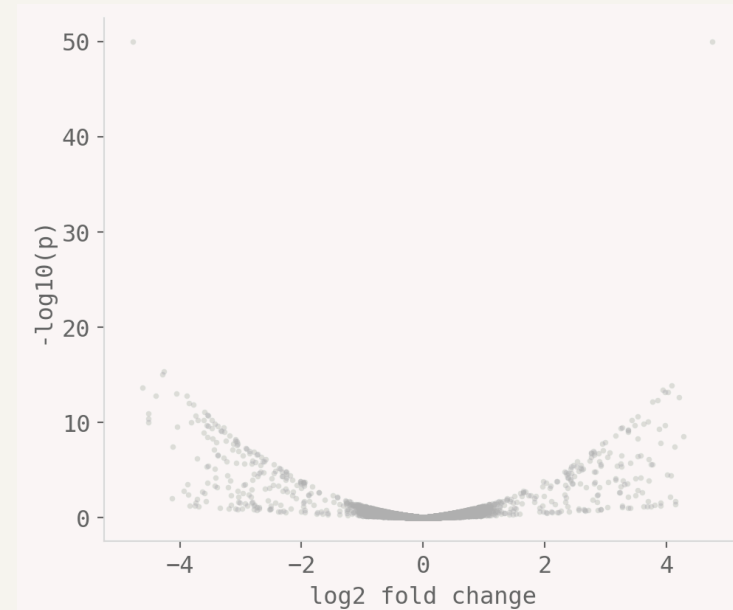
Volcano • Step 1 — Effect vs Significance

DIFFERENTIAL EXPRESSION

1 • The cloud

Derive the significance axis ($-\log_{10}$ of the p-value) and scatter every gene. The volcano shape appears: a wide base of unchanged genes, two rising arms.

```
1 from genomics_course.data import load_deseq2
2
3 de = load_deseq2(contrast="drought")           # one row per gene
4 de["neg_log10_p"] = -np.log10(de.pvalue)       # significance axis
5
6 fig, ax = plt.subplots(figsize=(5.8, 4.8))
7 ax.scatter(de.log2FoldChange, de.neg_log10_p, color=GREY, s=8, alpha=.4, linewidths=
8 ax.set_xlabel("log2 fold change")
9 ax.set_ylabel("-log10(p)")
10 ax.spines[["top", "right"]].set_visible(False)
11 plt.show()
```



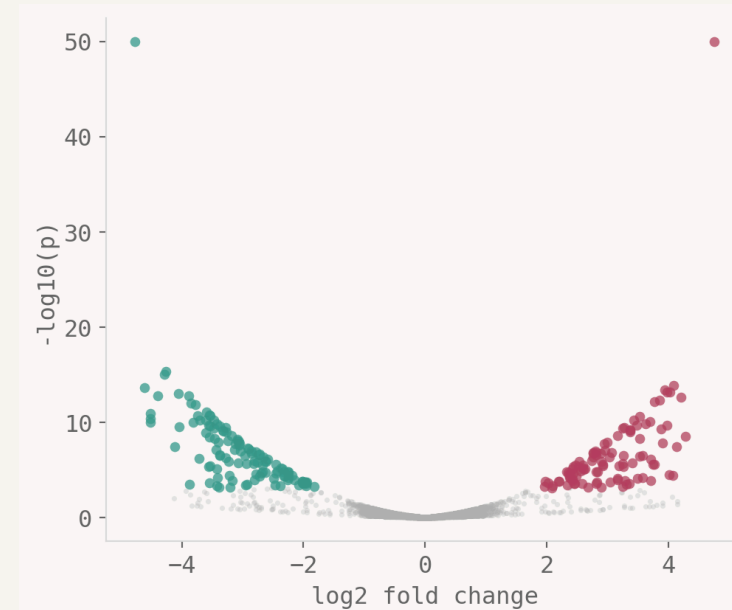
Volcano • Step 2 — Classify & Colour

DIFFERENTIAL EXPRESSION

2 • Direction

Label each gene up / down / ns from its fold change and adjusted p-value, then colour by that label. Red up, green down, grey for the background.

```
1 from genomics_course.data import load_deseq2
2
3 de = load_deseq2(contrast="drought")
4 de["neg_log10_p"] = -np.log10(de.pvalue)
5 de["reg"] = np.where((de.log2FoldChange > 1) & (de.padj < 0.05), "up",      # classi
6                   np.where((de.log2FoldChange < -1) & (de.padj < 0.05), "down", "ns"))
7
8 cm = {"up": RED, "down": GREEN, "ns": GREY}      # red up, green down, grey ns
9 fig, ax = plt.subplots(figsize=(5.8, 4.8))
10 for st, grp in de.groupby("reg"):
11     ax.scatter(grp.log2FoldChange, grp.neg_log10_p, c=cm[st],
12               s=26 if st != "ns" else 7, alpha=.75 if st != "ns" else .3,
13               linewidths=0, zorder=3 if st != "ns" else 1)
14 ax.set_xlabel("log2 fold change")
15 ax.set_ylabel("-log10(p)")
16 ax.spines[["top", "right"]].set_visible(False)
17 plt.show()
```



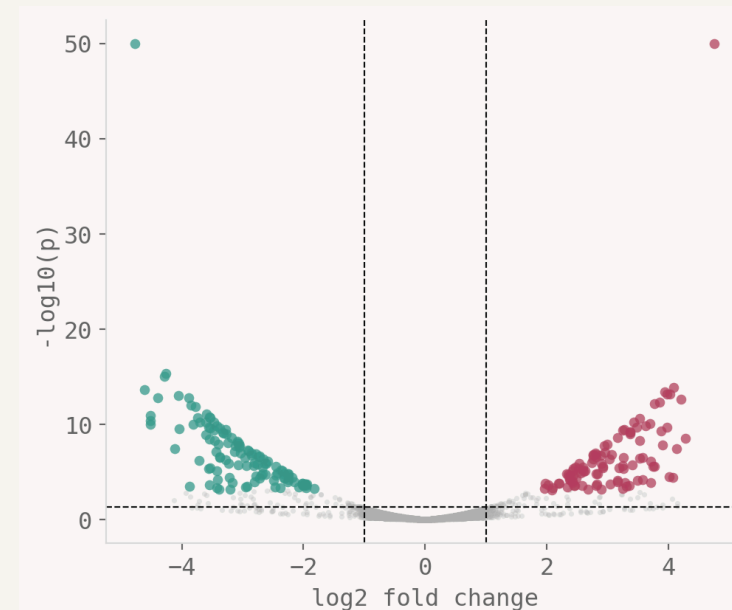
Volcano • Step 3 — Threshold Lines

DIFFERENTIAL EXPRESSION

3 • Cutoffs

Draw the cutoffs that define the call: a horizontal line at $p = 0.05$ and vertical lines at $\pm 1 \log_2\text{FC}$. Now the colour regions have visible borders.

```
1 from genomics_course.data import load_deseq2
2
3 de = load_deseq2(contrast="drought")
4 de["neg_log10_p"] = -np.log10(de.pvalue)
5 de["reg"] = np.where((de.log2FoldChange > 1) & (de.padj < 0.05), "up",
6                     np.where((de.log2FoldChange < -1) & (de.padj < 0.05), "down", "ns"))
7
8 cm = {"up": RED, "down": GREEN, "ns": GREY}
9 fig, ax = plt.subplots(figsize=(5.8, 4.8))
10 for st, grp in de.groupby("reg"):
11     ax.scatter(grp.log2FoldChange, grp.neg_log10_p, c=cm[st],
12               s=26 if st != "ns" else 7, alpha=.75 if st != "ns" else .3,
13               linewidths=0, zorder=3 if st != "ns" else 1)
14 ax.axhline(-np.log10(0.05), color=INK, lw=.8, ls="--") # significance cutoff p=0.0
15 ax.axvline(1, color=INK, lw=.8, ls="--") # fold-change cutoffs
16 ax.axvline(-1, color=INK, lw=.8, ls="--")
17 ax.set_xlabel("log2 fold change")
18 ax.set_ylabel("-log10(p)")
19 ax.spines[["top", "right"]].set_visible(False)
20 plt.show()
```



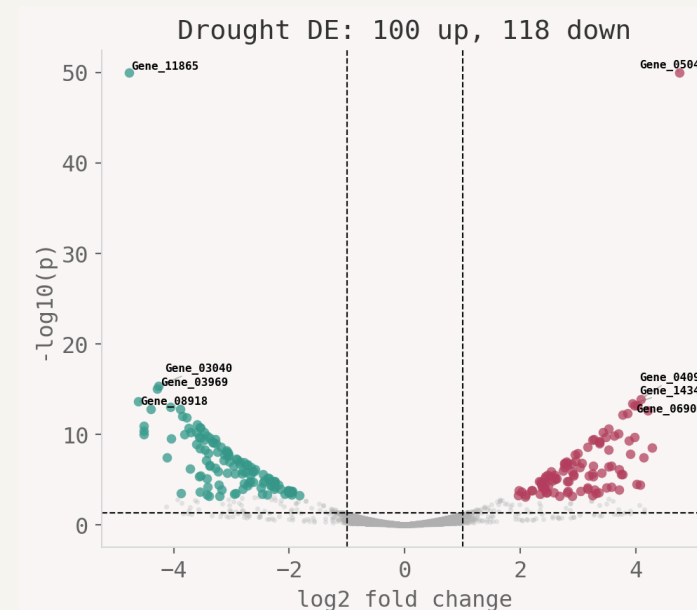
Volcano • Step 4 — Label the Top Genes

DIFFERENTIAL EXPRESSION

4 • Annotate

Take the most significant hits, label them, and let `adjustText` nudge the labels apart so they don't overlap. The title states the count.

```
1 from genomics_course.data import load_deseq2
2 from adjustText import adjust_text          # nudges labels apart
3
4 de = load_deseq2(contrast="drought")
5 de["neg_log10_p"] = -np.log10(de.pvalue)
6 de["reg"] = np.where((de.log2FoldChange > 1) & (de.padj < 0.05), "up",
7                     np.where((de.log2FoldChange < -1) & (de.padj < 0.05), "down", "ns"))
8
9 cm = {"up": RED, "down": GREEN, "ns": GREY}
10 fig, ax = plt.subplots(figsize=(5.8, 4.8))
11 for st, grp in de.groupby("reg"):
12     ax.scatter(grp.log2FoldChange, grp.neg_log10_p, c=cm[st],
13               s=26 if st != "ns" else 7, alpha=.75 if st != "ns" else .3,
14               linewidths=0, zorder=3 if st != "ns" else 1)
15 ax.axhline(-np.log10(0.05), color=INK, lw=.8, ls="--")
16 ax.axvline(1, color=INK, lw=.8, ls="--")
17 ax.axvline(-1, color=INK, lw=.8, ls="--")
18 top = de[de.reg != "ns"].nsmallest(8, "pvalue")          # 8 most significant
19 texts = [ax.text(r.log2FoldChange, r.neg_log10_p, r.gene, fontsize=6, fontweight="
20               for _, r in top.iterrows())]
21 adjust_text(texts, ax=ax, arrowprops=dict(arrowstyle=">", color=GREY, lw=.6)) #
22 ax.set_xlabel("log2 fold change")
```



THE BOTTOM LINE

A volcano puts **effect size** (x) against **significance** (y). Classify, colour by direction (red up, green down, grey background), draw the threshold lines, and label the top hits. On the real drought contrast that's ~100 up and ~118 down.

Interactive Volcano

DIFFERENTIAL EXPRESSION • PLOTLY

From Static to Interactive

DIFFERENTIAL EXPRESSION • PLOTLY

Same data, now you can hover and zoom

The matplotlib volcano is perfect for print. For exploration you want **hover to read the gene**, zoom into a cluster, and toggle a class on and off. Plotly gives all three from the same DataFrame, and the figure embeds live in the slide.

A note on **Dash**: Dash adds dropdowns, sliders, and callbacks, but it runs as its own web server and cannot live inside a slide. The interactive figure below is the part that embeds. The final slide shows the Dash wrapper for when you want the standalone app.

Plotly • Step 1 — The Scatter

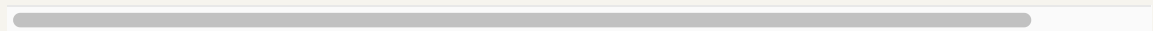
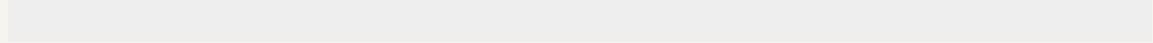
DIFFERENTIAL EXPRESSION • PLOTLY

1 • One call

`px.scatter` builds the whole figure from the DataFrame. Reuse the `reg` and `neg_log10_p` columns from the matplotlib build, no new data work.

```
1 import plotly.express as px
2 from genomics_course.data import load_deseq2
3
4 de = load_deseq2(contrast="drought")
5 de["neg_log10_p"] = -np.log10(de.pvalue)
6 de["reg"] = np.where((de.log2FoldChange > 1) & (de.padj < 0.05), "up",
7                     np.where((de.log2FoldChange < -1) & (de.padj < 0.05), "down", "ns"))
8
9 fig = px.scatter(de, x="log2FoldChange", y="neg_log10_p",
10                 labels={"log2FoldChange": "log2 fold change",
11                         "neg_log10_p": "-log10(p)"})
12 fig.show()
```

WebGL is not supported by
your browser - visit
<https://get.webgl.org> for
more info



Plotly · Step 2 — Colour & Hover

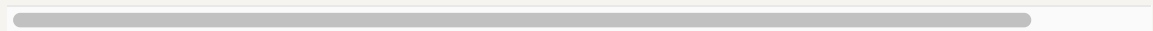
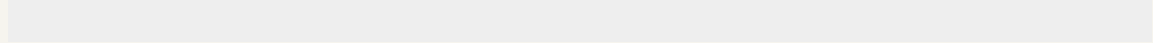
DIFFERENTIAL EXPRESSION · PLOTLY

2 · Direction + identity

Colour by `reg` with the same red/green/grey map, and set `hover_name` so pointing at any dot reveals the gene and its p-value. This is the payoff over a static image.

```
1 import plotly.express as px
2 from genomics_course.data import load_deseq2
3
4 de = load_deseq2(contrast="drought")
5 de["neg_log10_p"] = -np.log10(de.pvalue)
6 de["reg"] = np.where((de.log2FoldChange > 1) & (de.padj < 0.05), "up",
7                     np.where((de.log2FoldChange < -1) & (de.padj < 0.05), "down", "ns"))
8
9 fig = px.scatter(de, x="log2FoldChange", y="neg_log10_p", color="reg",
10                 color_discrete_map={"up": RED, "down": GREEN, "ns": GREY},
11                 category_orders={"reg": ["up", "down", "ns"]},
12                 hover_name="gene", hover_data={"pvalue": ":.1e", "reg": False},
13                 labels={"log2FoldChange": "log2 fold change", "neg_log10_p": "-log1
14 fig.show()
```

WebGL is not supported by
your browser - visit
<https://get.webgl.org> for
more info



Plotly · Step 3 — Thresholds & Polish

DIFFERENTIAL EXPRESSION · PLOTLY

3 · Make it look nicer

Add the cutoff lines, shrink the markers, drop a clean template, and let the cream slide show through with a transparent background. Now it matches the deck.

```
1 import plotly.express as px
2 from genomics_course.data import load_deseq2
3
4 de = load_deseq2(contrast="drought")
5 de["neg_log10_p"] = -np.log10(de.pvalue)
6 de["reg"] = np.where((de.log2FoldChange > 1) & (de.padj < 0.05), "up",
7                     np.where((de.log2FoldChange < -1) & (de.padj < 0.05), "down", "ns"))
8
9 fig = px.scatter(de, x="log2FoldChange", y="neg_log10_p", color="reg",
10                 color_discrete_map={"up": RED, "down": GREEN, "ns": GREY},
11                 category_orders={"reg": ["up", "down", "ns"]},
12                 hover_name="gene", hover_data={"pvalue": ":.1e", "reg": False},
13                 labels={"log2FoldChange": "log2 fold change", "neg_log10_p": "-log10(p-value)"})
14 fig.add_hline(y=-np.log10(0.05), line_dash="dash", line_color=INK, line_width=1)
15 fig.add_vline(x=1, line_dash="dash", line_color=INK, line_width=1)
16 fig.add_vline(x=-1, line_dash="dash", line_color=INK, line_width=1)
17 fig.update_traces(marker=dict(size=5, opacity=0.6, line=dict(width=0)))
18 fig.update_layout(template="simple_white", height=440,
19                 paper_bgcolor="rgba(0,0,0,0)",
20                 font=dict(family="Asap", color=INK, size=12),
21                 legend_title_text="", margin=dict(l=10, r=10, t=30, b=10))
22 fig.show()
```

WebGL is not supported by
your browser - visit
<https://get.webgl.org> for
more info

Plotly turns the same DataFrame into an **explorable** volcano: hover for the gene, zoom into a cluster, click the legend to isolate a class. One `px.scatter`, a colour map, threshold lines, and a transparent template to match the deck.

Wrapping It in a Dash App

DIFFERENTIAL EXPRESSION • PLOTLY

For a standalone, deployable tool (runs as its own server)

Dash adds controls and callbacks, a p-value slider that re-filters the volcano live. This can't run inside a slide; save it as `app.py` and run `python app.py`.

```
1 from dash import Dash, dcc, html, Input, Output
2 import plotly.express as px
3 from genomics_course.data import load_deseq2
4
5 de = load_deseq2(contrast="drought")
6 de["neg_log10_p"] = -np.log10(de.pvalue)
7
8 app = Dash(__name__)
9 app.layout = html.Div([
10     html.H3("Volcano explorer"),
11     dcc.Slider(0, 5, 0.5, value=1.3, id="p-cut"), # significance slider
12     dcc.Graph(id="volcano"),
13 ])
14
15 @app.callback(Output("volcano", "figure"), Input("p-cut", "value"))
16 def draw(p_cut):
17     d = de.copy()
18     d["reg"] = np.where((d.log2FoldChange > 1) & (d.neg_log10_p > p_cut), "up",
19                       np.where((d.log2FoldChange < -1) & (d.neg_log10_p > p_cut), "down", "ns"))
20     fig = px.scatter(d, x="log2FoldChange", y="neg_log10_p", color="reg",
21                     color_discrete_map={"up": RED, "down": GREEN, "ns": GREY},
22                     hover_name="gene")
```

Dimensionality Reduction

MANY FEATURES INTO TWO AXES · DAY 01

Why Reduce, and Which Method

DIMENSIONALITY REDUCTION • THEORY

Three projections, three jobs

Genomic data is wide: thousands of genes, hundreds of samples. You can't see 5000 dimensions, so you **project** to two while keeping what matters. The three standard tools keep different things, so they answer different questions.

Method	Keeps	Speed	Best genomics use
PCA	Global variance, real distances, interpretable axes	Fast	Population structure from genotypes/ancestry; bulk RNA-seq QC; batch detection
t-SNE	Local neighbourhoods (tight clusters)	Slow	Single-cell RNA-seq cluster visualisation
UMAP	Local + some global; reproducible	Fast	Single-cell (the scanpy default); large datasets

The rule of thumb: **PCA to understand structure and distances, t-SNE / UMAP to see clusters.**

The Three Rules That Save You

DIMENSIONALITY REDUCTION • THEORY

Before you trust an embedding

PCA is linear; t-SNE and UMAP are not. PCA axes are combinations of features you can interpret and whose distances are real. In t-SNE and UMAP, **distances between clusters carry no meaning**, only “these points are neighbours.”

Run t-SNE / UMAP on PCA-reduced data. For thousands of genes, reduce to ~50 PCs first, then embed. It is faster and denoises. (Our ancestry data is already 5-dimensional, so we skip that here.)

Seed everything. t-SNE and UMAP are stochastic; set `random_state` or the picture changes every run.

Dim. Reduction • The Data

DIMENSIONALITY REDUCTION

Real ancestry: 120 individuals, 5 populations

`load_admixture()` gives each individual's ancestry split across five source populations (`K1` – `K5`, summing to 1) plus the population label. This is the textbook home of PCA in genomics: **revealing population structure**.

```
1 from genomics_course.data import load_admixture
2
3 adm = load_admixture() # individual, population, K1..K5
4 adm.head(4)
```

	individual	population	K1	K2	K3	
0	EUR_000	EUR	0.978699	0.002754	0.000058	0.000499
1	EUR_001	EUR	0.978085	0.011594	0.000160	0.000160
2	EUR_002	EUR	0.977956	0.009453	0.000056	0.000499
3	EUR_003	EUR	0.971030	0.001839	0.025570	0.000499

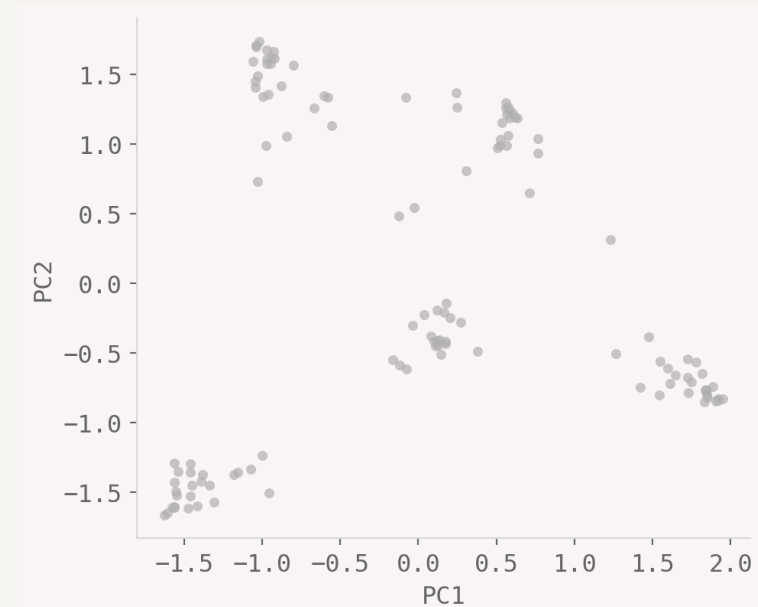
PCA • Step 1 — Project

DIMENSIONALITY REDUCTION • PCA

1 • Scale, fit, plot

Scale each feature, fit PCA, and scatter the first two components. Structure is already visible, but we can't yet tell which cluster is which.

```
1 from genomics_course.data import load_admixture
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.decomposition import PCA
4
5 adm = load_admixture()
6 X = adm[["K1", "K2", "K3", "K4", "K5"]].values
7 Xs = StandardScaler().fit_transform(X)      # centre + scale each feature
8 pca = PCA(n_components=5, random_state=42)
9 coords = pca.fit_transform(Xs)             # project onto components
10
11 fig, ax = plt.subplots(figsize=(5.4, 4.6))
12 ax.scatter(coords[:, 0], coords[:, 1], color=GREY, s=24, alpha=.7, linewidths=0)
13 ax.set_xlabel("PC1"); ax.set_ylabel("PC2")
14 ax.spines[["top", "right"]].set_visible(False)
15 plt.show()
```



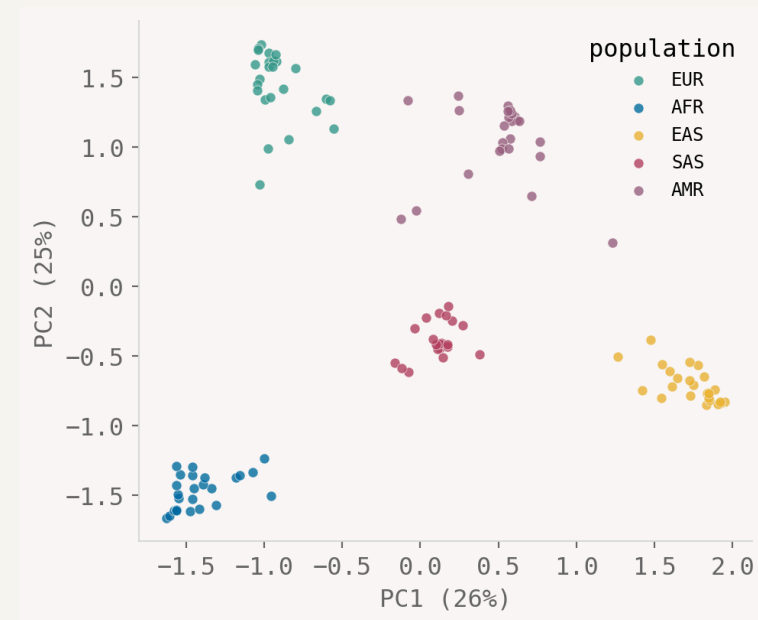
PCA • Step 2 — Colour by Population

DIMENSIONALITY REDUCTION • PCA

2 • Structure appears

Colour by population and add the variance each axis explains. The five groups separate, and PCA's real distances show **AMR sitting between the others**, it is the admixed population.

```
1 from genomics_course.data import load_admixture
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.decomposition import PCA
4
5 adm = load_admixture()
6 X = adm[["K1", "K2", "K3", "K4", "K5"]].values
7 Xs = StandardScaler().fit_transform(X)
8 pca = PCA(n_components=5, random_state=42)
9 coords = pca.fit_transform(Xs)
10 var = pca.explained_variance_ratio_ * 100      # % variance per PC
11 pal = {"EUR": GREEN, "AFR": BLUE, "EAS": AMBER, "SAS": RED, "AMR": PURPLE}
12
13 fig, ax = plt.subplots(figsize=(5.4, 4.6))
14 for p, c in pal.items():
15     m = adm.population == p
16     ax.scatter(coords[m, 0], coords[m, 1], color=c, s=24, alpha=.8, edgecolors=CREAM)
17 ax.set_xlabel(f"PC1 ({var[0]:.0f}%); ax.set_ylabel(f"PC2 ({var[1]:.0f}%))
18 ax.legend(frameon=False, fontsize=9, title="population")
19 ax.spines[["top", "right"]].set_visible(False)
20 plt.show()
```



PCA • How Many Components?

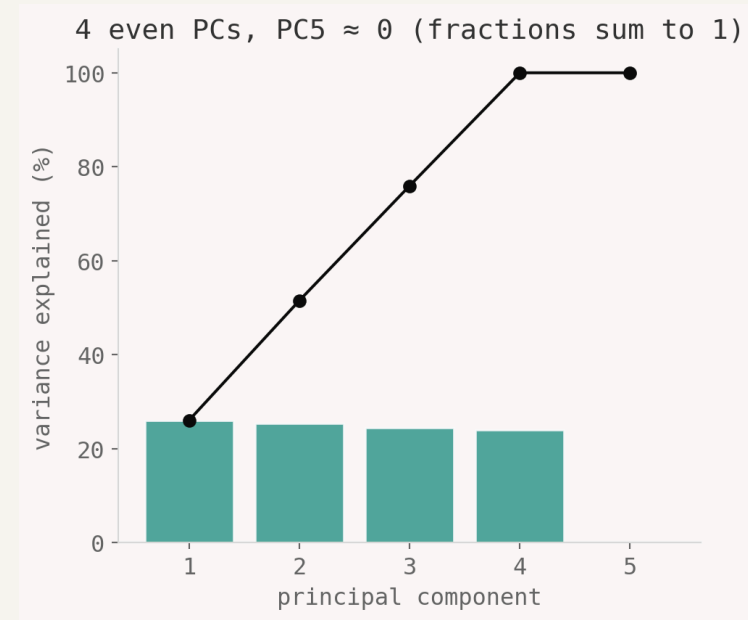
DIMENSIONALITY REDUCTION • PCA

The scree plot, and a compositional quirk

The scree plot shows variance per component.

Normally you look for an “elbow.” Here you get **four near-equal components and a fifth at zero**: because the five fractions sum to 1, the data really lives in four dimensions.

```
1 from genomics_course.data import load_admixture
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.decomposition import PCA
4
5 adm = load_admixture()
6 X = adm[["K1", "K2", "K3", "K4", "K5"]].values
7 Xs = StandardScaler().fit_transform(X)
8 pca = PCA(n_components=5, random_state=42).fit(Xs)
9 var = pca.explained_variance_ratio_ * 100
10 cum = var.cumsum()                                # cumulative variance
11
12 fig, ax = plt.subplots(figsize=(5.4, 4.6))
13 pcs = range(1, 6)
14 ax.bar(pcs, var, color=GREEN, alpha=.85, edgecolor="white") # per-PC variance
15 ax.plot(pcs, cum, color=INK, marker="o", lw=1.5)           # cumulative
16 ax.set_xlabel("principal component"); ax.set_ylabel("variance explained (%)")
17 ax.set_title("4 even PCs, PC5 ≈ 0 (fractions sum to 1)", color="#333333")
18 ax.spines[["top", "right"]].set_visible(False)
19 plt.show()
```



t-SNE • Local Clusters

DIMENSIONALITY REDUCTION • T-SNE

Same data, tightened into clusters

t-SNE drops the linear constraint and pulls neighbours together. The populations become tight, well-separated blobs, great for spotting clusters, but the gaps between them mean nothing.

```
1 from genomics_course.data import load_admixture
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.decomposition import PCA
4 from sklearn.manifold import TSNE
5
6 adm = load_admixture()
7 X = adm[["K1", "K2", "K3", "K4", "K5"]].values
8 Xs = StandardScaler().fit_transform(X)
9 emb = TSNE(n_components=2, perplexity=30, init="pca", random_state=42).fit_transform(Xs)
10
11 pal = {"EUR": GREEN, "AFR": BLUE, "EAS": AMBER, "SAS": RED, "AMR": PURPLE}
12 fig, ax = plt.subplots(figsize=(5.4, 4.6))
13 for p, c in pal.items():
14     m = adm.population == p
15     ax.scatter(emb[m, 0], emb[m, 1], color=c, s=24, alpha=.8, edgecolors=CREAM, line
16 ax.set_xlabel("t-SNE 1"); ax.set_ylabel("t-SNE 2")
17 ax.legend(frameon=False, fontsize=9)
18 ax.spines[["top", "right"]].set_visible(False)
19 plt.show()
```



UMAP • Local + Some Global

DIMENSIONALITY REDUCTION • UMAP

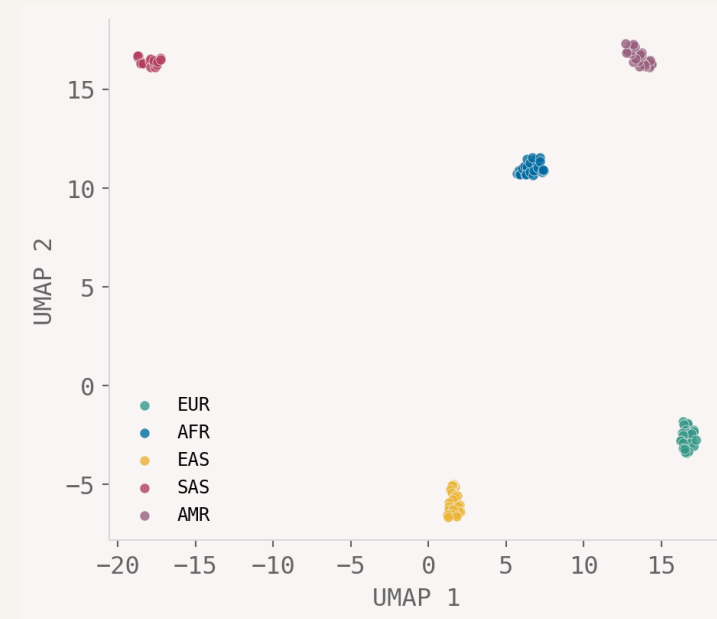
Faster, reproducible, the single-cell default

UMAP keeps tight clusters like t-SNE but runs faster and preserves a little more of the global layout.

`n_neighbors` trades local detail for global structure;

`min_dist` controls how tightly points pack.

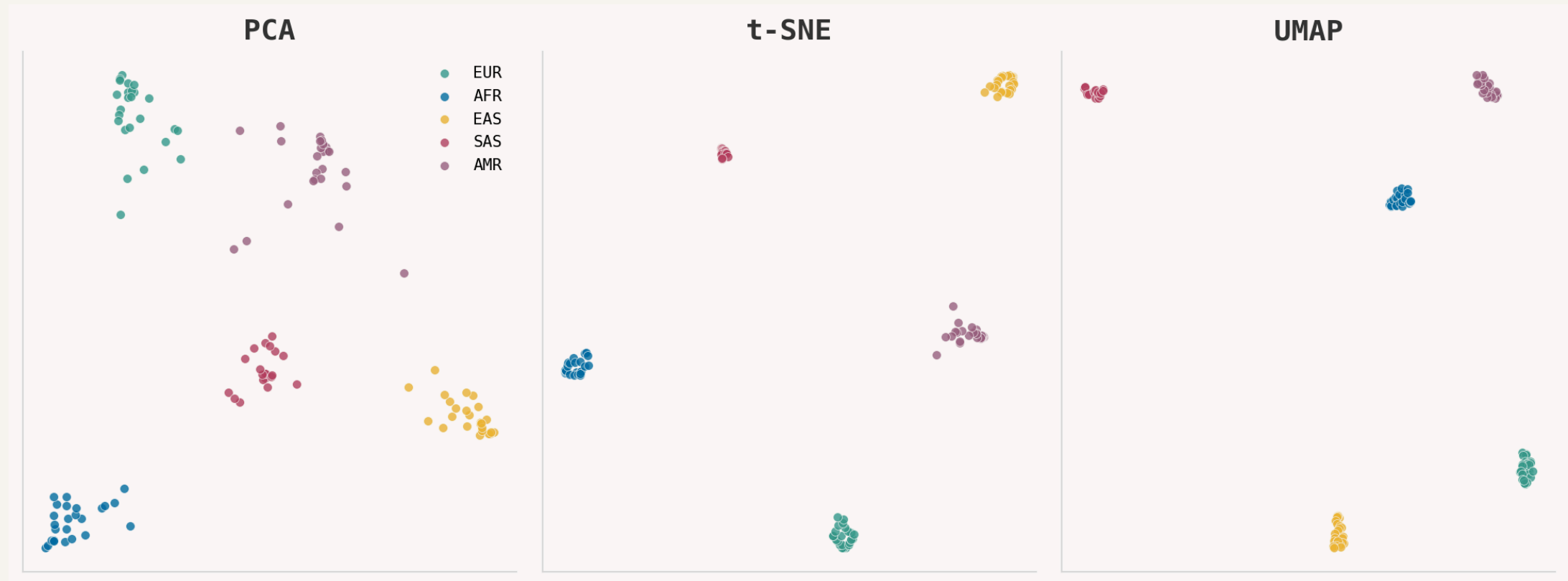
```
1 from genomics_course.data import load_admixture
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.decomposition import PCA
4 import umap
5
6 adm = load_admixture()
7 X = adm[["K1", "K2", "K3", "K4", "K5"]].values
8 Xs = StandardScaler().fit_transform(X)
9 emb = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=42).fit_transform(Xs)
10
11 pal = {"EUR": GREEN, "AFR": BLUE, "EAS": AMBER, "SAS": RED, "AMR": PURPLE}
12 fig, ax = plt.subplots(figsize=(5.4, 4.6))
13 for p, c in pal.items():
14     m = adm.population == p
15     ax.scatter(emb[m, 0], emb[m, 1], color=c, s=24, alpha=.8, edgecolors=CREAM, line
16 ax.set_xlabel("UMAP 1"); ax.set_ylabel("UMAP 2")
17 ax.legend(frameon=False, fontsize=9)
18 ax.spines[["top", "right"]].set_visible(False)
19 plt.show()
```



Side by Side

DIMENSIONALITY REDUCTION

Same ancestry data, three projections



THE BOTTOM LINE

PCA keeps real distances and interpretable axes, reach for it first and for population structure. **t-SNE** and **UMAP** trade distance for crisp clusters, reach for them on single-cell. Same five populations here, three different lenses.

UMAP Is a Network

DIMENSIONALITY REDUCTION • UMAP

The embedding is a neighbour graph laid flat

UMAP works in two moves: first it builds a weighted **k-nearest-neighbour graph** (who is close to whom), then it lays that graph out in 2D. `umap.plot.connectivity` draws the graph itself, points plus the edges. Dense clumps are the populations (everyone is everyone's neighbour); the thin threads between clumps are the handful of cross-population links. Seeing the graph explains *why* UMAP clusters: it is a network embedding, not a projection.

```
1 from genomics_course.data import load_admixture
2 from sklearn.preprocessing import StandardScaler
3 import umap, umap.plot
4
5 adm = load_admixture()
6 Xs = StandardScaler().fit_transform(adm[["K1", "K2", "K3", "K4", "K5"]].values)
7
8 reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=42).fit(Xs) # .fit
9 umap.plot.connectivity(reducer, show_points=True) # draw the neighbour graph
```

UMAP: n_neighbors=15, min_dist=0.1

THE BOTTOM LINE

UMAP first builds a **kNN graph**, then embeds it. `connectivity` shows that graph directly: tight knots are clusters, thin bridges are the rare links between them. The 2D scatter you usually see is just this network, flattened.

Bar • The Data

BAR CHARTS

BRCA1 expression across five tissues

`load_tissue_expression()` returns one value per tissue: BRCA1 in transcripts per million. Heart is highest, the fact we want the chart to say.

```
1 from genomics_course.data import load_tissue_expression # course loader, real valu
2
3 te = load_tissue_expression() # columns: tissue, tpm
4 te
```

	tissue	tpm
0	Heart	91
1	Lung	85
2	Liver	72
3	Kidney	63
4	Brain	58

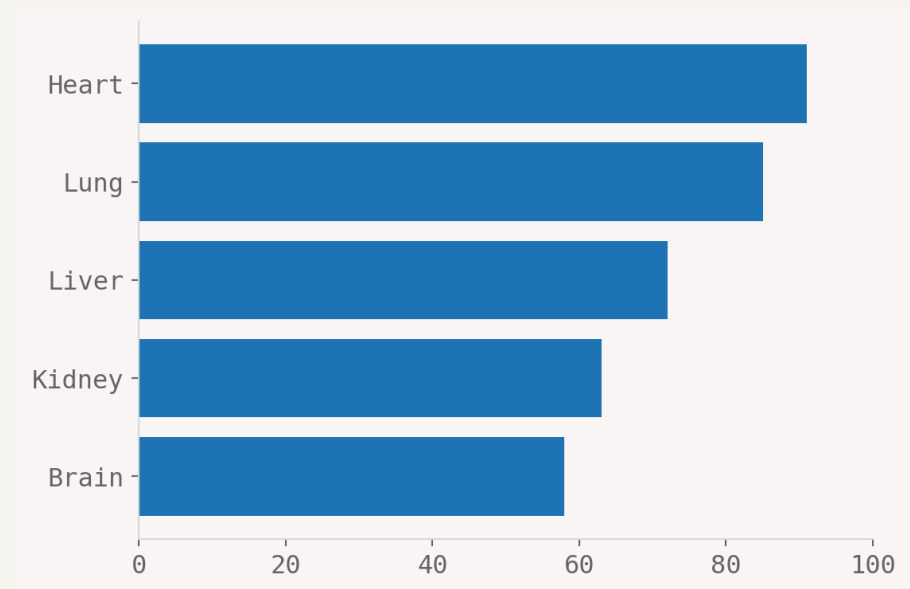
Bar • Step 1 — Plain Bars

BAR CHARTS • BUILD

1 • Default matplotlib

Sort low-to-high so the largest bar ends on top, then draw. Functional, but every bar shouts equally and the frame is noisy.

```
1 from genomics_course.data import load_tissue_expression
2
3 te = load_tissue_expression().sort_values("tpm") # sort so the biggest bar ends on
4
5 fig, ax = plt.subplots(figsize=(6.2, 4.4))      # one panel
6 ax.barh(te.tissue, te.tpm)                      # horizontal bars, default colou
7 ax.set_xlim(0, 100)                            # start at 0 so bar length == va
8 plt.show()
```



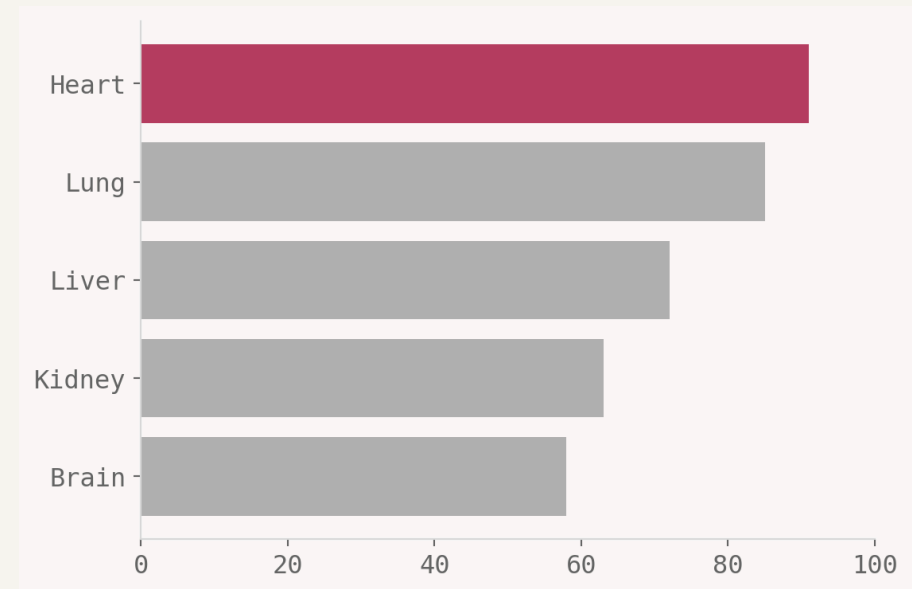
Bar • Step 2 — Colour Carries the Story

BAR CHARTS • BUILD

2 • One bold, the rest quiet

Build a colour list: red for Heart, grey for everything else. The eye lands on the one bar that matters before reading a single label.

```
1 from genomics_course.data import load_tissue_expression
2
3 te = load_tissue_expression().sort_values("tpm")           # tissue, tpm; sorted
4 colors = [RED if t == "Heart" else GREY for t in te.tissue] # one bold colour, res
5
6 fig, ax = plt.subplots(figsize=(6.2, 4.4))               # one panel
7 ax.barh(te.tissue, te.tpm, color=colors)                  # apply the per-bar co
8 ax.set_xlim(0, 100)                                       # zero baseline
9 plt.show()
```



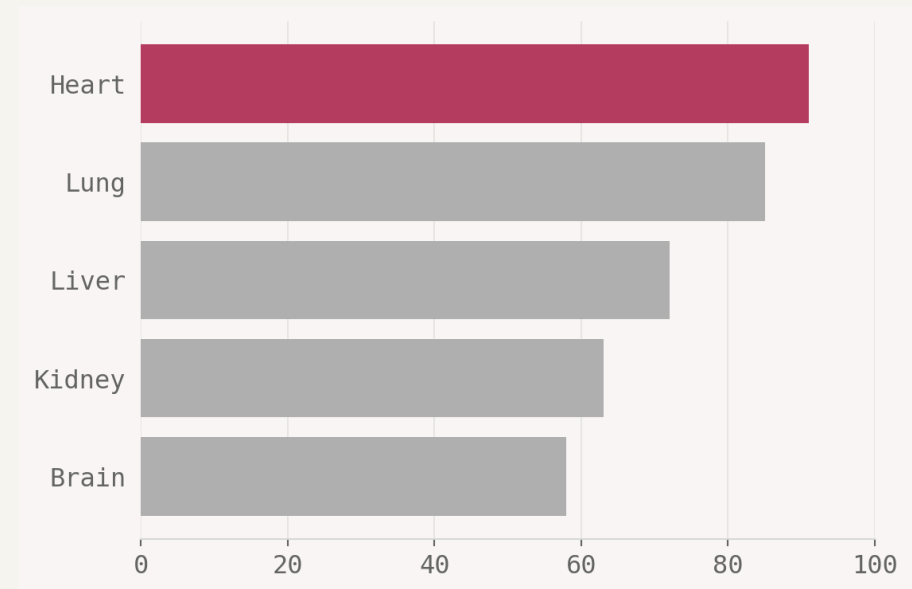
Bar • Step 3 — Drop the Frame

BAR CHARTS • BUILD

3 • Less ink

Hide the left, right, and top spines and the y-ticks, then add a faint grid on the x-axis only. Only the scale that matters survives.

```
1 from genomics_course.data import load_tissue_expression
2
3 te = load_tissue_expression().sort_values("tpm")           # tissue, tpm; sorted
4 colors = [RED if t == "Heart" else GREY for t in te.tissue] # one bold, rest grey
5
6 fig, ax = plt.subplots(figsize=(6.2, 4.4))                # one panel
7 ax.barh(te.tissue, te.tpm, color=colors)                   # coloured bars
8 for s in ["left", "right", "top"]:                          # walk the three unwan
9     ax.spines[s].set_visible(False)                        # remove each one
10 ax.tick_params(left=False)                                 # drop the y-axis tick
11 ax.grid(axis="x", alpha=.3, lw=.6); ax.set_axisbelow(True) # faint x grid, behind
12 ax.set_xlim(0, 100)                                       # zero baseline
13 plt.show()
```



Bar • Step 4 — Label the Bars

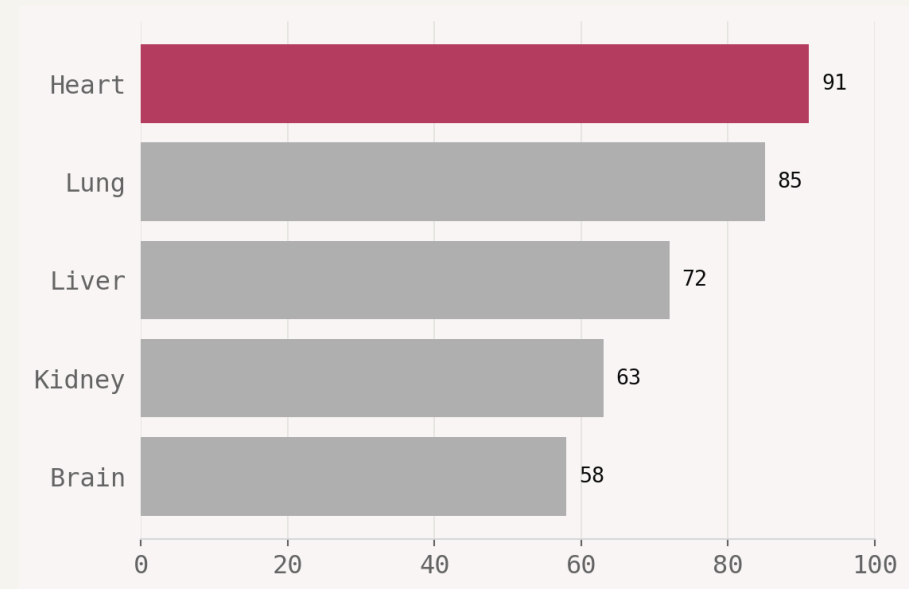
BAR CHARTS • BUILD

4 • Values on the bars

`ax.bar_label` writes each value at the end of its bar.

Now the reader gets the exact number without chasing a gridline.

```
1 from genomics_course.data import load_tissue_expression
2
3 te = load_tissue_expression().sort_values("tpm")           # tissue, tpm; sorted
4 colors = [RED if t == "Heart" else GREY for t in te.tissue] # one bold, rest grey
5
6 fig, ax = plt.subplots(figsize=(6.2, 4.4))               # one panel
7 ax.barh(te.tissue, te.tpm, color=colors)                  # coloured bars
8 for s in ["left", "right", "top"]:                        # remove unwanted spin
9     ax.spines[s].set_visible(False)
10 ax.tick_params(left=False)                                # no y ticks
11 ax.grid(axis="x", alpha=.3, lw=.6); ax.set_axisbelow(True) # faint x grid behind
12 ax.bar_label(ax.containers[0], padding=6, color=INK, fontsize=10) # value at the end
13 ax.set_xlim(0, 100)                                       # zero baseline
14 plt.show()
```



Bar • Step 5 — Say What It Shows

BAR CHARTS • BUILD

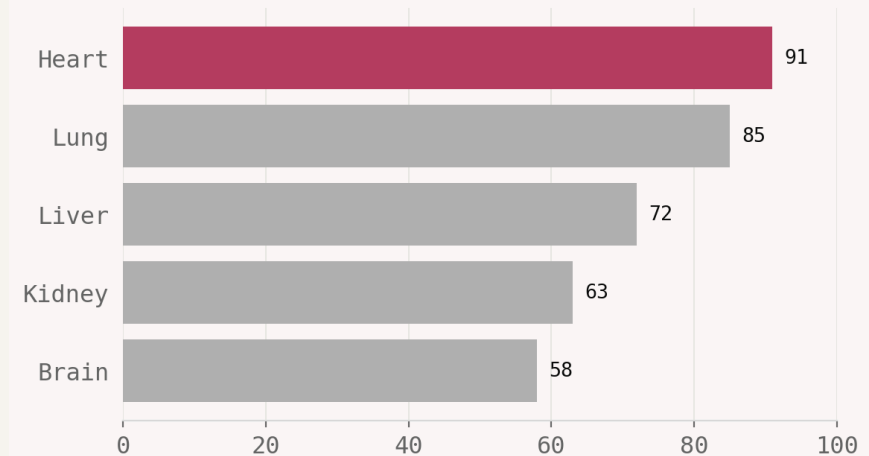
5 • A descriptive title

A title that states the finding, not the variables. “BRCA1 is highest in heart tissue” does the interpreting for the reader; a subtitle gives the units.

```
1 from genomics_course.data import load_tissue_expression
2
3 te = load_tissue_expression().sort_values("tpm")           # tissue, tpm; sorted
4 colors = [RED if t == "Heart" else GREY for t in te.tissue] # one bold, rest grey
5
6 fig, ax = plt.subplots(figsize=(6.2, 4.4))                # one panel
7 ax.barh(te.tissue, te.tpm, color=colors)                  # coloured bars
8 for s in ["left", "right", "top"]:                         # remove unwanted spin
9     ax.spines[s].set_visible(False)
10 ax.tick_params(left=False)                                # no y ticks
11 ax.grid(axis="x", alpha=.3, lw=.6); ax.set_axisbelow(True) # faint x grid behind
12 ax.bar_label(ax.containers[0], padding=6, color=INK, fontsize=10) # value labels
13 ax.set_xlim(0, 100)                                       # zero baseline
14 fig.suptitle("BRCA1 is highest in heart tissue", fontsize=13, fontweight="bold", #
15              color=INK, x=.04, ha="left", y=.98)          # pin it left, near th
16 fig.text(.04, .90, "Expression across five tissues (TPM)", fontsize=10, color=MUTED)
17 plt.tight_layout(rect=[0, 0, 1, .88])                     # leave headroom for t
18 plt.show()
```

BRCA1 is highest in heart tissue

Expression across five tissues (TPM)



THE BOTTOM LINE

Colour carries the story. Spend the loud colour on the row that matters; frame, ticks, and legend disappear. The recipe: **sort** → **one bold colour** → **drop the frame** → **label the bars** → **title the finding**.

GENOMICS VIZ WITH PYTHON

Exercise • Your Turn

BAR CHARTS • EXERCISE

Highlight the under-sequenced sample

CODING EXERCISE

`load_reads_per_sample()` gives reads (millions) for five samples. Build a publication-ready bar chart that makes the **lowest-depth sample** the story: bold it, grey the rest, label the bars, and title the finding.

```
1 from genomics_course.data import load_reads_per_sample
2
3 rps = load_reads_per_sample().sort_values("reads") # sample, reads; smallest first
4 low = rps.sample.iloc[0]                          # the under-sequenced sample
5
6 # 1. colours: RED for `low`, GREY for the rest
7 # 2. ax.barh, drop left/right/top spines, quiet x-grid
8 # 3. ax.bar_label the bars
9 # 4. title that names the finding, e.g. "S05 is under-sequenced"
```

Stacked Bars • Composition Over Time

METAGENOMICS • THEORY

Watch the whole shift, day by day

A stacked bar shows **composition**: each bar is a whole (one sample) split into parts (taxa) that sum to 1. Line the bars up **along time** and you don't just compare samples, you watch the community change: a perturbation hits, one taxon blooms, the rest recover.

Two rules keep it readable: keep the **taxon order and colours identical** across every bar, so a band means the same thing on day 1 and day 16, and mark the **event** (here, antibiotics) so the cause sits next to the effect. Microbiome tables arrive tidy/long, so the first move is a pivot to wide.

Stacked Bars • The Data

METAGENOMICS

Daily phylum abundance across an antibiotic course

`load_microbiome()` returns tidy long form: one row per day-taxon, abundances summing to 1 each day, with a `phase` label (baseline / antibiotic / recovery).

```
1 from genomics_course.data import load_microbiome
2
3 mb = load_microbiome()           # day, phase, taxon, abundance
4 mb.head(4)
```

	day	phase	taxon	abundance
0	1	baseline	Firmicutes	0.423255
1	1	baseline	Bacteroidetes	0.360016
2	1	baseline	Proteobacteria	0.040348
3	1	baseline	Actinobacteria	0.073075

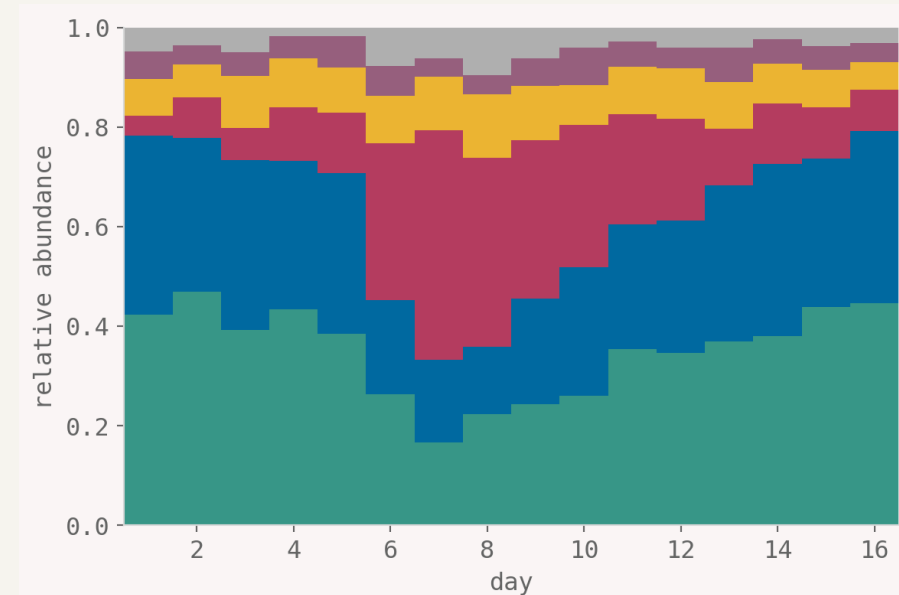
Stacked Bars • Step 1 — Pivot & Stack

METAGENOMICS • BUILD

1 • Long to wide, stacked along time

Pivot to one row per day, one column per taxon, then stack the taxa with a running `bottom`. The x-axis is time, so the bloom is already visible mid-course.

```
1 from genomics_course.data import load_microbiome
2
3 mb = load_microbiome()                                # long: day, ph
4 wide = mb.pivot(index="day", columns="taxon", values="abundance") # rows = days, c
5 taxa = ["Firmicutes", "Bacteroidetes", "Proteobacteria",      # fixed taxon or
6         "Actinobacteria", "Verrucomicrobia", "Other"]
7 tax_pal = {"Firmicutes": GREEN, "Bacteroidetes": BLUE,        # one fixed color
8            "Proteobacteria": RED, "Actinobacteria": AMBER,
9            "Verrucomicrobia": PURPLE, "Other": GREY}
10 days = wide.index.values                                # x positions = t
11
12 fig, ax = plt.subplots(figsize=(6.8, 4.4))              # one panel
13 bottom = np.zeros(len(wide))                            # running base he
14 for t in taxa:                                          # add one taxon i
15     ax.bar(days, wide[t], bottom=bottom, color=tax_pal[t], # this taxon's si
16           width=1.0, linewidth=0, label=t)             # width 1 so days
17     bottom += wide[t].values                            # raise base for
18 ax.set_xlim(days.min() - 0.5, days.max() + 0.5)        # tight x range
19 ax.set_ylim(0, 1)                                       # composition fil
20 ax.set_xlabel("day"); ax.set_ylabel("relative abundance")
21 plt.show()
```



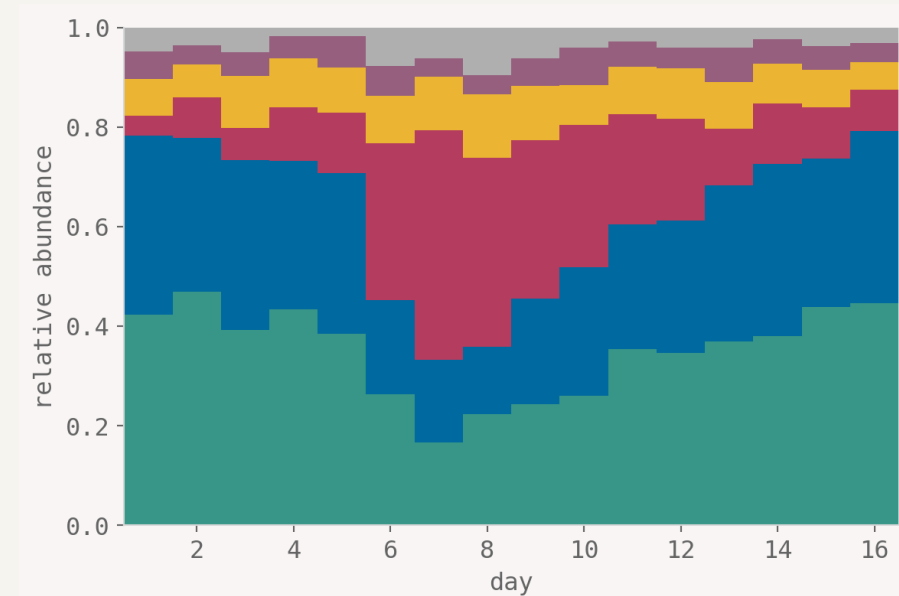
Stacked Bars • Step 2 — Order by Abundance

METAGENOMICS • BUILD

2 • Most abundant at the bottom

Replace the fixed taxon list with one sorted by mean abundance. The dominant phyla anchor the base of every bar; rare ones ride on top, the convention that makes the stack legible.

```
1 from genomics_course.data import load_microbiome
2
3 mb = load_microbiome()                                # long form
4 wide = mb.pivot(index="day", columns="taxon", values="abundance") # days × taxa
5 taxa = (mb.groupby("taxon").abundance.mean()           # mean abundance
6         .sort_values(ascending=False).index)          # ...most abundant
7 tax_pal = {"Firmicutes": GREEN, "Bacteroidetes": BLUE, # fixed colour per
8            "Proteobacteria": RED, "Actinobacteria": AMBER,
9            "Verrucomicrobia": PURPLE, "Other": GREY}
10 days = wide.index.values                             # days on x
11
12 fig, ax = plt.subplots(figsize=(6.8, 4.4))            # one panel
13 bottom = np.zeros(len(wide))                          # running base
14 for t in taxa:                                       # layer in abundar
15     ax.bar(days, wide[t], bottom=bottom, color=tax_pal[t],
16           width=1.0, linewidth=0, label=t)
17     bottom += wide[t].values                         # raise base
18 ax.set_xlim(days.min() - 0.5, days.max() + 0.5)
19 ax.set_ylim(0, 1)
20 ax.set_xlabel("day"); ax.set_ylabel("relative abundance")
21 plt.show()
```



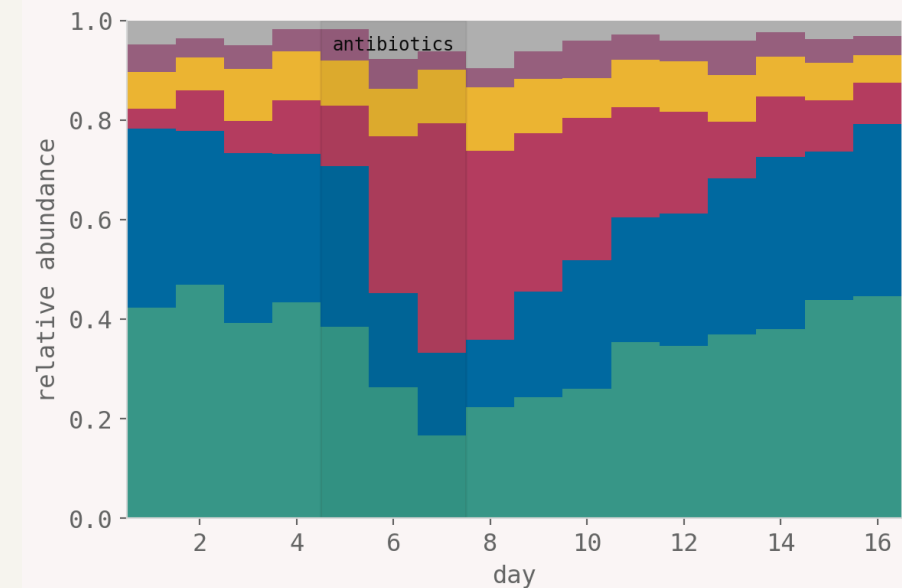
Stacked Bars • Step 3 — Mark the Event

METAGENOMICS • BUILD

3 • Put the cause next to the effect

Shade the antibiotic window and label it. Now the Proteobacteria swell sits directly under its trigger, the bloom reads as a *consequence*, not a coincidence.

```
1 from genomics_course.data import load_microbiome
2
3 mb = load_microbiome()                                # long form
4 wide = mb.pivot(index="day", columns="taxon", values="abundance") # days x taxa
5 taxa = (mb.groupby("taxon").abundance.mean()           # taxa, most abund
6         .sort_values(ascending=False).index)
7 tax_pal = {"Firmicutes": GREEN, "Bacteroidetes": BLUE, # fixed colour per
8            "Proteobacteria": RED, "Actinobacteria": AMBER,
9            "Verrucomicrobia": PURPLE, "Other": GREY}
10 days = wide.index.values
11
12 fig, ax = plt.subplots(figsize=(6.8, 4.4))
13 bottom = np.zeros(len(wide))
14 for t in taxa:                                         # stack the taxa
15     ax.bar(days, wide[t], bottom=bottom, color=tax_pal[t],
16            width=1.0, linewidth=0, label=t)
17     bottom += wide[t].values
18 ax.axvspan(4.5, 7.5, color=INK, alpha=.06)           # shade the antibiotic
19 ax.text(6, 0.94, "antibiotics", ha="center", color=INK, fontsize=9) # name the event
20 ax.set_xlim(days.min() - 0.5, days.max() + 0.5)
21 ax.set_ylim(0, 1)
22 ax.set_xlabel("day"); ax.set_ylabel("relative abundance")
```



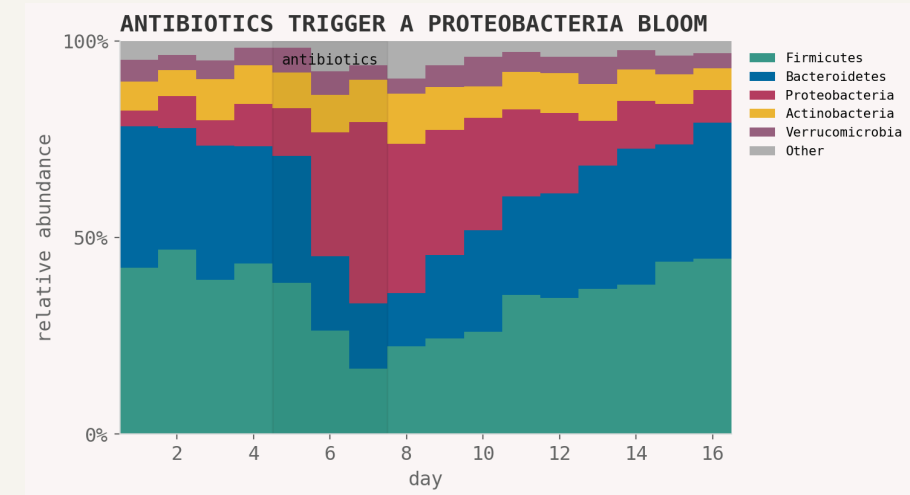
Stacked Bars • Step 4 — Percent Axis & Finding

METAGENOMICS • BUILD

4 • Name the result

Add the taxon legend, a percent y-axis, and a title that states the biology. The story reads in one glance: bloom under antibiotics, recovery after.

```
1 import matplotlib.ticker as mtick # percent-axis f
2 from genomics_course.data import load_microbiome
3
4 mb = load_microbiome() # long form
5 wide = mb.pivot(index="day", columns="taxon", values="abundance") # days x taxa
6 taxa = (mb.groupby("taxon").abundance.mean() # taxa, most abun
7         .sort_values(ascending=False).index)
8 tax_pal = {"Firmicutes": GREEN, "Bacteroidetes": BLUE, # fixed colour per
9            "Proteobacteria": RED, "Actinobacteria": AMBER,
10            "Verrucomicrobia": PURPLE, "Other": GREY}
11 days = wide.index.values
12
13 fig, ax = plt.subplots(figsize=(6.8, 4.4))
14 bottom = np.zeros(len(wide))
15 for t in taxa: # stack the taxa
16     ax.bar(days, wide[t], bottom=bottom, color=tax_pal[t],
17           width=1.0, linewidth=0, label=t)
18     bottom += wide[t].values
19 ax.axvspan(4.5, 7.5, color=INK, alpha=.06) # antibiotic win
20 ax.text(6, 0.94, "antibiotics", ha="center", color=INK, fontsize=9)
21 ax.set_xlim(days.min() - 0.5, days.max() + 0.5)
22 ax.set_ylim(0, 1) # ax.set_yticks([0, .5, 1]) # clean 0 / 50 / 1
```



THE BOTTOM LINE

Along time, a stacked bar shows the community moving: **pivot to days×taxa → order taxa by abundance → mark the event → percent axis + finding**. Fixed colours and band order let the eye track one taxon across the whole course, the Proteobacteria bloom rises under antibiotics and fades in recovery.

Exercise • Your Turn

METAGENOMICS • EXERCISE

Measure the bloom

CODING EXERCISE

The eye sees the bloom; now quantify it. Find Proteobacteria's baseline vs peak abundance and the day it peaks.

```
1 from genomics_course.data import load_microbiome
2
3 mb = load_microbiome()
4
5 # 1. proteo = mb[mb.taxon == "Proteobacteria"]
6 # 2. baseline mean = proteo[proteo.phase == "baseline"].abundance.mean()
7 # 3. peak day      = proteo.loc[proteo.abundance.idxmax(), "day"]
8 # 4. report fold-change peak / baseline, and annotate the peak day on the stacked bar
```

Treemap • The Data

COMPOSITION • TREEMAP

Collapse time to one community

The stacked bar showed every day. Average across them and you get the whole community as one set of parts, exactly what a treemap wants.

```
1 from genomics_course.data import load_microbiome
2
3 mb = load_microbiome()                # day, phase, taxon,
4 comm = (mb.groupby("taxon").abundance.mean()    # mean abundance acr
5         .sort_values(ascending=False).reset_index()) # biggest first
6 comm["pct"] = comm.abundance * 100           # as a percentage for
7 comm
```

	taxon	abundance	pct
0	Firmicutes	0.350030	35.003037
1	Bacteroidetes	0.275609	27.560873
2	Proteobacteria	0.189069	18.906862
3	Actinobacteria	0.090760	9.075969
4	Verrucomicrobia	0.050794	5.079387
5	Other	0.043739	4.373871

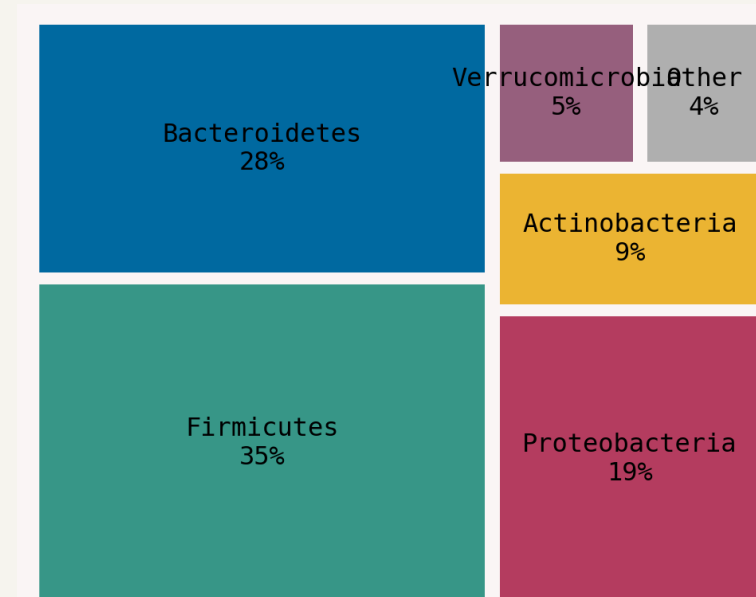
Treemap • Step 1 — Tile by Value

COMPOSITION • BUILD

1 • squarify

`squarify.plot` takes sizes and labels and tiles them largest-first. `pad=True` leaves thin gaps so tiles read as separate. Turn the axis off, it's a canvas, not a plot.

```
1 import squarify                                     # the treemap layout
2 from genomics_course.data import load_microbiome
3
4 mb = load_microbiome()                               # long form
5 comm = (mb.groupby("taxon").abundance.mean()         # community = mean
6         .sort_values(ascending=False).reset_index()) # biggest first (squ
7 comm["pct"] = comm.abundance * 100                  # percentages for th
8 labels = [f"{t}\n{p:.0f}%" for t, p in zip(comm.taxon, comm.pct)] # "Firmicutes\n33
9
10 fig, ax = plt.subplots(figsize=(6.2, 5.0)); ax.set_axis_off() # blank canvas, no a
11 squarify.plot(sizes=comm.abundance, label=labels, pad=True,    # tile area == abund
12               color=[GREEN, BLUE, RED, AMBER, PURPLE, GREY], ax=ax) # explicit taxo
13 plt.show()
```



Treemap · Step 2 — Palette from pypalettes

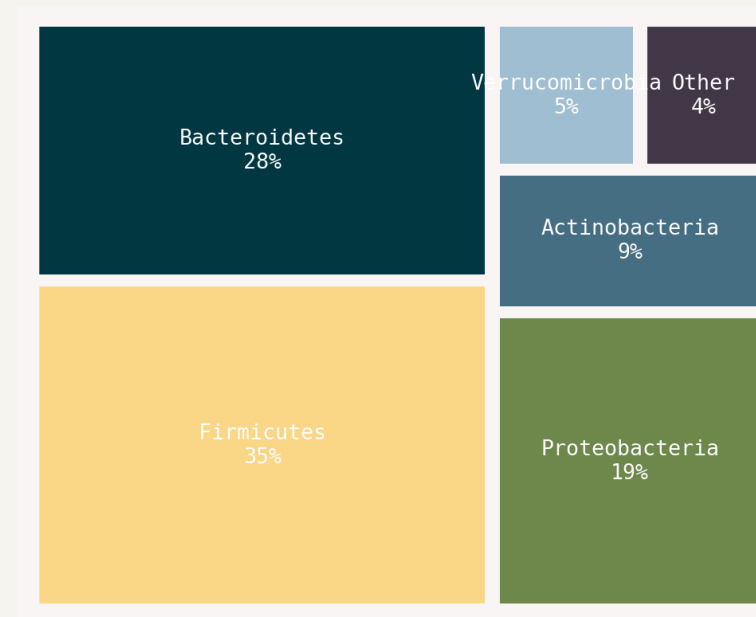
COMPOSITION · BUILD

2 · Borrow a curated palette

`pypalettes` ships hundreds of named palettes.

`load_cmap` fetches one; `pd.factorize` turns the category column into colour indices, so each taxon gets its own swatch. White label text reads cleanly on the darker tiles.

```
1 import squarify
2 from pypalettes import load_cmap          # curated palette 1
3 from genomics_course.data import load_microbiome
4
5 mb = load_microbiome()                   # long form
6 comm = (mb.groupby("taxon").abundance.mean() # community mean ov
7         .sort_values(ascending=False).reset_index())
8 comm["pct"] = comm.abundance * 100        # percentages
9 labels = [f"{t}\n{p:.0f}%" for t, p in zip(comm.taxon, comm.pct)] # tile labels
10
11 cmap = load_cmap("Acadia")               # any named palette
12 codes, _ = pd.factorize(comm.taxon)      # taxon → integer i
13 colors = [cmap(c) for c in codes]        # one palette colour
14
15 fig, ax = plt.subplots(figsize=(6.2, 5.0)); ax.set_axis_off() # blank canvas
16 squarify.plot(sizes=comm.abundance, label=labels, color=colors, pad=True, # tiles i
17               text_kwargs={"color": "white", "fontsize": 10}, ax=ax) # white l
18 plt.show()
```



Exercise • Your Turn

COMPOSITION • EXERCISE

Treemap the assay catalogue

CODING EXERCISE

`load_assay_counts()` lists how many public datasets exist per assay type. Build a labelled treemap with a `pypalettes` palette of your choice, and title it after the leading assay.

```
1 import squarify
2 from pypalettes import load_cmap
3 from genomics_course.data import load_assay_counts
4
5 ac = load_assay_counts().sort_values("n_datasets", ascending=False) # assay, n_datasets
6
7 # 1. labels like "RNA-seq\n342"
8 # 2. cmap = load_cmap("<pick one>"); codes, _ = pd.factorize(ac.assay)
9 # 3. squarify.plot(sizes=ac.n_datasets, ..., pad=True, text_kwargs={"color": "white"})
10 # 4. title that names the leader
```


Admixture • The Data

POPULATION STRUCTURE

One row per individual, ancestry split across K populations

`load_admixture()` returns each individual's inferred ancestry across five source populations (`K1` – `K5`, summing to 1) plus its assigned population. An admixture plot turns each row into one stacked bar.

```
1 from genomics_course.data import load_admixture
2
3 adm = load_admixture() # individual, population, K1..K5
4 adm.head(4)
```

	individual	population	K1	K2	K3	
0	EUR_000	EUR	0.978699	0.002754	0.000058	0.000489
1	EUR_001	EUR	0.978085	0.011594	0.000160	0.000160
2	EUR_002	EUR	0.977956	0.009453	0.000056	0.000489
3	EUR_003	EUR	0.971030	0.001839	0.025570	0.000489

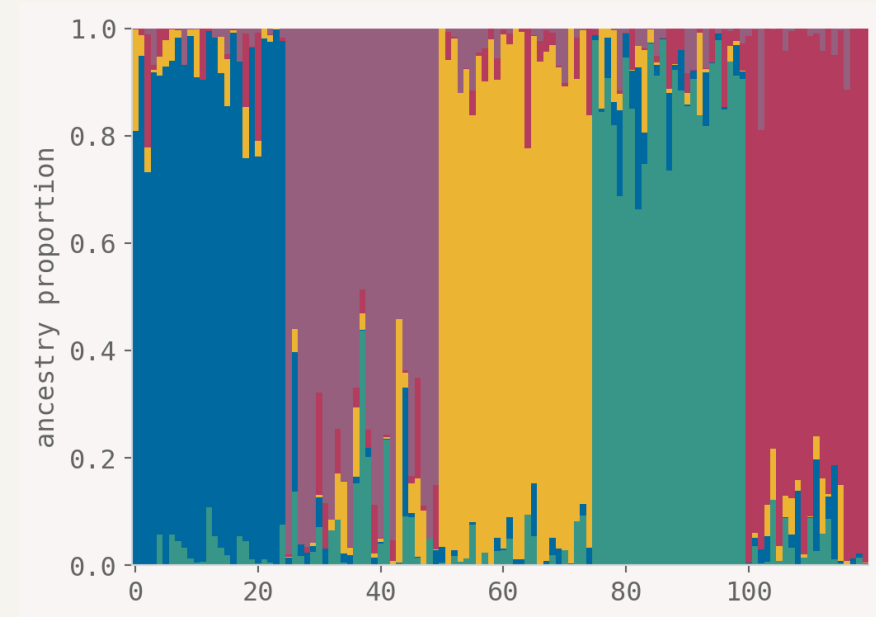
Admixture • Step 1 — Stack the Components

POPULATION STRUCTURE • BUILD

1 • Stacked bars

One bar per individual, each K segment stacked on the last via a running `bottom`. With `width=1.0` and `linewidth=0` the bars touch into a solid block. Within each population the order is still random, so it looks noisy.

```
1 from genomics_course.data import load_admixture
2
3 adm = load_admixture()                                # individual, popu
4 adm = adm.sample(frac=1, random_state=1).sort_values("population", kind="stable").re  # the five ancestr
5 Kc = ["K1", "K2", "K3", "K4", "K5"]                  # one colour per a
6 pal = [GREEN, BLUE, AMBER, RED, PURPLE]
7
8 fig, ax = plt.subplots(figsize=(6.0, 4.4))            # one panel
9 x = np.arange(len(adm)); bottom = np.zeros(len(adm))  # x = individuals,
10 for k, col in enumerate(Kc):                          # stack one K comp
11     ax.bar(x, adm[col], bottom=bottom, color=pal[k], width=1.0, linewidth=0, label=c  # raise base for t
12     bottom += adm[col].values
13 ax.set_xlim(-0.5, len(adm) - 0.5); ax.set_ylim(0, 1) # tight x, full 0-1
14 ax.set_ylabel("ancestry proportion")
15 plt.show()
```



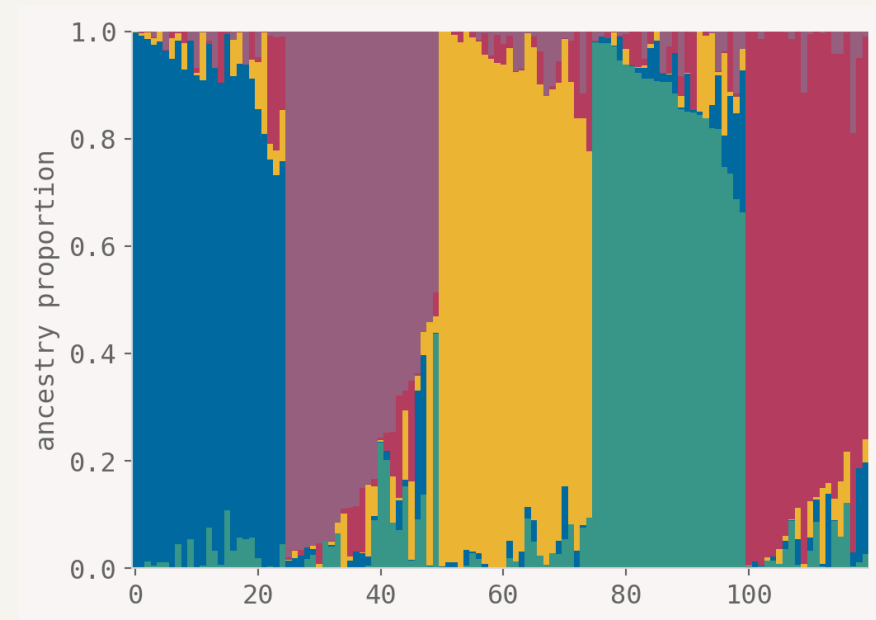
Admixture • Step 2 — Sort Within Population

POPULATION STRUCTURE • BUILD

2 • Order

For each population, find its dominant ancestry and sort its individuals by that column. The noise resolves into a clean gradient, the single most important step for a readable admixture plot.

```
1 from genomics_course.data import load_admixture
2
3 adm = load_admixture()                                # individual, pc
4 adm = adm.sample(frac=1, random_state=1).sort_values("population", kind="stable").
5 Kc = ["K1", "K2", "K3", "K4", "K5"]                  # ancestry column
6 pal = [GREEN, BLUE, AMBER, RED, PURPLE]               # ancestry color
7
8 parts = []                                             # collect each s
9 for p in adm.population.drop_duplicates():             # for each popu
10     block = adm[adm.population == p]                  # just that popu
11     dom = block[Kc].mean().idxmax()                   # its dominant ar
12     parts.append(block.sort_values(dom, ascending=False)) # sort individual
13 adm = pd.concat(parts, ignore_index=True)             # stitch the sort
14
15 fig, ax = plt.subplots(figsize=(6.0, 4.4))           # one panel
16 x = np.arange(len(adm)); bottom = np.zeros(len(adm)) # x = individuals
17 for k, col in enumerate(Kc):                          # stack the K cc
18     ax.bar(x, adm[col], bottom=bottom, color=pal[k], width=1.0, linewidth=0, label=
19     bottom += adm[col].values                         # raise base
20 ax.set_xlim(-0.5, len(adm) - 0.5); ax.set_ylim(0, 1)
21 ax.set_ylabel("ancestry proportion")
22 plt.show()
```



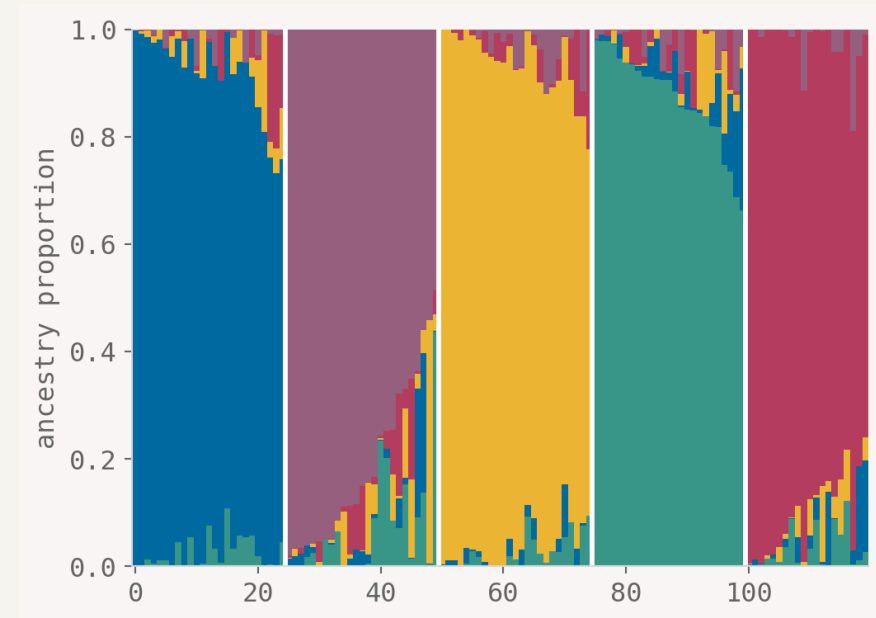
Admixture • Step 3 — Separate the Populations

POPULATION STRUCTURE • BUILD

3 • White lines

Find where one population ends and the next begins (cumulative group sizes) and draw a thin white line at each boundary. Now the five groups read as distinct blocks.

```
1 from genomics_course.data import load_admixture
2
3 adm = load_admixture()                                # individual, pc
4 adm = adm.sample(frac=1, random_state=1).sort_values("population", kind="stable").
5 Kc = ["K1", "K2", "K3", "K4", "K5"]                  # ancestry column
6 pal = [GREEN, BLUE, AMBER, RED, PURPLE]              # ancestry color
7
8 parts = []                                             # sort within each
9 for p in adm.population.drop_duplicates():
10     block = adm[adm.population == p]
11     dom = block[Kc].mean().idxmax()
12     parts.append(block.sort_values(dom, ascending=False))
13 adm = pd.concat(parts, ignore_index=True)
14
15 pops = adm.population.drop_duplicates().tolist()      # populations in
16 sizes = adm.population.value_counts()[pops].values   # how many indivi
17 bounds = np.cumsum(sizes)                             # cumulative edge
18
19 fig, ax = plt.subplots(figsize=(6.0, 4.4))           # one panel
20 x = np.arange(len(adm)); bottom = np.zeros(len(adm))
21 for k, col in enumerate(Kc):                          # stack the K cc
22     ax.bar(x, adm[col], bottom=bottom, color=pal[k], width=1.0, linewidth=0, label=
```



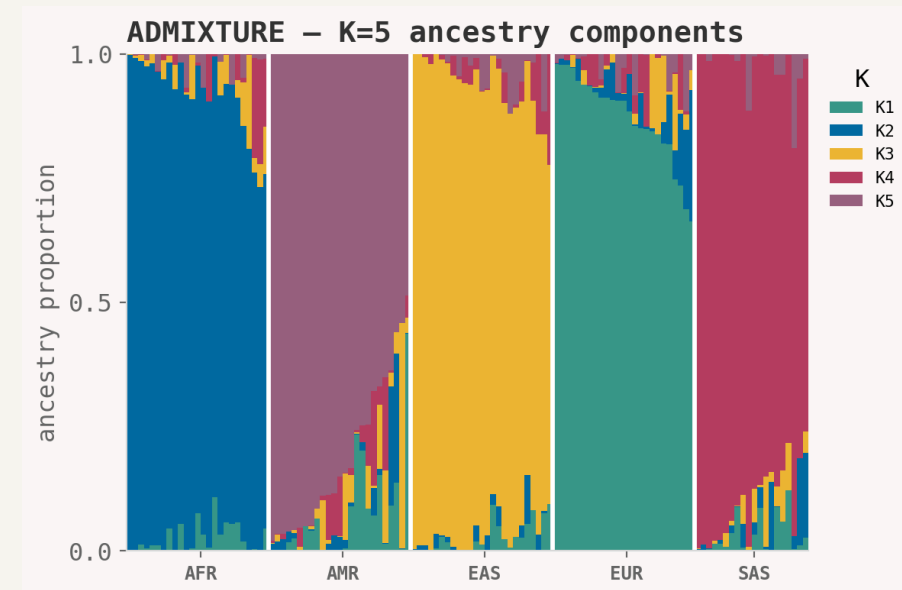
Admixture • Step 4 — Label & Polish

POPULATION STRUCTURE • BUILD

4 • Read it

Put one tick at the centre of each population block, label it, add the K legend and a title. The figure now reads itself: who is mostly one ancestry, who is admixed.

```
1 from genomics_course.data import load_admixture
2
3 adm = load_admixture()                                # individual, pc
4 adm = adm.sample(frac=1, random_state=1).sort_values("population", kind="stable").
5 Kc = ["K1", "K2", "K3", "K4", "K5"]                    # ancestry column
6 pal = [GREEN, BLUE, AMBER, RED, PURPLE]                # ancestry color
7
8 parts = []                                             # sort within ea
9 for p in adm.population.drop_duplicates():
10     block = adm[adm.population == p]
11     dom = block[Kc].mean().idxmax()
12     parts.append(block.sort_values(dom, ascending=False))
13 adm = pd.concat(parts, ignore_index=True)
14
15 pops = adm.population.drop_duplicates().tolist()       # population or c
16 sizes = adm.population.value_counts()[pops].values    # block sizes
17 bounds = np.cumsum(sizes)                             # cumulative boi
18
19 fig, ax = plt.subplots(figsize=(6.0, 4.4))            # one panel
20 x = np.arange(len(adm)); bottom = np.zeros(len(adm))
21 for k, col in enumerate(Kc):                          # stack the K cc
22     ax.bar(x, adm[col], bottom=bottom, color=pal[k], width=1.0, linewidth=0, label=
```



THE BOTTOM LINE

Each bar is one individual, each segment an ancestry fraction. The recipe: **stack → sort within population → separate with white lines → label**. Sorting is what turns noise into structure, AMR shows the most admixture, the rest are largely single-ancestry.

Admixture • The Data

POPULATION STRUCTURE

One row per individual, ancestry split across K populations

`load_admixture()` returns each individual's inferred ancestry across five source populations (`K1` – `K5`, summing to 1) plus its assigned population. An admixture plot turns each row into one stacked bar.

```
1 from genomics_course.data import load_admixture
2
3 adm = load_admixture() # individual, population, K1..K5
4 adm.head(4)
```

	individual	population	K1	K2	K3	
0	EUR_000	EUR	0.978699	0.002754	0.000058	0.000499
1	EUR_001	EUR	0.978085	0.011594	0.000160	0.000160
2	EUR_002	EUR	0.977956	0.009453	0.000056	0.000495
3	EUR_003	EUR	0.971030	0.001839	0.025570	0.000495

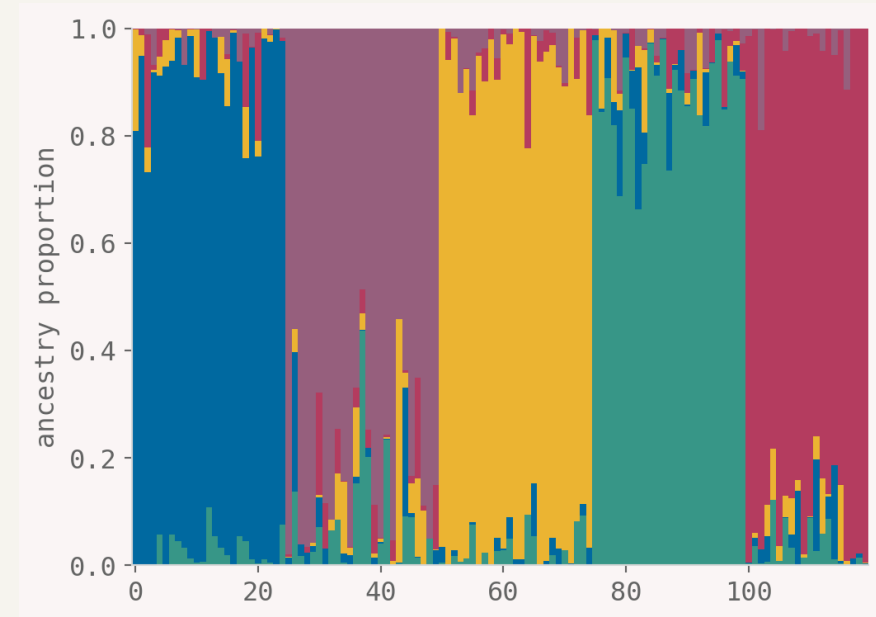
Admixture • Step 1 — Stack the Components

POPULATION STRUCTURE • BUILD

1 • Stacked bars

One bar per individual, each K segment stacked on the last via a running `bottom`. With `width=1.0` and `linewidth=0` the bars touch into a solid block. Within each population the order is still random, so it looks noisy.

```
1 from genomics_course.data import load_admixture
2
3 adm = load_admixture()                                # individual, popu
4 adm = adm.sample(frac=1, random_state=1).sort_values("population", kind="stable").re  # the five ancestr
5 Kc = ["K1", "K2", "K3", "K4", "K5"]                  # one colour per a
6 pal = [GREEN, BLUE, AMBER, RED, PURPLE]
7
8 fig, ax = plt.subplots(figsize=(6.0, 4.4))            # one panel
9 x = np.arange(len(adm)); bottom = np.zeros(len(adm))  # x = individuals,
10 for k, col in enumerate(Kc):                          # stack one K comp
11     ax.bar(x, adm[col], bottom=bottom, color=pal[k], width=1.0, linewidth=0, label=c
12     bottom += adm[col].values                          # raise base for t
13 ax.set_xlim(-0.5, len(adm) - 0.5); ax.set_ylim(0, 1) # tight x, full 0-1
14 ax.set_ylabel("ancestry proportion")
15 plt.show()
```



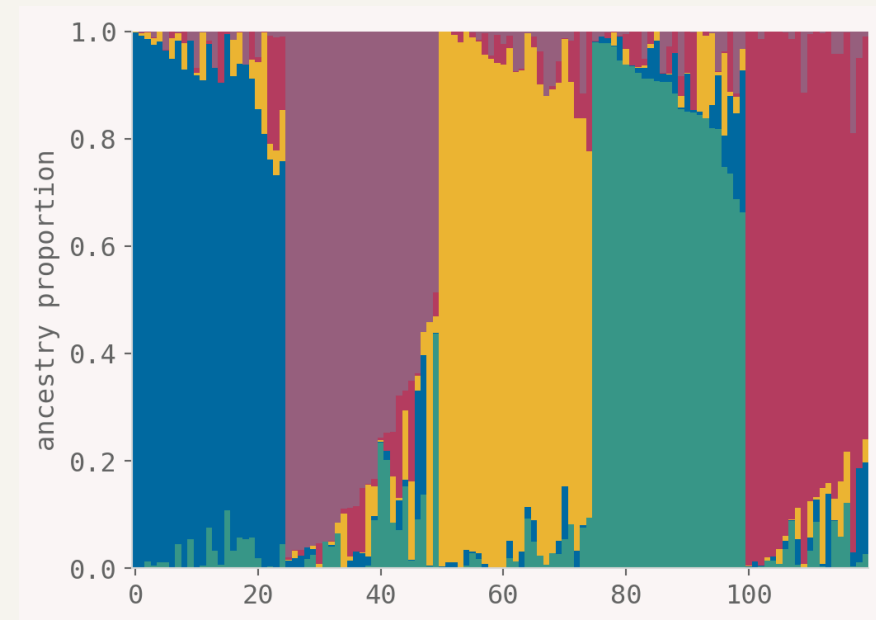
Admixture • Step 2 — Sort Within Population

POPULATION STRUCTURE • BUILD

2 • Order

For each population, find its dominant ancestry and sort its individuals by that column. The noise resolves into a clean gradient, the single most important step for a readable admixture plot.

```
1 from genomics_course.data import load_admixture
2
3 adm = load_admixture()                                # individual, pc
4 adm = adm.sample(frac=1, random_state=1).sort_values("population", kind="stable").
5 Kc = ["K1", "K2", "K3", "K4", "K5"]                  # ancestry column
6 pal = [GREEN, BLUE, AMBER, RED, PURPLE]               # ancestry color
7
8 parts = []                                             # collect each s
9 for p in adm.population.drop_duplicates():             # for each popu
10     block = adm[adm.population == p]                  # just that popu
11     dom = block[Kc].mean().idxmax()                    # its dominant ar
12     parts.append(block.sort_values(dom, ascending=False)) # sort individual
13 adm = pd.concat(parts, ignore_index=True)              # stitch the sort
14
15 fig, ax = plt.subplots(figsize=(6.0, 4.4))            # one panel
16 x = np.arange(len(adm)); bottom = np.zeros(len(adm)) # x = individuals
17 for k, col in enumerate(Kc):                           # stack the K cc
18     ax.bar(x, adm[col], bottom=bottom, color=pal[k], width=1.0, linewidth=0, label=
19     bottom += adm[col].values                          # raise base
20 ax.set_xlim(-0.5, len(adm) - 0.5); ax.set_ylim(0, 1)
21 ax.set_ylabel("ancestry proportion")
22 plt.show()
```



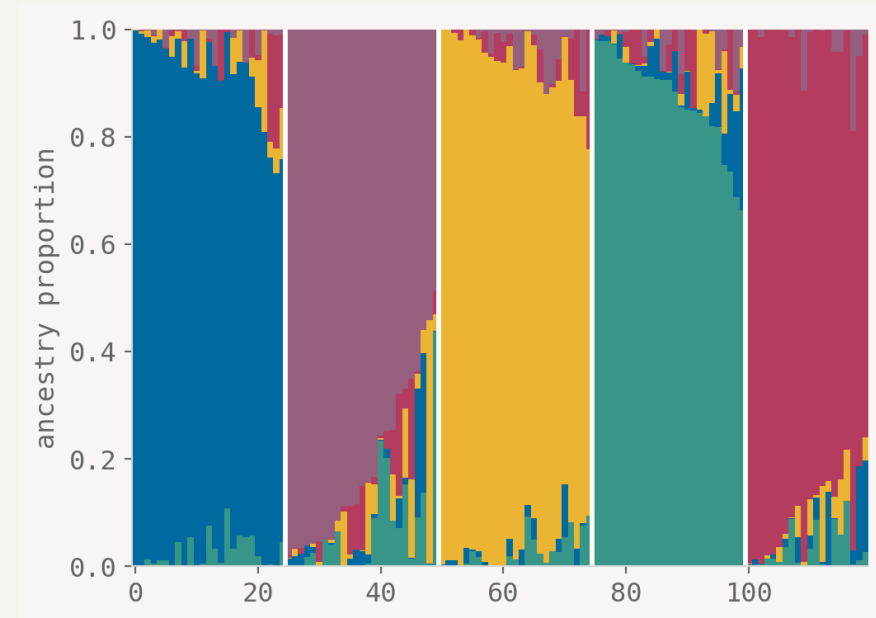
Admixture • Step 3 — Separate the Populations

POPULATION STRUCTURE • BUILD

3 • White lines

Find where one population ends and the next begins (cumulative group sizes) and draw a thin white line at each boundary. Now the five groups read as distinct blocks.

```
1 from genomics_course.data import load_admixture
2
3 adm = load_admixture()                                # individual, pc
4 adm = adm.sample(frac=1, random_state=1).sort_values("population", kind="stable").
5 Kc = ["K1", "K2", "K3", "K4", "K5"]                  # ancestry column
6 pal = [GREEN, BLUE, AMBER, RED, PURPLE]               # ancestry color
7
8 parts = []                                             # sort within each
9 for p in adm.population.drop_duplicates():
10     block = adm[adm.population == p]
11     dom = block[Kc].mean().idxmax()
12     parts.append(block.sort_values(dom, ascending=False))
13 adm = pd.concat(parts, ignore_index=True)
14
15 pops = adm.population.drop_duplicates().tolist()      # populations in
16 sizes = adm.population.value_counts()[pops].values   # how many indivi
17 bounds = np.cumsum(sizes)                             # cumulative edge
18
19 fig, ax = plt.subplots(figsize=(6.0, 4.4))           # one panel
20 x = np.arange(len(adm)); bottom = np.zeros(len(adm))
21 for k, col in enumerate(Kc):                          # stack the K cc
22     ax.bar(x, adm[col], bottom=bottom, color=pal[k], width=1.0, linewidth=0, label=
```



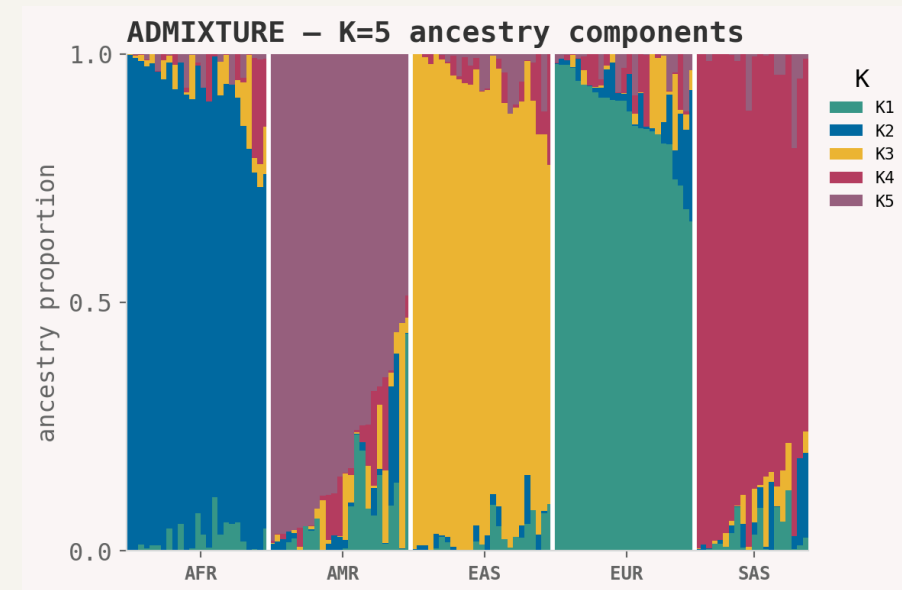
Admixture • Step 4 — Label & Polish

POPULATION STRUCTURE • BUILD

4 • Read it

Put one tick at the centre of each population block, label it, add the K legend and a title. The figure now reads itself: who is mostly one ancestry, who is admixed.

```
1 from genomics_course.data import load_admixture
2
3 adm = load_admixture()                                # individual, pc
4 adm = adm.sample(frac=1, random_state=1).sort_values("population", kind="stable").
5 Kc = ["K1", "K2", "K3", "K4", "K5"]                  # ancestry column
6 pal = [GREEN, BLUE, AMBER, RED, PURPLE]              # ancestry color
7
8 parts = []                                             # sort within ea
9 for p in adm.population.drop_duplicates():
10     block = adm[adm.population == p]
11     dom = block[Kc].mean().idxmax()
12     parts.append(block.sort_values(dom, ascending=False))
13 adm = pd.concat(parts, ignore_index=True)
14
15 pops = adm.population.drop_duplicates().tolist()      # population or c
16 sizes = adm.population.value_counts()[pops].values  # block sizes
17 bounds = np.cumsum(sizes)                            # cumulative boi
18
19 fig, ax = plt.subplots(figsize=(6.0, 4.4))           # one panel
20 x = np.arange(len(adm)); bottom = np.zeros(len(adm))
21 for k, col in enumerate(Kc):                         # stack the K cc
22     ax.bar(x, adm[col], bottom=bottom, color=pal[k], width=1.0, linewidth=0, label=
```



THE BOTTOM LINE

Each bar is one individual, each segment an ancestry fraction. The recipe: **stack** → **sort within population** → **separate with white lines** → **label**. Sorting is what turns noise into structure, AMR shows the most admixture, the rest are largely single-ancestry.

Heatmaps

EXPRESSION MATRICES · DAY 01

What a Heatmap Shows

EXPRESSION HEATMAPS · THEORY

Colour, clustering, annotation

A heatmap turns a matrix into a grid of coloured cells: gene × sample expression, a sample-by-sample correlation matrix, an epigenomic signal. Colour encodes the value, so the choice of map matters, **diverging** (e.g. `RdBu_r`, centred at 0) for z-scores that go both ways, sequential for one-directional quantities.

Two moves make it readable. **Z-score each gene** so a quietly-expressed gene and a loud one are on the same scale (colour then means “high/low *for this gene*”). Then **cluster** rows and columns so similar genes and similar samples sit together, the dendrograms reorder the matrix until the structure appears.

`seaborn.clustermap` does both with good defaults;

`PyComplexHeatmap` adds rich annotation tracks.

Heatmap • The Data

EXPRESSION HEATMAPS

A real expression matrix + its sample metadata

`load_expression()` is a log2 matrix, genes down the rows, samples across the columns.

`load_expression_meta()` is the matching per-sample table (condition, batch, QC), the annotation we'll layer on top.

```
1 from genomics_course.data import load_expression, load_expression_meta
2
3 expr = load_expression()           # 40 genes × 24 samples
4 meta = load_expression_meta()      # sample, condition, batch, qc
5 expr.iloc[:3, :4]
```

	C01	C02	C03	C04
gene				
G000	8.056462	8.033121	7.388702	8.039300
G001	8.009095	8.200648	8.748127	8.871357
G002	8.308926	8.052406	7.713790	8.060053

Heatmap • Step 1 — Z-Score & Colour

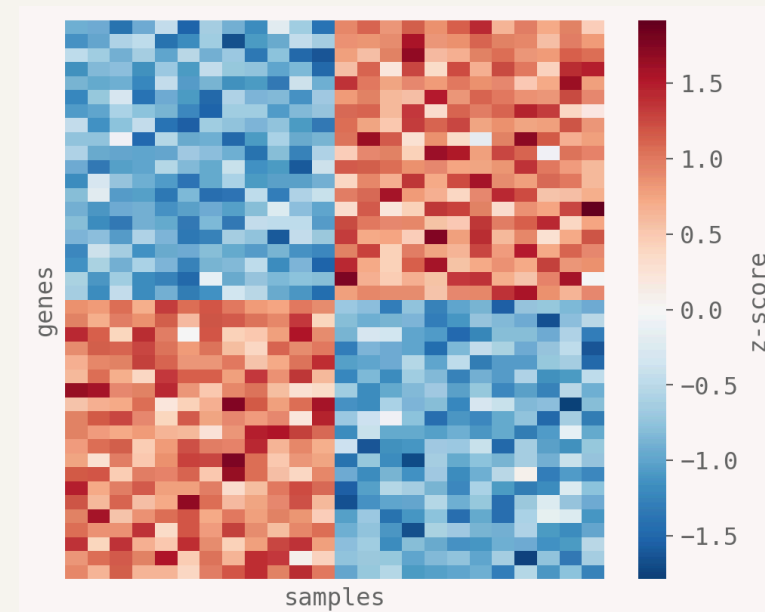
EXPRESSION HEATMAPS • BUILD

1 • Put every gene on one scale

Z-score each gene (row), so colour means high or low *relative to that gene's own mean*. Drop it into

`sns.heatmap` with a diverging map centred at 0. The blocks are faintly there, but the rows and columns are still in file order.

```
1 import seaborn as sns                                # heatmap / clustermap 1
2 from genomics_course.data import load_expression
3
4 expr = load_expression()                             # genes × samples (log2)
5 z = expr.sub(expr.mean(1), axis=0).div(expr.std(1), axis=0) # z-score each gene: mea
6
7 fig, ax = plt.subplots(figsize=(6.0, 5.0))           # one panel
8 sns.heatmap(z, cmap="RdBu_r", center=0, ax=ax,       # diverging map, white a
9             cbar_kws={"label": "z-score"},           # colourbar legend
10            xticklabels=False, yticklabels=False)     # 40×24 labels would be
11 ax.set_xlabel("samples"); ax.set_ylabel("genes")
12 plt.show()
```



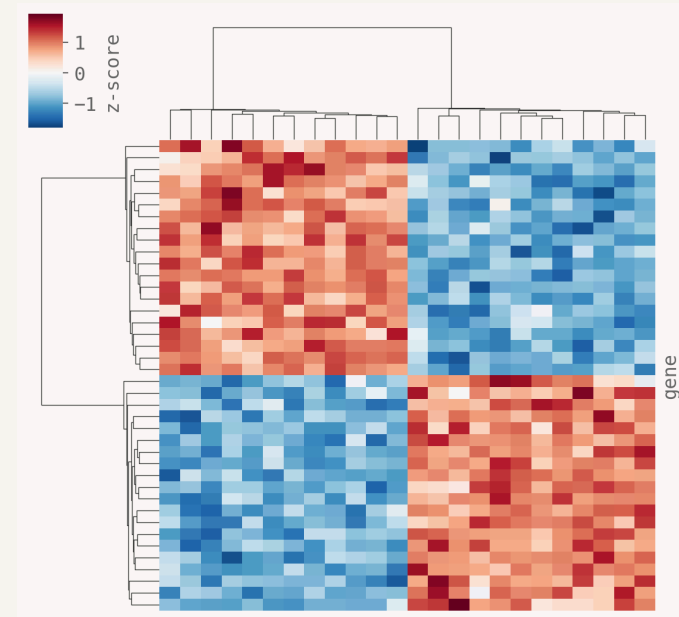
Heatmap • Step 2 — Cluster It

EXPRESSION HEATMAPS • BUILD

2 • Let the structure sort itself

Swap `heatmap` for `clustermap`. It hierarchically clusters rows and columns and reorders both, adding dendrograms. Similar genes pull together, similar samples pull together, and the two modules and two conditions snap into clean blocks.

```
1 import seaborn as sns
2 from genomics_course.data import load_expression
3
4 expr = load_expression() # genes x samples
5 z = expr.sub(expr.mean(1), axis=0).div(expr.std(1), axis=0) # per-gene z-score
6
7 cg = sns.clustermap(z, cmap="RdBu_r", center=0, # cluster rows AND column
8                   figsize=(6.0, 5.5), xticklabels=False, yticklabels=False,
9                   cbar_kws={"label": "z-score"}) # dendrograms added auto
10 plt.show()
```



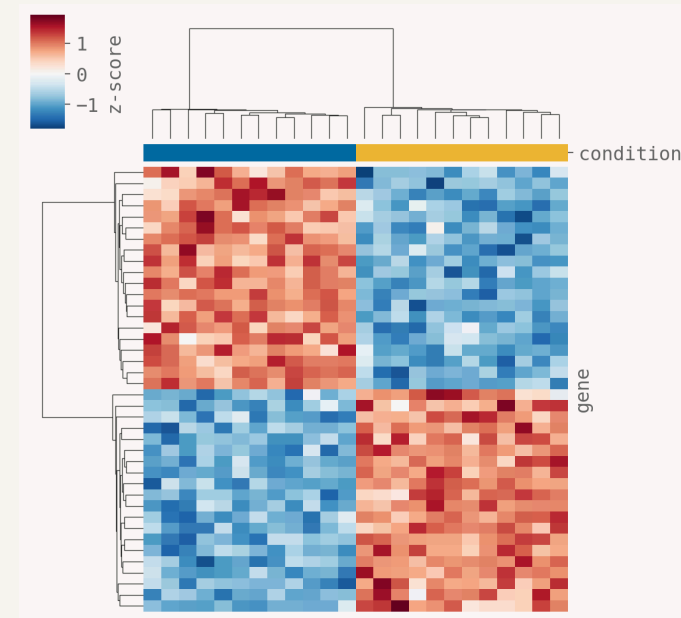
Heatmap • Step 3 — Annotate the Samples

EXPRESSION HEATMAPS • BUILD

3 • Bring in the metadata

Map each sample's condition to a colour and pass it as `col_colors`. A strip appears above the columns, so you can check that the clusters actually recover the biology, not a batch artefact.

```
1 import seaborn as sns
2 from genomics_course.data import load_expression, load_expression_meta
3
4 expr = load_expression()                # genes × samples
5 meta = load_expression_meta()           # sample, condition, bat
6 z = expr.sub(expr.mean(1), axis=0).div(expr.std(1), axis=0) # per-gene z-score
7
8 cond_pal = {"Control": BLUE, "Drought": AMBER} # one colour per conditi
9 col_colors = meta.set_index("sample")["condition"].map(cond_pal) # colour for each
10
11 cg = sns.clustermap(z, cmap="RdBu_r", center=0, figsize=(6.0, 5.5),
12                     col_colors=col_colors, # condition strip above
13                     xticklabels=False, yticklabels=False, cbar_kws={"label": "z-scor
14 plt.show()
```



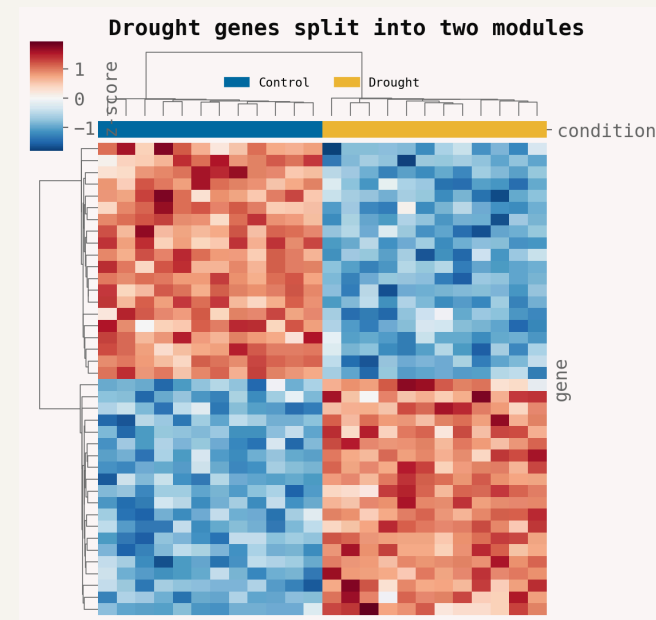
Heatmap • Step 4 — Polish & Title

EXPRESSION HEATMAPS • BUILD

4 • Slim the trees, add a key, name the finding

Thin the dendrograms, mute their colour, add a legend for the annotation strip, and title with the result. The figure now reads itself: two gene modules, cleanly split by condition.

```
1 import seaborn as sns
2 from matplotlib.patches import Patch # to build the condition legend
3 from genomics_course.data import load_expression, load_expression_meta
4
5 expr = load_expression() # genes x samples
6 meta = load_expression_meta() # metadata
7 z = expr.sub(expr.mean(1), axis=0).div(expr.std(1), axis=0) # per-gene z-score
8
9 cond_pal = {"Control": BLUE, "Drought": AMBER} # condition colours
10 col_colors = meta.set_index("sample")["condition"].map(cond_pal) # per-column color
11
12 cg = sns.clustermap(z, cmap="RdBu_r", center=0, figsize=(6.0, 5.5),
13                    col_colors=col_colors, xticklabels=False, yticklabels=False,
14                    cbar_kws={"label": "z-score"},
15                    dendrogram_ratio=(.12, .12), # slimmer dendrograms
16                    tree_kws={"colors": MUTED, "linewidths": .6}) # muted tree lines
17 handles = [Patch(color=c, label=k) for k, c in cond_pal.items()] # legend swatches
18 cg.ax_col_dendrogram.legend(handles=handles, ncol=2, # put the key over the
19                             frameon=False, loc="center", fontsize=8)
20 cg.fig.suptitle("Drought genes split into two modules", # title states the finding
21                fontweight="bold", color=INK, y=1.02)
22 plt.show()
```



THE BOTTOM LINE

The recipe: **z-score each gene** → **cluster rows and columns** → **annotate the samples** → **title the finding**. Z-scoring makes genes comparable; clustering reorders the matrix until structure appears; the annotation strip proves the clusters are biology, not batch.

Level Up · PyComplexHeatmap

EXPRESSION HEATMAPS · ADVANCED

Several annotation tracks at once

For publication figures with stacked tracks (categorical strips + numeric mini-barplots), `PyComplexHeatmap` mirrors R's `ComplexHeatmap`. One `HeatmapAnnotation` holds every track.

```
1 # pip install PyComplexHeatmap
2 from PyComplexHeatmap import HeatmapAnnotation, anno_simple, anno_barplot, ClusterMapPlotter
3 from genomics_course.data import load_expression, load_expression_meta
4
5 expr = load_expression()
6 meta = load_expression_meta().set_index("sample") # align metadata to matrix columns
7 z = expr.sub(expr.mean(1), axis=0).div(expr.std(1), axis=0) # per-gene z-score
8
9 col_ha = HeatmapAnnotation( # one object, several stacked tracks
10     Condition=anno_simple(meta["condition"], # categorical strip
11         colors={"Control": BLUE, "Drought": AMBER}),
12     Batch=anno_simple(meta["batch"]), # second categorical strip
13     QC=anno_barplot(meta["qc"], colors=GREEN, height=12), # numeric mini-barplot
14     axis=1, verbose=0)
15
16 ClusterMapPlotter(data=z, top_annotation=col_ha, cmap="RdBu_r", # clustered heatmap + tracks
17     row_cluster=True, col_cluster=True, label="z-score",
18     show_rownames=False, show_colnames=False)
```

Exercise • Sample QC

EXPRESSION HEATMAPS • EXERCISE

Did the replicates behave?

REAL SCENARIO

Before trusting any analysis, check that replicates agree and nothing is mislabelled. Build a **sample × sample correlation heatmap**, annotate it by condition, and read whether the two groups form clean blocks, or whether a sample sits in the wrong one (a possible swap).

```
1 import seaborn as sns
2 from genomics_course.data import load_expression, load_expression_meta
3
4 expr = load_expression()
5 meta = load_expression_meta()
6
7 # 1. corr = expr.corr() # sample × sample Pearson matrix
8 # 2. col = meta.set_index("sample")["condition"].map({"Control": BLUE, "Drought": AMBER})
9 # 3. sns.clustermap(corr, cmap="rocket", row_colors=col, col_colors=col)
10 # 4. read it: two clean blocks? any sample correlating with the wrong group?
```

Exercise • Top Variable Genes

EXPRESSION HEATMAPS • EXERCISE

You can't cluster 20,000 genes

REAL SCENARIO

The standard first look at an RNA-seq matrix isn't every gene, it's the most *variable* ones, the genes carrying the signal. Select the top 20 by row variance and cluster just those.

```
1 import seaborn as sns
2 from genomics_course.data import load_expression
3
4 expr = load_expression()
5 z = expr.sub(expr.mean(1), axis=0).div(expr.std(1), axis=0)
6
7 # 1. top = z.var(axis=1).sort_values(ascending=False).head(20).index # 20 most variable genes
8 # 2. sns.clustermap(z.loc[top], cmap="RdBu_r", center=0, yticklabels=True)
9 # 3. do the same two modules still separate the conditions with only 20 genes?
```

Waffle & Waterfall

COHORT SUMMARIES · DAY 01

Waffle Charts

COHORT SUMMARIES • THEORY

Part-to-whole as a grid of squares

A waffle is a 10×10-ish grid where each square is one unit and each colour block is a category, more honest than a pie because you can *count* the squares. Use it for whole-number compositions: variant classifications, cohort cell types, genotype counts.

It works best with a handful of categories that sum to a round total. Here: the outcome of a clinical variant panel, where the punchline is how much grey (VUS) there is.

Waffle • The Data

COHORT SUMMARIES

Variant interpretations from a panel screen

`load_variant_classes()` is a tiny table: each ACMG category and how many variants fell into it. They sum to 150, a clean number to tile.

```
1 from genomics_course.data import load_variant_classes
2
3 vc = load_variant_classes() # classification, n
4 vc
```

	classification	n
0	Pathogenic	12
1	Likely pathogenic	18
2	VUS	55
3	Likely benign	24
4	Benign	41

Waffle • Step 1 — Tile the Counts

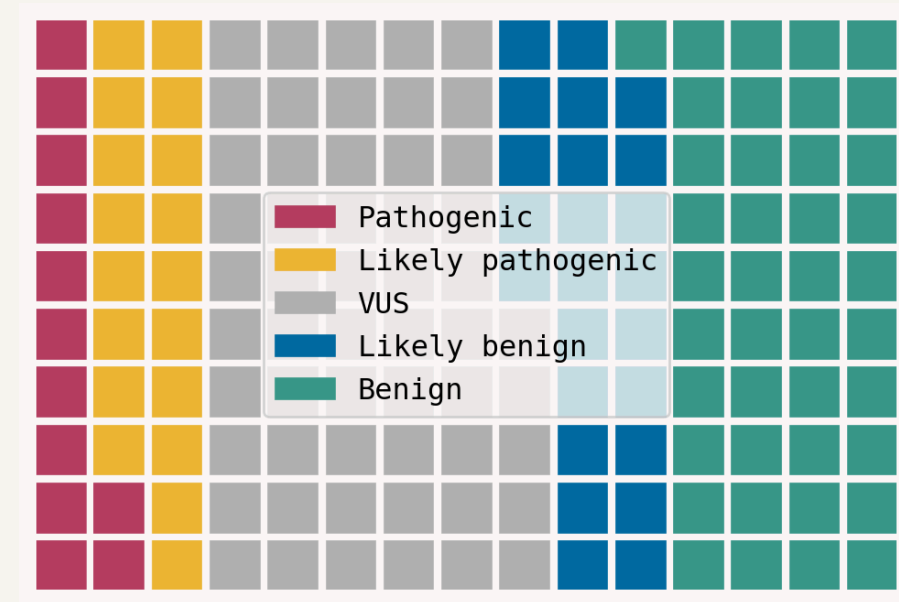
COHORT SUMMARIES • BUILD

1 • Counts to squares

PyWaffle plugs into matplotlib via

FigureClass=Waffle. Give it rows and a values dict of category → count; it lays one square per unit, block by block.

```
1 # pip install pywaffle
2 from pywaffle import Waffle
3 from genomics_course.data import load_variant_classes
4
5 vc = load_variant_classes()                # classification, n
6 pal = [RED, AMBER, GREY, BLUE, GREEN]     # colour per ACMG class
7 fig = plt.figure(FigureClass=Waffle, rows=10,      # 10 rows → 15 cols for 150
8                 values=dict(zip(vc.classification, vc.n)), # category → count
9                 colors=pal, figsize=(6.6, 3.6),
10                 block_aspect_ratio=1)         # square blocks
11 plt.show()
```



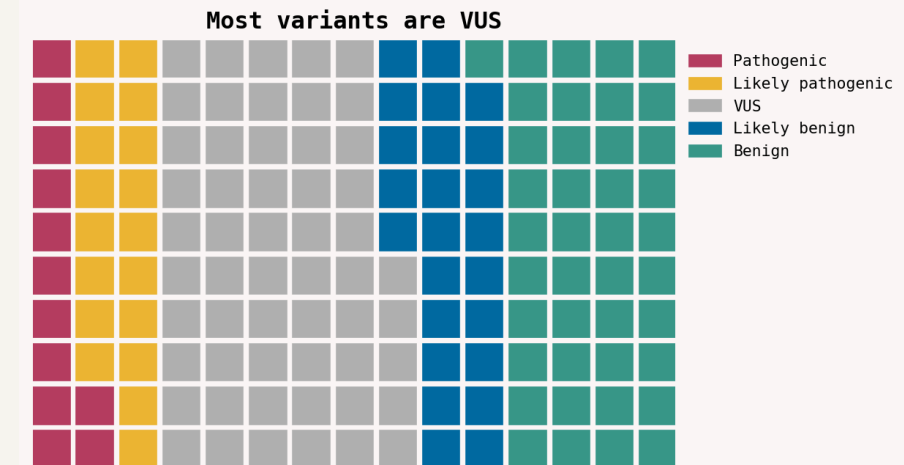
Waffle · Step 2 — Legend & Finding

COHORT SUMMARIES · BUILD

2 · Name the categories and the message

Add a legend outside the grid and a title that states the takeaway. The grey expanse needs no explanation: most variants are of uncertain significance.

```
1 from pywaffle import Waffle
2 from genomics_course.data import load_variant_classes
3
4 vc = load_variant_classes()                # classification, n
5 pal = [RED, AMBER, GREY, BLUE, GREEN]     # colour per ACMG class
6 fig = plt.figure(FigureClass=Waffle, rows=10,
7                 values=dict(zip(vc.classification, vc.n)),
8                 colors=pal, figsize=(6.6, 3.6),
9                 block_aspect_ratio=1,
10                legend={"loc": "upper left", "bbox_to_anchor": (1, 1), # key, out
11                       "fontsize": 8, "frameon": False},
12                title={"label": "Most variants are VUS",                # title =
13                      "fontsize": 12, "fontweight": "bold"})
14 plt.show()
```



THE BOTTOM LINE

A waffle turns counts into countable squares: **values dict → tile → legend → title**. Reach for it over a pie when categories are few and the total is a round number, here, the wall of grey says “VUS dominates” instantly.

Waterfall Plots

COHORT SUMMARIES • THEORY

A cohort ranked into a skyline

A waterfall is a bar per sample, sorted by the value, so the cohort reads as a descending skyline. It's the staple cancer-genomics view of **tumour mutation burden**: who carries the most mutations, and does a subtype or marker cluster at the top.

The moves: sort descending, colour the bars by a categorical (subtype), and overlay a marker for a second variable (here MSI status) so two facts share one axis.

Waterfall • The Data

COHORT SUMMARIES

TMB across a 60-patient cohort

`load_tmb_cohort()` gives one row per patient: their TMB, breast-cancer `subtype`, and microsatellite status (`msi`). MSI-H tumours are biologically hypermutated, we expect them high.

```
1 from genomics_course.data import load_tmb_cohort
2
3 co = load_tmb_cohort() # patient, tmb, subtype, msi
4 co.head(4)
```

	patient	tmb	subtype	msi
0	P000	5.4	HER2+	MSS
1	P001	3.3	Luminal B	MSS
2	P002	8.8	Luminal A	MSS
3	P003	16.1	Luminal B	MSS

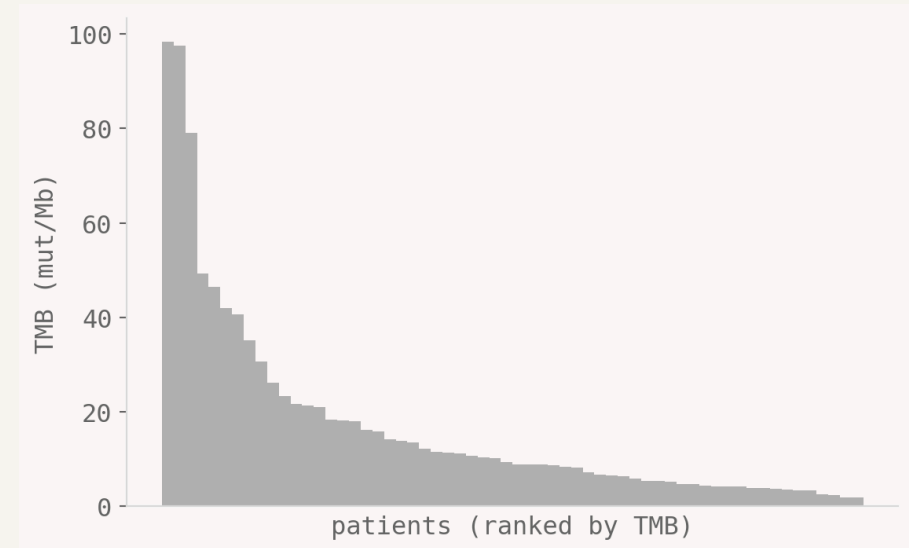
Waterfall • Step 1 — Rank into Bars

COHORT SUMMARIES • BUILD

1 • Sort, then one bar each

Sort by TMB descending and draw a bar per patient with `width=1.0` so they form a continuous skyline. The shape, a steep head and a long tail, is the whole point.

```
1 from genomics_course.data import load_tmb_cohort
2
3 co = load_tmb_cohort().sort_values("tmb", ascending=False).reset_index(drop=True) #
4
5 fig, ax = plt.subplots(figsize=(6.6, 4.2))
6 ax.bar(range(len(co)), co.tmb, color=GREY, width=1.0, linewidth=0) # one bar per pa
7 ax.set_xticks([]) # individual ids
8 ax.set_xlabel("patients (ranked by TMB)"); ax.set_ylabel("TMB (mut/Mb)")
9 for s in ["top", "right"]: ax.spines[s].set_visible(False)
10 plt.show()
```



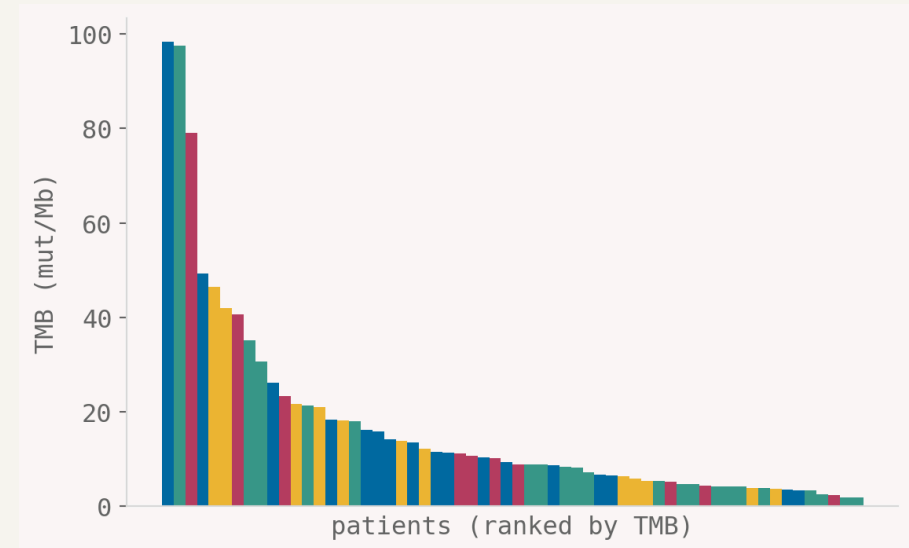
Waterfall • Step 2 — Colour by Subtype

COHORT SUMMARIES • BUILD

2 • Map a colour to each subtype

Build a subtype → colour dict and map it onto the sorted bars. Now you can scan whether any subtype clusters in the high-TMB head.

```
1 from genomics_course.data import load_tmb_cohort
2
3 co = load_tmb_cohort().sort_values("tmb", ascending=False).reset_index(drop=True)
4 sub_pal = {"Luminal A": GREEN, "Luminal B": BLUE,          # one colour per subtype
5            "HER2+": AMBER, "TNBC": RED}
6
7 fig, ax = plt.subplots(figsize=(6.6, 4.2))
8 ax.bar(range(len(co)), co.tmb, width=1.0, linewidth=0,
9        color=co.subtype.map(sub_pal))          # colour each bar by its su
10 ax.set_xticks([])
11 ax.set_xlabel("patients (ranked by TMB)"); ax.set_ylabel("TMB (mut/Mb)")
12 for s in ["top", "right"]: ax.spines[s].set_visible(False)
13 plt.show()
```



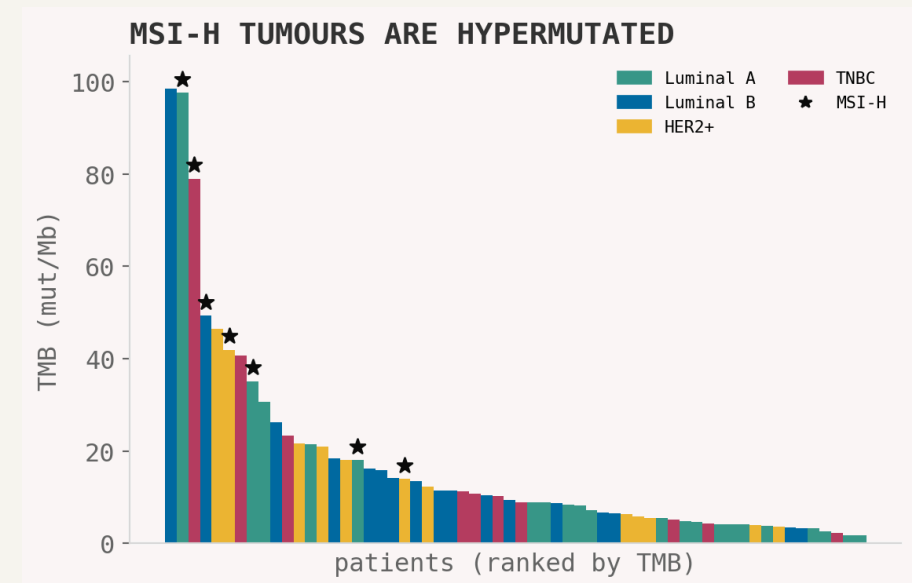
Waterfall · Step 3 — Flag MSI-H, Title It

COHORT SUMMARIES · BUILD

3 · A second variable, and the finding

Drop a star above each MSI-H patient's bar, build a combined legend (subtype patches + the star), and title with the result. The stars cluster in the head, MSI-H tumours are the hypermutators.

```
1 from genomics_course.data import load_tmb_cohort
2 from matplotlib.patches import Patch # for the legend swatches
3
4 co = load_tmb_cohort().sort_values("tmb", ascending=False).reset_index(drop=True)
5 sub_pal = {"Luminal A": GREEN, "Luminal B": BLUE, "HER2+": AMBER, "TNBC": RED}
6
7 fig, ax = plt.subplots(figsize=(6.6, 4.2))
8 ax.bar(range(len(co)), co.tmb, width=1.0, linewidth=0, color=co.subtype.map(sub_pal))
9 ax.set_xticks([])
10 ax.set_xlabel("patients (ranked by TMB)"); ax.set_ylabel("TMB (mut/Mb)")
11 msi = co.msi == "MSI-H" # boolean mask of MSI-H patients
12 ax.scatter(co.index[msi], co.tmb[msi] + co.tmb.max() * .03, # a star just above each
13           marker="*", color=INK, s=55, zorder=5)
14 for s in ["top", "right"]: ax.spines[s].set_visible(False)
15 ax.set_title("MSI-H TUMOURS ARE HYPERMUTATED", color="#333333", fontweight="bold", 1
16             handles = [Patch(color=c, label=k) for k, c in sub_pal.items()] # subtype swatches
17 handles += [plt.Line2D([0], [0], marker="*", color=INK, ls="none", label="MSI-H")]
18 ax.legend(handles=handles, frameon=False, fontsize=8, ncol=2)
19 plt.show()
```



THE BOTTOM LINE

A waterfall ranks a cohort into a skyline: **sort descending → colour by subtype → overlay a second variable → title the finding**. Here the MSI-H stars pile up in the high-TMB head, exactly the biology you'd expect.

Exercise • Waffle

COHORT SUMMARIES • EXERCISE

Compose a cohort

CODING EXERCISE

`load_assay_counts()` lists datasets per assay type. Waffle it: one square per (scaled) dataset, a legend outside the grid, and a title naming the most common assay.

```
1 from pywaffle import Waffle
2 from genomics_course.data import load_assay_counts
3
4 ac = load_assay_counts()          # assay, n_datasets
5
6 # 1. values = dict(zip(ac.assay, ac.n_datasets))  (scale down if counts are large)
7 # 2. plt.figure(FigureClass=Waffle, rows=10, values=..., colors=[GREEN, BLUE, AMBER, RED])
8 # 3. legend outside right; block_aspect_ratio=1
9 # 4. title that names the leading assay
```

Exercise • Waterfall

COHORT SUMMARIES • EXERCISE

Rank a different cohort

REAL SCENARIO

Re-rank the same cohort, but colour by **MSI status** instead of subtype, and add a dashed line at the cohort's 85th-percentile TMB (a rough hypermutator threshold). Which side of the line do the MSI-H bars fall on?

```
1 from genomics_course.data import load_tmb_cohort
2
3 co = load_tmb_cohort().sort_values("tmb", ascending=False).reset_index(drop=True)
4
5 # 1. colour bars by msi: {"MSI-H": RED, "MSS": GREY}
6 # 2. cut = co.tmb.quantile(0.85)
7 # 3. ax.axhline(cut, ls="--", color=INK) # hypermutator threshold
8 # 4. title it; read whether MSI-H sits above the line
```


Peak Visualization

CHIP / ATAC PEAKS • DAY 01

What Is a Peak

CHIP / ATAC PEAKS • THEORY

Signal piled up at a genomic position

A peak is a region where sequencing reads pile up: a transcription factor bound there (ChIP-seq), the chromatin was open (ATAC-seq), or a histone carried a mark. The standard view is a **genome browser**: each assay a horizontal signal track, stacked over a shared coordinate, with genes annotated underneath.

The reading skill is combinatorial. **H3K4me3** marks active promoters, **H3K27ac** marks active promoters *and* enhancers, **ATAC** marks any open chromatin. So a region that is open and acetylated but lacks H3K4me3 is an **enhancer**, not a promoter. We'll build that exact comparison.

Peaks • The Data

CHIP / ATAC PEAKS

Three signal tracks + a gene table

`load_peaks()` is the signal, one row per genomic position, one column per mark. `load_peak_genes()` is the annotation track: where each gene sits and which strand it's on.

```
1 from genomics_course.data import load_peaks, load_peak_genes
2
3 sig = load_peaks()           # position, H3K27ac, H3K4me3, ATAC
4 genes = load_peak_genes()    # name, start, end, strand
5 sig.head(3)
```

	position	H3K27ac	H3K4me3	ATAC
0	0.00000	0.069866	0.048841	0.000581
1	20.02002	0.053866	0.048752	0.019810
2	40.04004	0.054921	0.019941	0.081723

Peaks • Step 1 — One Track

CHIP / ATAC PEAKS • BUILD

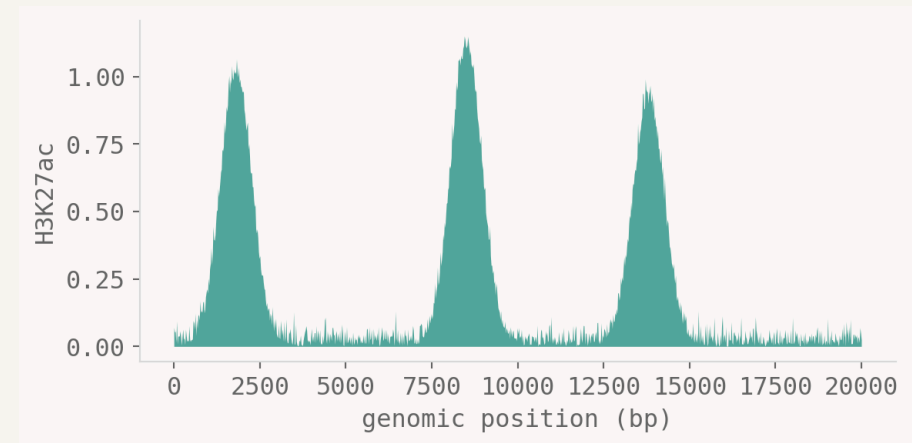
1 • Fill the area under one signal

A signal track is just `fill_between(position, signal)`.

Drop the frame so the genomic axis reads cleanly.

Three clear H3K27ac peaks appear, two promoters and one enhancer.

```
1 from genomics_course.data import load_peaks
2
3 sig = load_peaks()                                # position, H3K27ac, H3K4me
4
5 fig, ax = plt.subplots(figsize=(6.6, 3.0))
6 ax.fill_between(sig.position, sig.H3K27ac, color=GREEN, alpha=.85, lw=0) # area und
7 ax.set_ylabel("H3K27ac"); ax.set_xlabel("genomic position (bp)")
8 for s in ["top", "right"]: ax.spines[s].set_visible(False)
9 plt.show()
```



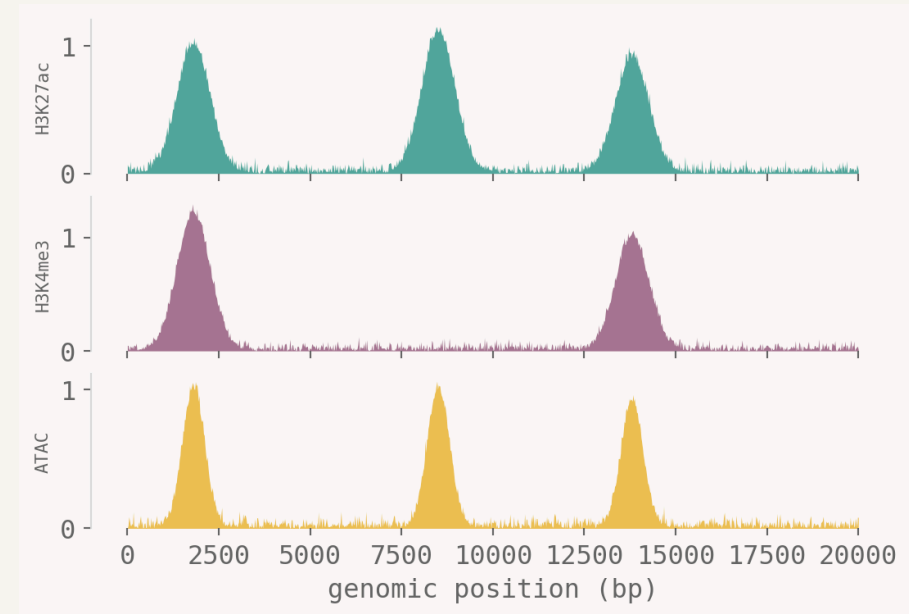
Peaks • Step 2 — Stack the Marks

CHIP / ATAC PEAKS • BUILD

2 • One subplot per mark, shared x-axis

Stack the three marks as rows with `sharex=True` so positions line up. Now you compare marks vertically: the middle peak is missing from the H3K4me3 row.

```
1 from genomics_course.data import load_peaks
2
3 sig = load_peaks()
4 marks = ["H3K27ac", "H3K4me3", "ATAC"]           # plotting order, top to bottom
5 mark_pal = {"H3K27ac": GREEN, "H3K4me3": PURPLE, "ATAC": AMBER}
6
7 fig, axes = plt.subplots(len(marks), 1, figsize=(6.6, 4.2), # one row per mark
8                           sharex=True,                    # align genomic coordinates
9                           gridspec_kw={"hspace": 0.15})
10 for ax, m in zip(axes, marks):                       # draw each mark in its row
11     ax.fill_between(sig.position, sig[m], color=mark_pal[m], alpha=.85, lw=0)
12     ax.set_ylabel(m, fontsize=8); ax.set_ylim(bottom=0)
13     for s in ["top", "right", "bottom"]: ax.spines[s].set_visible(False)
14 axes[-1].set_xlabel("genomic position (bp)")
15 plt.show()
```



Peaks • Step 3 — The Gene Track

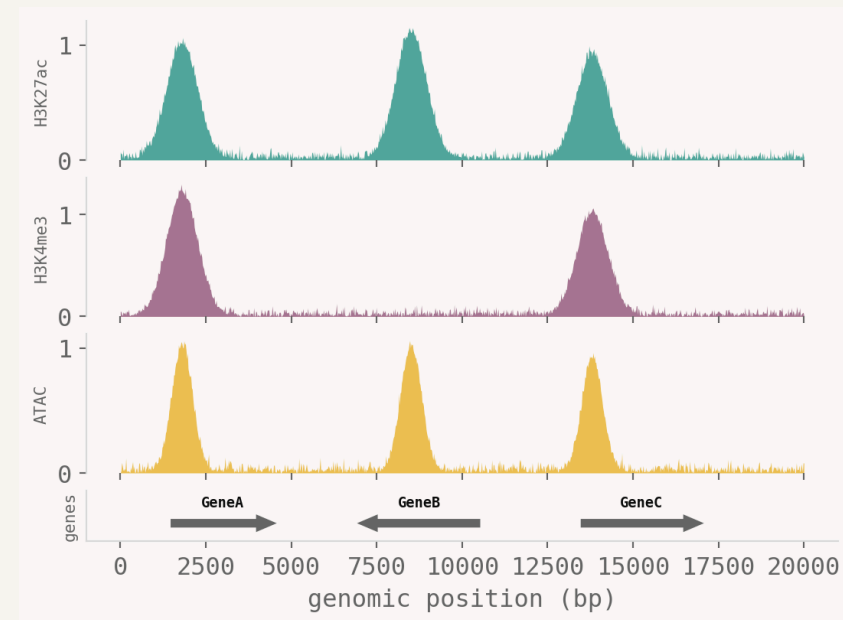
CHIP / ATAC PEAKS • BUILD

3 • Annotate genes underneath with strand

Add a short track at the bottom and draw each gene as an arrow pointing in its strand direction (`FancyArrow`).

Now the peaks have context: which sit over a gene's start (its TSS).

```
1 from genomics_course.data import load_peaks, load_peak_genes
2 from matplotlib.patches import FancyArrow # arrow = gene body + st
3
4 sig = load_peaks(); genes = load_peak_genes()
5 marks = ["H3K27ac", "H3K4me3", "ATAC"]
6 mark_pal = {"H3K27ac": GREEN, "H3K4me3": PURPLE, "ATAC": AMBER}
7
8 fig, axes = plt.subplots(len(marks) + 1, 1, figsize=(6.6, 4.6), sharex=True,
9                           gridspec_kw={"height_ratios": [3, 3, 3, 1.1], "hspace": 0})
10 for ax, m in zip(axes, marks):
11     ax.fill_between(sig.position, sig[m], color=mark_pal[m], alpha=.85, lw=0)
12     ax.set_ylabel(m, fontsize=8); ax.set_ylim(bottom=0)
13     for s in ["top", "right", "bottom"]: ax.spines[s].set_visible(False)
14 gax = axes[-1] # the gene annotation row
15 for _, g in genes.iterrows():
16     x0 = g.start if g.strand == "+" else g.end # arrow tail at the 5' end
17     dx = (g.end - g.start) * (1 if g.strand == "+" else -1) # points along the strand
18     gax.add_patch(FancyArrow(x0, 0.5, dx, 0, width=0.18, length_includes_head=True,
19                             head_width=0.4, head_length=500, color=MUTED, zorder=2))
20     gax.text((g.start + g.end) / 2, 0.95, g["name"], ha="center", fontsize=7,
21             fontweight="bold", color=INK)
22 gax.set_ylim(0, 1.4); gax.set_yticks([1]); gax.set_ylabel("genes", fontsize=8)
```



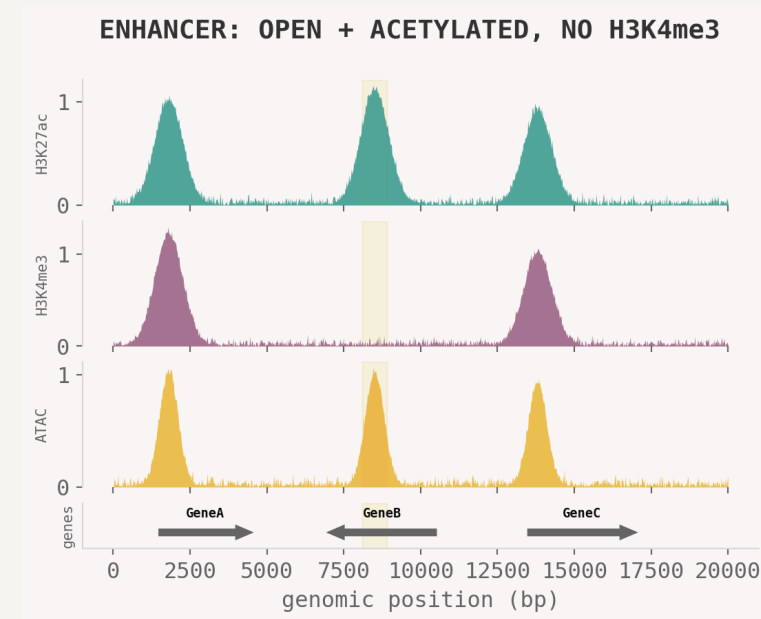
Peaks · Step 4 — Highlight the Enhancer

CHIP / ATAC PEAKS · BUILD

4 · Shade the region, name the finding

Draw an `axvspan` across every track at the middle peak and title with the call. The shaded column is high in H3K27ac and ATAC but flat in H3K4me3: an active **enhancer**, not a promoter.

```
1 from genomics_course.data import load_peaks, load_peak_genes
2 from matplotlib.patches import FancyArrow
3
4 sig = load_peaks(); genes = load_peak_genes()
5 marks = ["H3K27ac", "H3K4me3", "ATAC"]
6 mark_pal = {"H3K27ac": GREEN, "H3K4me3": PURPLE, "ATAC": AMBER}
7
8 fig, axes = plt.subplots(len(marks) + 1, 1, figsize=(6.6, 4.6), sharex=True,
9                           gridspec_kw={"height_ratios": [3, 3, 3, 1.1], "hspace": 0})
10 for ax, m in zip(axes, marks):
11     ax.fill_between(sig.position, sig[m], color=mark_pal[m], alpha=.85, lw=0)
12     ax.axvspan(8100, 8900, color=AMBER, alpha=.12, zorder=0) # highlight the enhancer
13     ax.set_ylabel(m, fontsize=8); ax.set_ylim(bottom=0)
14     for s in ["top", "right", "bottom"]: ax.spines[s].set_visible(False)
15 gax = axes[-1]
16 gax.axvspan(8100, 8900, color=AMBER, alpha=.12, zorder=0) # same column on top
17 for _, g in genes.iterrows():
18     x0 = g.start if g.strand == "+" else g.end
19     dx = (g.end - g.start) * (1 if g.strand == "+" else -1)
20     gax.add_patch(FancyArrow(x0, 0.5, dx, 0, width=0.18, length_includes_head=True,
21                             head_width=0.4, head_length=500, color=MUTED, zorder=0))
22     gax.text((g.start + g.end) / 2, 0.95, g["name"], ha="center", fontsize=7)
```



THE BOTTOM LINE

A browser view is **one fill_between per mark → stack with shared x → annotate genes → highlight the region**. The marks read combinatorially: H3K27ac + ATAC + H3K4me3 = active promoter; H3K27ac + ATAC but **no** H3K4me3 = enhancer.

The Production Tool • pyGenomeTracks

CHIP / ATAC PEAKS • REFERENCE

Real bigWig/BED, driven by a config

From scratch is great for teaching; for real data you'd use **pyGenomeTracks**, a CLI that renders a browser figure from an INI config pointing at your `.bw` / `.bed` / `.gtf` files. Same mental model, production inputs.

```
1 # pip install pyGenomeTracks    (reads real bigWig / BED / GTF, not in-memory arrays)
2 config = ""
3 [H3K27ac]
4 file = H3K27ac.bw
5 color = #379A8B
6 height = 3
7 [H3K4me3]
8 file = H3K4me3.bw
9 color = #9A607F
10 height = 3
11 [genes]
12 file = genes.gtf
13 height = 1.5
14 ""
15 open("tracks.ini", "w").write(config)
16
17 # render a region to a figure from the shell:
18 # pyGenomeTracks --tracks tracks.ini --region chr1:1000000-1020000 -o region.pdf
```


Exercise · Read the Marks

CHIP / ATAC PEAKS · EXERCISE

Classify each peak

REAL SCENARIO

Use the data, not your eyes. For each peak centre ($\approx 1800, 8500, 13800$ bp), pull the signal of all three marks and decide: **promoter** (all three present) or **enhancer** (H3K27ac + ATAC, no H3K4me3)?

```
1 from genomics_course.data import load_peaks
2
3 sig = load_peaks()           # position, H3K27ac, H3K4me3, ATAC
4
5 # 1. for each centre c in [1800, 8500, 13800]:
6 #   row = sig.iloc[(sig.position - c).abs().argmin()] # nearest position
7 # 2. print row.H3K27ac, row.H3K4me3, row.ATAC
8 # 3. rule: H3K4me3 high → promoter ; H3K4me3 low but H3K27ac+ATAC high → enhancer
9 # 4. which of the three peaks is the enhancer?
```

Pathway Enrichment

FROM GENE LIST TO BIOLOGY • DAY 01

What Enrichment Asks

PATHWAY ENRICHMENT · THEORY

Is any pathway over-represented in my gene list?

You ran differential expression, you have a list of significant genes. Enrichment asks: do those genes fall into known pathways (GO terms, KEGG) **more than chance**? A pathway with 5 of your 50 genes when the genome would predict 1 is enriched, evidence that process is involved.

The workflow is always the same: **DE → significant genes → test each pathway against a background → rank by adjusted p-value → plot the top hits**. The fixing-up is multiple testing: you test thousands of pathways, so you read `p_adj`, not raw p.

Getting Enrichment • gget

PATHWAY ENRICHMENT • THE TOOL

One call to the Enrichr database

`gget enrichr` takes a gene list and a library name and returns a ranked table, no account, no upload. It hits a web API, so we ran it once and saved the result into `load_enrichment()`; the deck plots that offline.

```
1 # pip install gget (this call needs internet - we pre-saved its output)
2 import gget
3
4 drought_genes = ["RD29A", "DREB2A", "ABA1", "NCED3", "P5CS1", # your significant genes
5                  "RAB18", "COR15A", "LEA14", "RD22", "ERD10"]
6 res = gget.enrichr(drought_genes, database="GO_Biological_Process_2021")
7
8 # res columns include: path_name, adj_p_val, overlapping_genes
9 res[["path_name", "adj_p_val", "overlapping_genes"]].head()
10 # → we cleaned this into load_enrichment(): pathway, direction, p_adj, gene_count, gene_ratio
```

Pathway · The Data

PATHWAY ENRICHMENT

The pre-computed results table

`load_enrichment()` holds the enriched pathways for both directions: each pathway, its adjusted p-value, how many of your genes hit it (`gene_count`), the fraction of the pathway you hit (`gene_ratio`), and a functional `category`.

```
1 from genomics_course.data import load_enrichment
2
3 en = load_enrichment() # pathway, direction, p_adj, gene_count, gene_ratio, category
4 en.head(4)
```

	pathway	direction	p_adj	gene_count	gene_ratio
0	response to water deprivation	up	1.200000e-09	38	0.42
1	abscisic acid signaling	up	4.500000e-08	27	0.31
2	response to osmotic stress	up	7.000000e-07	31	0.28

Pathway Bars • Step 1 — Rank by Significance

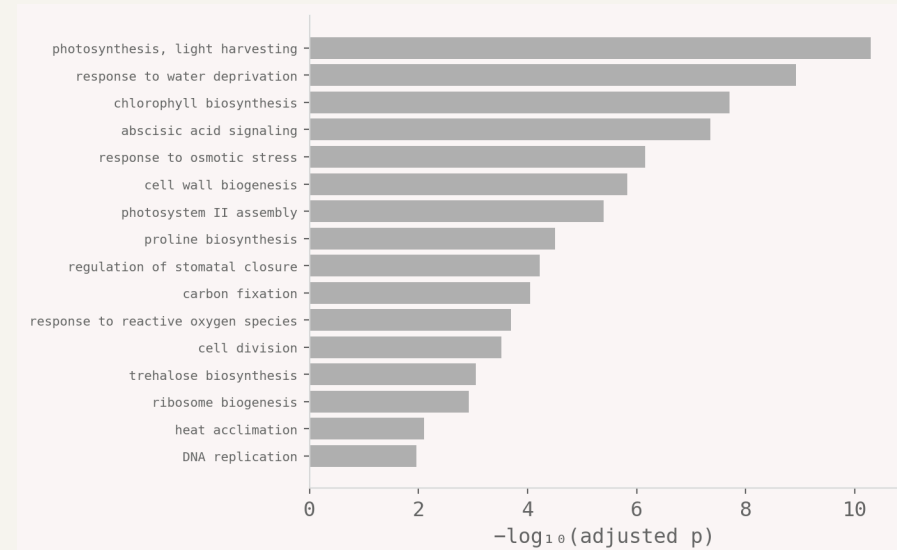
PATHWAY ENRICHMENT • BUILD

1 • $-\log_{10}(p)$, sorted into bars

p-values are tiny and uneven, so transform to

$-\log_{10}(p_{\text{adj}})$: bigger = more significant. Sort and draw horizontal bars so the long labels stay readable.

```
1 from genomics_course.data import load_enrichment
2
3 en = load_enrichment()                # pathway, direction, p_adj
4 en["nlp"] = -np.log10(en.p_adj)        # significance on a readabl
5 d = en.sort_values("nlp")              # smallest at bottom
6
7 fig, ax = plt.subplots(figsize=(6.2, 5.0))
8 ax.barh(range(len(d)), d.nlp, color=GREY)    # one bar per pathway
9 ax.set_yticks(range(len(d))); ax.set_yticklabels(d.pathway, fontsize=7.5)
10 ax.set_xlabel(" $-\log_{10}(\text{adjusted } p)$ ")
11 for s in ["top", "right"]: ax.spines[s].set_visible(False)
12 plt.show()
```



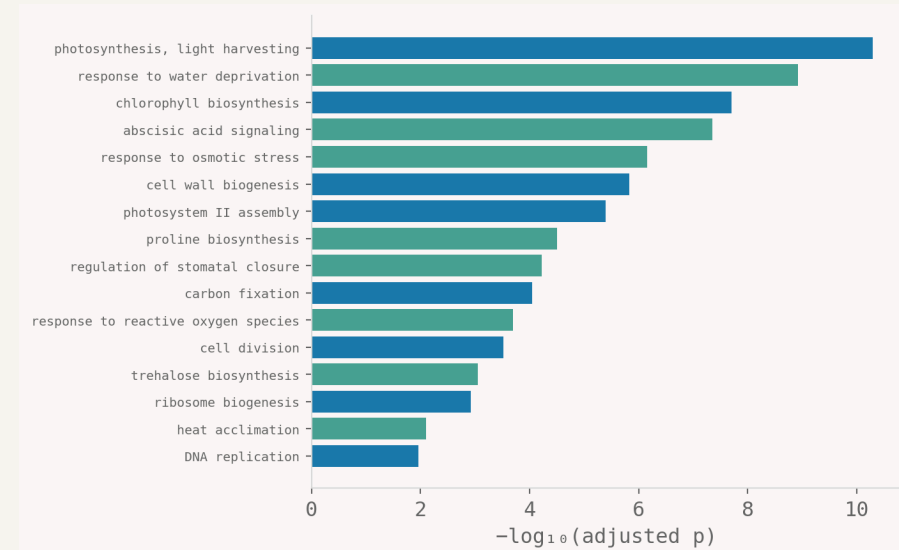
Pathway Bars • Step 2 — Colour by Direction

PATHWAY ENRICHMENT • BUILD

2 • Induced vs repressed

Map a colour to each regulation direction. Now you can see whether drought-induced pathways differ in kind from the repressed ones, they do.

```
1 from genomics_course.data import load_enrichment
2
3 en = load_enrichment()
4 en["nlp"] = -np.log10(en.p_adj)
5 d = en.sort_values("nlp")
6 dir_pal = {"up": GREEN, "down": BLUE} # induced vs repressed
7
8 fig, ax = plt.subplots(figsize=(6.2, 5.0))
9 ax.barh(range(len(d)), d.nlp, color=d.direction.map(dir_pal), alpha=.9) # colour ea
10 ax.set_yticks(range(len(d))); ax.set_yticklabels(d.pathway, fontsize=7.5)
11 ax.set_xlabel("-log10(adjusted p)")
12 for s in ["top", "right"]: ax.spines[s].set_visible(False)
13 plt.show()
```



Pathway Bars • Step 3 — Threshold & Finding

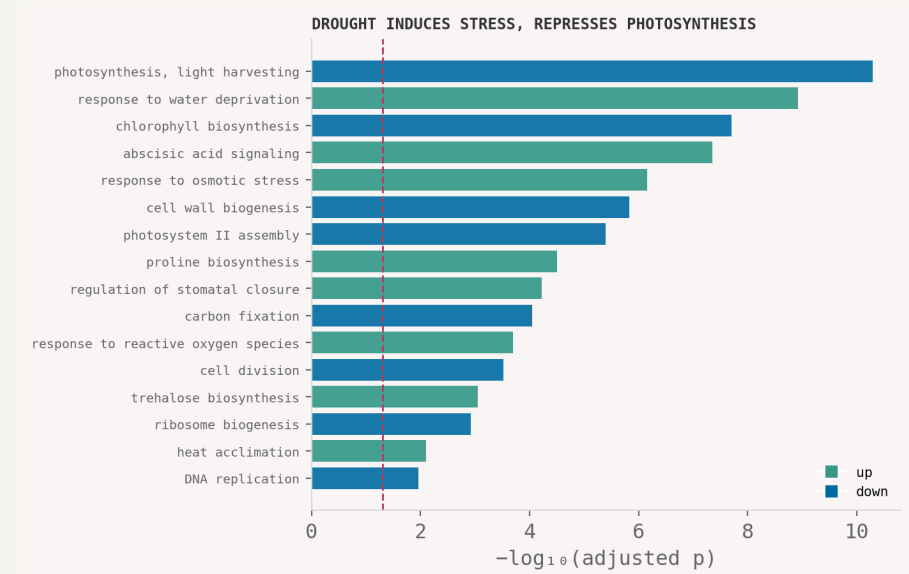
PATHWAY ENRICHMENT • BUILD

3 • Mark significance, name the biology

Add a dashed line at $p = 0.05$ (everything right of it is significant), a legend, and a title that states the result.

The split is clean: stress and signalling up, photosynthesis and growth down.

```
1 from genomics_course.data import load_enrichment
2 from matplotlib.lines import Line2D # for the legend swatches
3
4 en = load_enrichment()
5 en["nlp"] = -np.log10(en.p_adj)
6 d = en.sort_values("nlp")
7 dir_pal = {"up": GREEN, "down": BLUE}
8
9 fig, ax = plt.subplots(figsize=(6.2, 5.0))
10 ax.barh(range(len(d)), d.nlp, color=d.direction.map(dir_pal), alpha=.9)
11 ax.axvline(-np.log10(0.05), color=RED, ls="--", lw=1.1) # significance threshold
12 ax.set_yticks(range(len(d))); ax.set_yticklabels(d.pathway, fontsize=7.5)
13 ax.set_xlabel("-log10(adjusted p)")
14 for s in ["top", "right"]: ax.spines[s].set_visible(False)
15 handles = [Line2D([0], [0], marker="s", color="w", markerfacecolor=c, label=k, ms=8)
16             for k, c in dir_pal.items()] # up / down key
17 ax.legend(handles=handles, frameon=False, fontsize=8, loc="lower right")
18 ax.set_title("DROUGHT INDUCES STRESS, REPRESSES PHOTOSYNTHESIS",
19             color="#333333", fontweight="bold", fontsize=9, loc="left")
20 plt.show()
```



Pathway Bubbles • Step 1 — Add Two More Variables

PATHWAY ENRICHMENT • BUILD

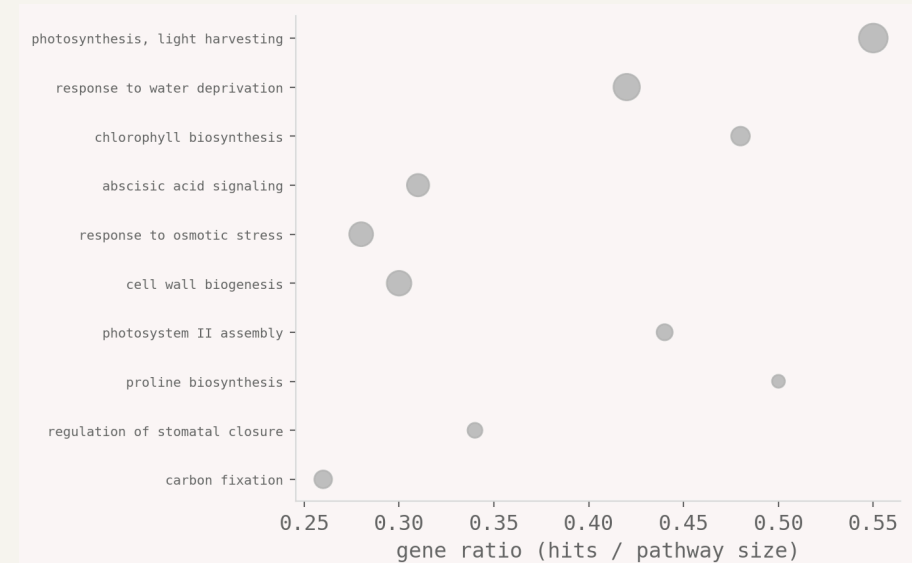
1 • Gene ratio on x, count as size

A bar shows one number; a bubble shows three. Put

`gene_ratio` on the x-axis and scale each marker by

`gene_count`, so a big bubble far right is a pathway you hit broadly and deeply.

```
1 from genomics_course.data import load_enrichment
2
3 en = load_enrichment()
4 en["nlp"] = -np.log10(en.p_adj)
5 top = en.nlargest(10, "nlp").sort_values("nlp") # ten most significant
6
7 fig, ax = plt.subplots(figsize=(6.2, 5.0))
8 ax.scatter(top.gene_ratio, range(len(top)), # x = ratio, y = pathway
9            s=top.gene_count * 6, color=GREY, alpha=.8) # bubble size = gene count
10 ax.set_yticks(range(len(top))); ax.set_yticklabels(top.pathway, fontsize=7.5)
11 ax.set_xlabel("gene ratio (hits / pathway size)")
12 for s in ["top", "right"]: ax.spines[s].set_visible(False)
13 plt.show()
```



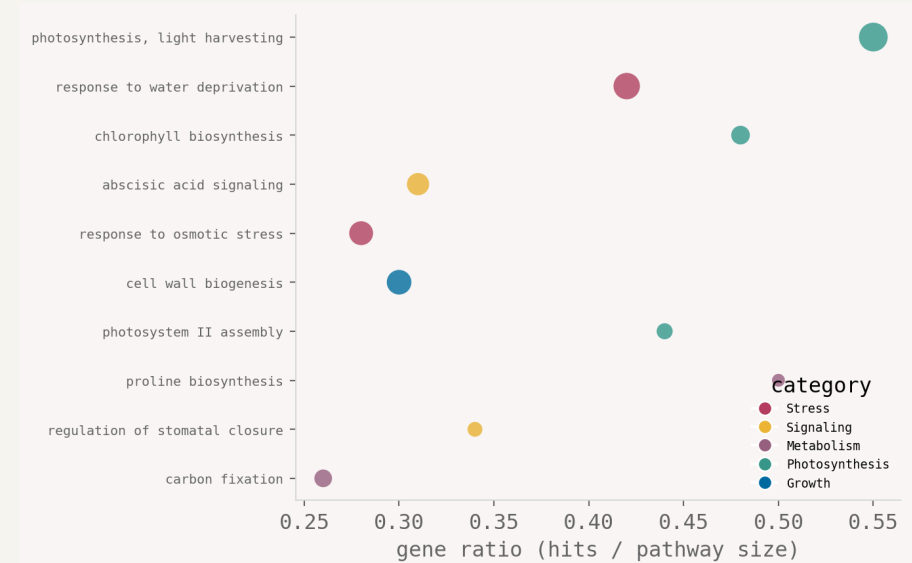
Pathway Bubbles • Step 2 — Colour by Category

PATHWAY ENRICHMENT • BUILD

2 • A fourth variable: function

Colour the bubbles by functional `category` and add a legend. Four dimensions, significance (rank), breadth (x), size (count), and function (colour), on one panel.

```
1 from genomics_course.data import load_enrichment
2 from matplotlib.lines import Line2D
3
4 en = load_enrichment()
5 en["nlp"] = -np.log10(en.p_adj)
6 top = en.nlargest(10, "nlp").sort_values("nlp")
7 cat_pal = {"Stress": RED, "Signaling": AMBER, "Metabolism": PURPLE, # colour per f
8           "Photosynthesis": GREEN, "Growth": BLUE}
9
10 fig, ax = plt.subplots(figsize=(6.2, 5.0))
11 ax.scatter(top.gene_ratio, range(len(top)), s=top.gene_count * 6,
12           c=top.category.map(cat_pal), alpha=.8, linewidths=0) # colour by category
13 ax.set_yticks(range(len(top))); ax.set_yticklabels(top.pathway, fontsize=7.5)
14 ax.set_xlabel("gene ratio (hits / pathway size)")
15 for s in ["top", "right"]: ax.spines[s].set_visible(False)
16 handles = [Line2D([0], [0], marker="o", color="w", markerfacecolor=c, label=k, ms=8)
17            for k, c in cat_pal.items()]
18 ax.legend(handles=handles, frameon=False, fontsize=7, title="category", loc="lower r
19 plt.show()
```



THE BOTTOM LINE

Enrichment turns a gene list into ranked biology: `gget enrichr` to get the table → $-\log_{10}(p)$ bars to rank → bubbles to add gene ratio, count, and function. Read `p_adj`, not raw p, and always show how many genes drove each hit.

Exercise · Read the Enrichment

PATHWAY ENRICHMENT · EXERCISE

Which pathways, and how confident?

REAL SCENARIO

A reviewer asks: of the drought-**induced** pathways, which clear significance, and which is supported by the most of your genes? Filter `load_enrichment()` and rank two ways.

```
1 from genomics_course.data import load_enrichment
2
3 en = load_enrichment()           # pathway, direction, p_adj, gene_count, ...
4
5 # 1. keep only direction == "up"
6 # 2. significant? en.p_adj < 0.05
7 # 3. most significant: sort by p_adj ascending, take top 3
8 # 4. best-supported: sort by gene_count descending, take top 3
9 # 5. do the two top-3 lists agree? (low p and high gene_count aren't the same thing)
```

Synteny & Rearrangements

COMPARING WHOLE GENOMES · PLOTSR

What plotsr Draws

PLOTSR • THEORY

Synteny and structural rearrangements across assemblies

`plotsr` visualises how several chromosome-level assemblies line up: long **syntenic** blocks (same order, same strand) plus the **rearrangements** between them, inversions, translocations, duplications. Each genome is a horizontal track; ribbons connect homologous regions between adjacent genomes.

It doesn't compute the comparison itself. It consumes the structural annotation from **SyRI**, so the pipeline is always: align the genomes, find rearrangements with SyRI, then plot with plotsr. Reach for it when you have two or more good assemblies and want to show their differences, not tabulate them.

Step 0 — Install

PLOTSR • INSTALL

Bioconda is the easy path

plotsr ships on bioconda, which pulls the pinned NumPy / pandas / matplotlib it needs. A fresh environment keeps it isolated.

```
1 # one clean environment
2 conda create -n syteny -c bioconda plotsr
3 conda activate syteny
4
5 plotsr -h           # confirm it's on PATH
```

If you install from source instead, plotsr pins **Python < 3.12** and **NumPy < 2**, build the env accordingly.

```
1 $ plotsr -h
2 usage: plotsr --sr SR --genomes GENOMES -o OUT
3     --sr          SyRI output (repeat per pair)
4     --genomes     genomes information file
5     --tracks      feature tracks (genes, SNPs...)
6     --markers     labelled markers (BED)
7     --cfg         visual config
8     --reg / --chr zoom to region / chrom
9     --itx         cross-genome (itx) mode
```

Where the Input Comes From

PLOTSR • UPSTREAM

Align → SyRI → plotsr

To compare genomes `A → B → C`, you align each adjacent pair and run SyRI on each. This is the data-generation step; plotsr only reads the result.

```
1 # 1) whole-genome alignment with minimap2 (asm5 for same-species)
2 minimap2 -ax asm5 --eqx A.fa B.fa | samtools sort -O BAM - > A_B.bam
3 samtools index A_B.bam
4
5 # 2) structural annotation with SyRI → A_Bsyri.out
6 syri -c A_B.bam -r A.fa -q B.fa -F B --prefix A_B
```

Each `*syri.out` lists every syntenic block and rearrangement between one pair, exactly what plotsr stacks.

Step 1 — The Example Data

PLOTSR • DATA

Four *Arabidopsis* accessions, ready to plot

The repo ships a complete example: Col-0, Ler, Cvi, Eri. Grab it and you have all inputs without running an alignment.

```
1 git clone https://github.com/schneebergerlab/plotsr.git
2 cd plotsr/example
3
4 ls
5 # genomes.txt
6 # *.chrlen          (chrom lengths)
7 # col_lersyri.filtered.out
8 # ler_cvisyri.filtered.out
9 # cvi_erisyri.filtered.out
```

The three `*syri.filtered.out` files are the pairwise comparisons (Col↔Ler, Ler↔Cvi, Cvi↔Eri); the order of the genomes is the order they'll stack.

Step 2 — `genomes.txt`

PLOTS R • DATA

One line per genome, in stacking order

Each row is a genome: a length source, a display name, and style tags. `ft:cl` means the file is a **chromosome-length table** (faster than parsing a FASTA).

```
1 $ cat genomes.txt
```

```
1 #file          name  tags
2 TAIR10_Filtered.chrlen col-0  ft:cl;lw:1.5
3 ler.chrlen     ler    ft:cl;lw:1.5
4 cvi.chrlen     cvi    ft:cl;lw:1.5
5 eri.chrlen     eri    ft:cl;lw:1.5
```

A `.chrlen` is just two columns:

```
1 Chr1  30427671
2 Chr2  19698289
3 Chr3  23459830
```

Step 3 — The SyRI Annotation

PLOTSr • DATA

Each row is one block, tagged by type

The columns are reference coords, query coords, an ID, its parent block, and the **annotation type**. plotsr colours by that type.

```
1 $ head -5 col_lersyri.filtered.out
```

Types it draws: **SYN** (syntenic), **INV** (inversion), **TRANS** (translocation), **DUP** (duplication), **NOTAL** (not aligned).

1	Chr1	1	1084	-	-	-	-	-	NOTAL1	-	NOTAL
2	Chr1	1085	1771083	-	-	Chr1	1	1773924	SYN1	-	SYN
3	Chr1	1085	158550	-	-	Chr1	1	157500	SYNAL1	SYN1	SYNAL
4	Chr1	158551	158596	-	-	Chr1	157501	157556	HDR1	SYN1	HDR
5	Chr1	158597	270144	-	-	Chr1	157557	269099	SYNAL2	SYN1	SYNAL

Step 4 — The Minimal Plot

PLOTSR • BUILD

Three comparisons, one genomes file

Pass one `--sr` per pairwise comparison, plus `--genomes`, and name an output. `plotsr` stacks the four accessions and draws every block.

```
1 plotsr \  
2   --sr col_lersyri.filtered.out \  
3   --sr ler_cvisyri.filtered.out \  
4   --sr cvi_erisyri.filtered.out \  
5   --genomes genomes.txt \  
6   -o output_plot.png
```

Grey is syntenic; orange/green/blue mark the rearrangements.

Step 5 — Add Feature Tracks

PLOTSR • BUILD

Layer gene density, SNPs, centromeres

`--tracks tracks.txt` adds annotation lanes above the genomes. Each line in the file is a GFF/BED/bedGraph with its own style tags (bin width, colours, label).

```
1 plotsr \  
2 --sr col_lersyri.filtered.out \  
3 --sr ler_cvisyri.filtered.out \  
4 --sr cvi_erisyri.filtered.out \  
5 --genomes genomes.txt \  
6 --tracks tracks.txt \  
7 -o output_plot.png
```

```
1 $ cat tracks.txt  
2 TAIR10_GFF3_genes.gff Genes ft:gff;bw:10000;...  
3 1001genomes.snps.bed SNPs bw:10000;lc:sienna;...  
4 Giraut2011_cents.bed Centromeres bc:palegreen;...
```

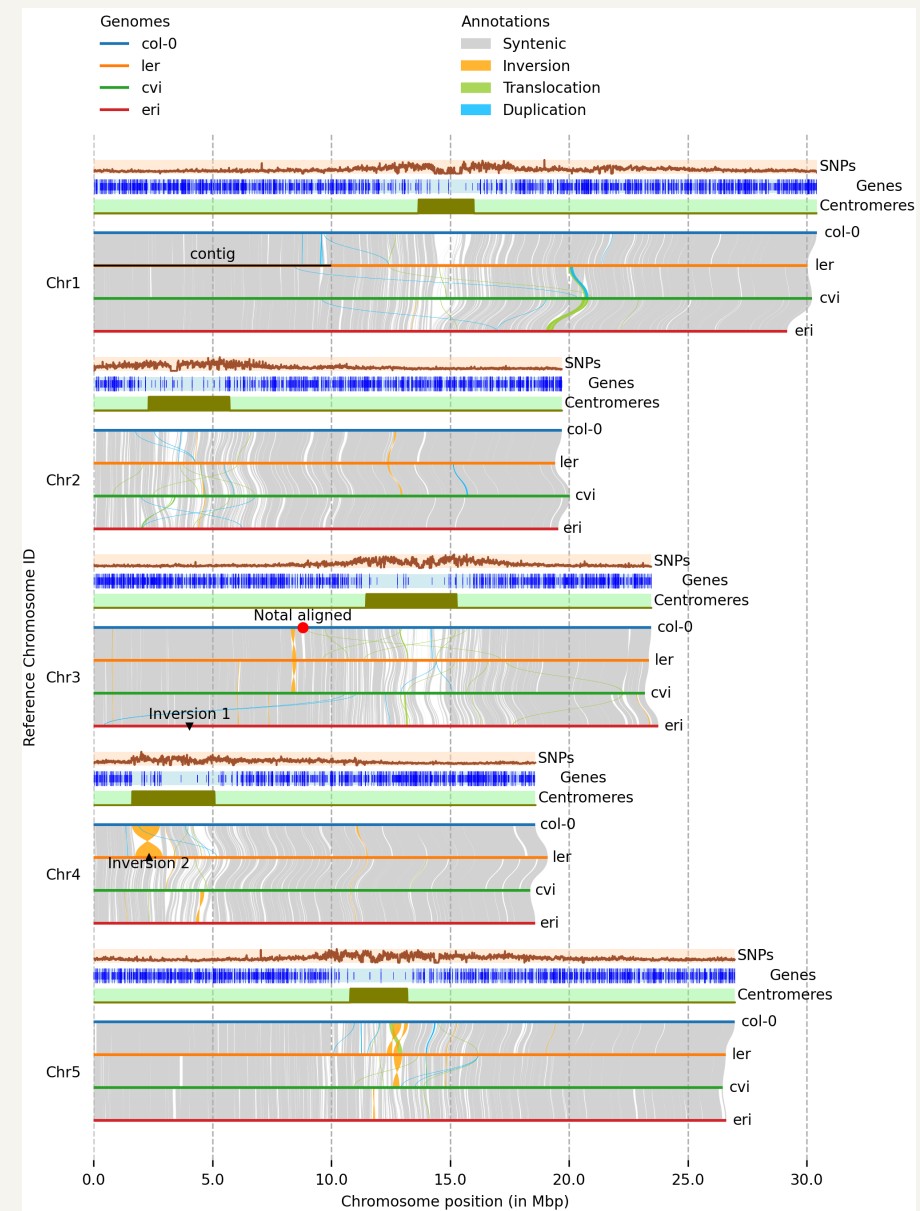
Step 6 — Markers, Config, Sizing

PLOTSR • BUILD

The publication figure

`--markers` labels specific loci, `--cfg` tunes legend/colours, and `-S -W -H -f` set syntenic-ribbon spread, width, height, and font size.

```
1 plotsr \
2 --sr col_lersyri.filtered.out \
3 --sr ler_cvisyri.filtered.out \
4 --sr cvi_erisyri.filtered.out \
5 --genomes genomes.txt \
6 --tracks tracks.txt \
7 --markers markers.bed \
8 --cfg base.cfg \
9 -o output_plot.png -S 0.5 -W 7 -H 10 -f 8
```



Exercise • Your Turn

PLOTSR • EXERCISE

Zoom and switch modes

COMMAND-LINE EXERCISE

Starting from the full command, (1) zoom into the inversion-rich region of chromosome 3, then (2) redraw the whole thing in cross-genome (`--itx`) mode.

```
1 # 1) zoom a single region (note: --reg alone, not with --chr)
2 plotsr ... --cfg base.cfg --reg col-0:Chr3:66000000-100000000 -o zoom.png
3
4 # 2) itx mode - chromosomes drawn end-to-end across genomes
5 plotsr ... --cfg base.cfg --itx -S 0.7 -o itx.png
```

Which mode makes the Chr4 inversion easier to see, and why?

A Dumbbell for QC

MAKE IT PUBLICATION-READY

The job: summarise sequencing QC to your boss

You sent 12 samples for sequencing. For each, you have the **raw mapping rate** and the **callable rate after QC**.

The boss wants one chart: which samples held up, and which lost too much.

Start with the data, one row per sample, sorted so the worst callable rate sits at the bottom.

```
1 qc = load_qc()           # 12 samples
2 qc = qc.sort_values("callable") # worst at the bottom
3 y = range(len(qc))       # one row each
4 qc.head()
```

	sample	raw	callable	loss
0	S02	94.1	68.1	26.0
1	S08	94.3	72.3	22.0
2	S07	90.3	83.2	7.1
3	S11	91.8	85.1	6.7
4	S09	90.9	85.4	5.5

A Dumbbell for QC

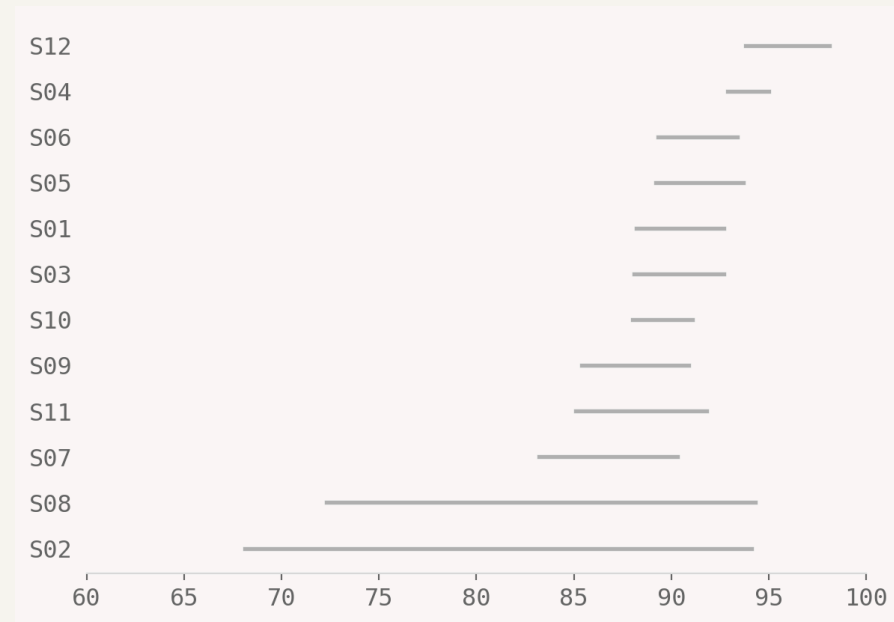
MAKE IT PUBLICATION-READY

Draw the bars: the gap is the story

One horizontal bar per sample, from callable to raw.

The bar length is the QC loss, longer means more lost.

```
1 fig, ax = plt.subplots(figsize=(7, 5))
2
3 for i, r in qc.iterrows():          # one bar per row
4     ax.plot([r.callable, r.raw],      # x: callable -> raw
5             [i, i], color=GREY, lw=2)
```



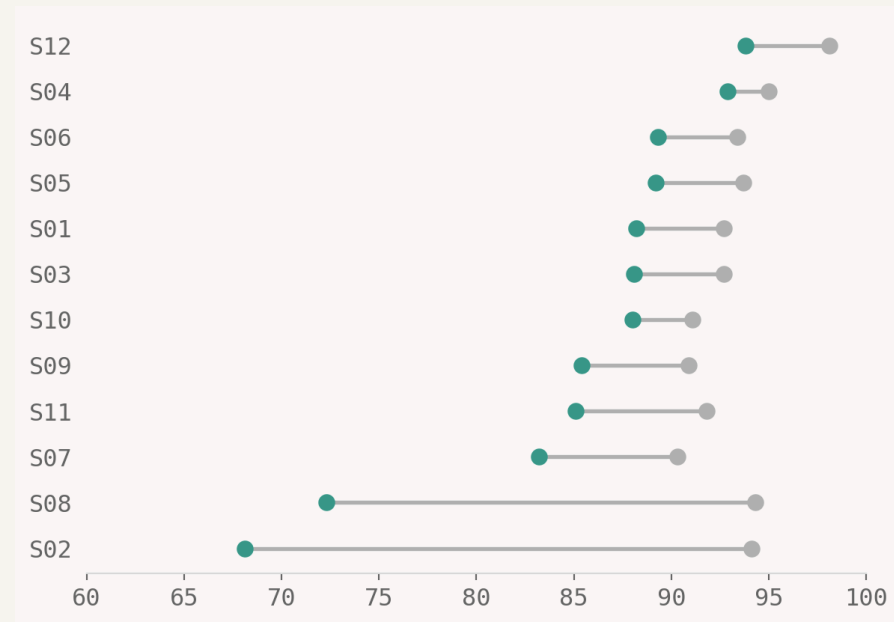
A Dumbbell for QC

MAKE IT PUBLICATION-READY

Add the dumbbell ends: raw and callable

Two dots per row: where the sample started (raw) and where it landed after QC (callable).

```
1 for i, r in qc.iterrows():
2     ax.plot([r.callable, r.raw],
3             [i, i], color=GREY, lw=2)
4
5 ax.scatter(qc.raw, y, color=GREY)      # raw mapping
6 ax.scatter(qc.callable, y, color=GREEN) # callable after QC
```



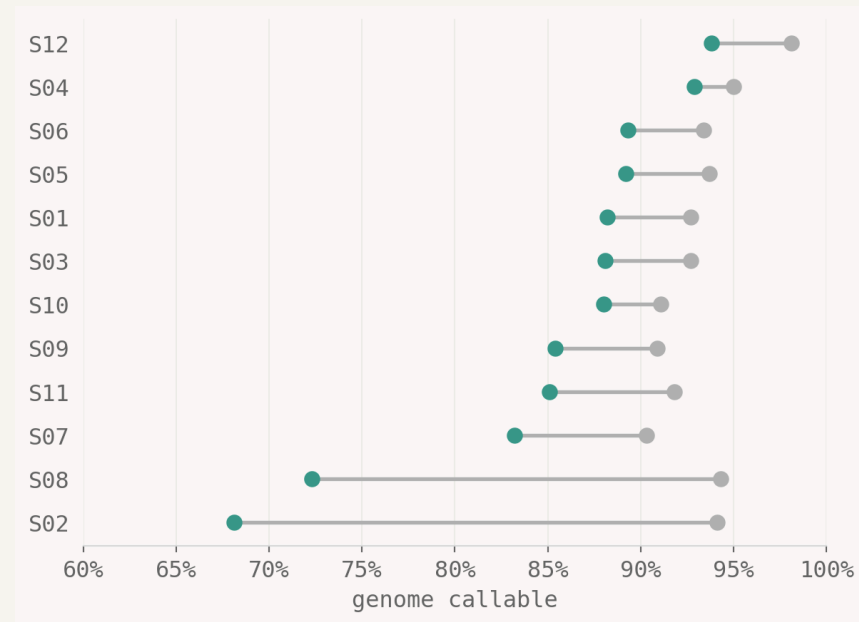
A Dumbbell for QC

MAKE IT PUBLICATION-READY

Style it: percent axis, quiet grid, labels

Format x as a percentage, add a faint x-grid so rows are easy to read across, and label the axis.

```
1 import matplotlib.ticker as mtick
2
3 ax.xaxis.set_major_formatter(
4     mtick.PercentFormatter(100, decimals=0))
5 ax.set_xlabel("genome callable")
6 ax.grid(axis="x", color=LINE, alpha=0.5)
```



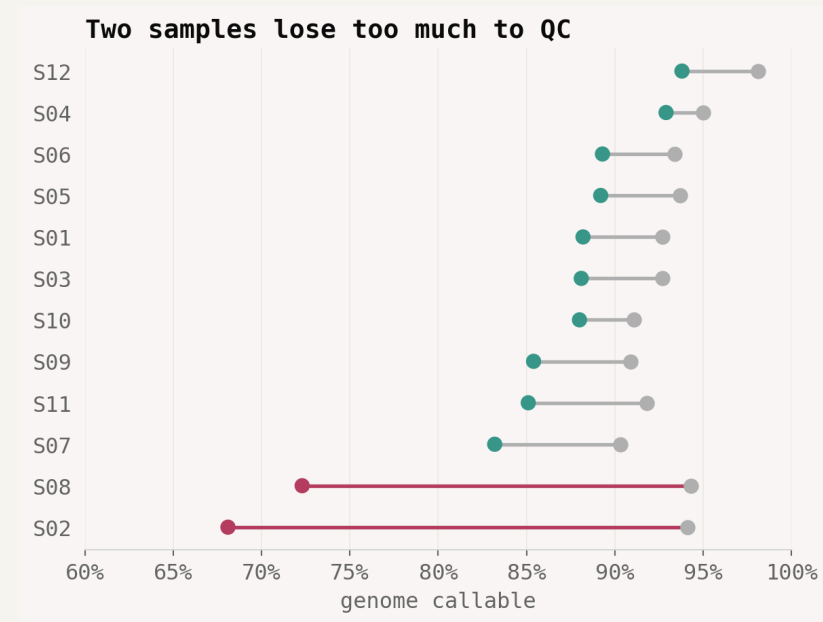
A Dumbbell for QC

MAKE IT PUBLICATION-READY

The message: one bold colour flags the problem

A sample that loses more than 15 points is a concern. Colour those bars and dots **red**, everything else stays quiet. The eye goes straight to S01 and S12.

```
1 qc["flag"] = qc.loss > 15          # the rule
2
3 bar = [RED if f else GREY for f in qc.flag]
4 dot = [RED if f else GREEN for f in qc.flag]
5
6 ax.set_title("Two samples lose too much to QC",
7             fontweight="bold", loc="left")
```



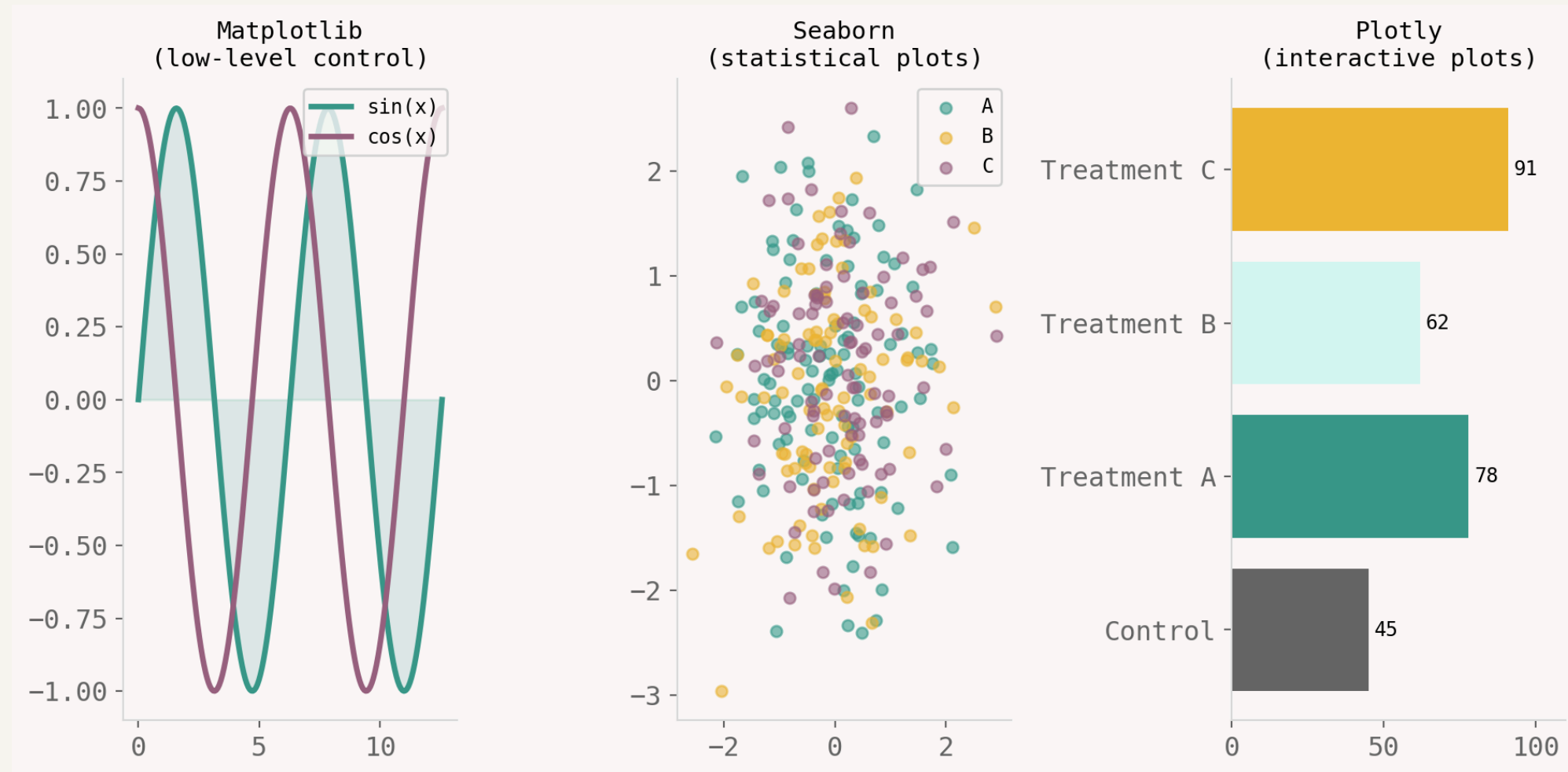
THE BOTTOM LINE

A dumbbell turns “before vs after” into one readable picture. Sort by the value that matters, and let **one bold colour** carry the verdict.

General-Purpose Libraries

THE PYTHON VIZ STACK

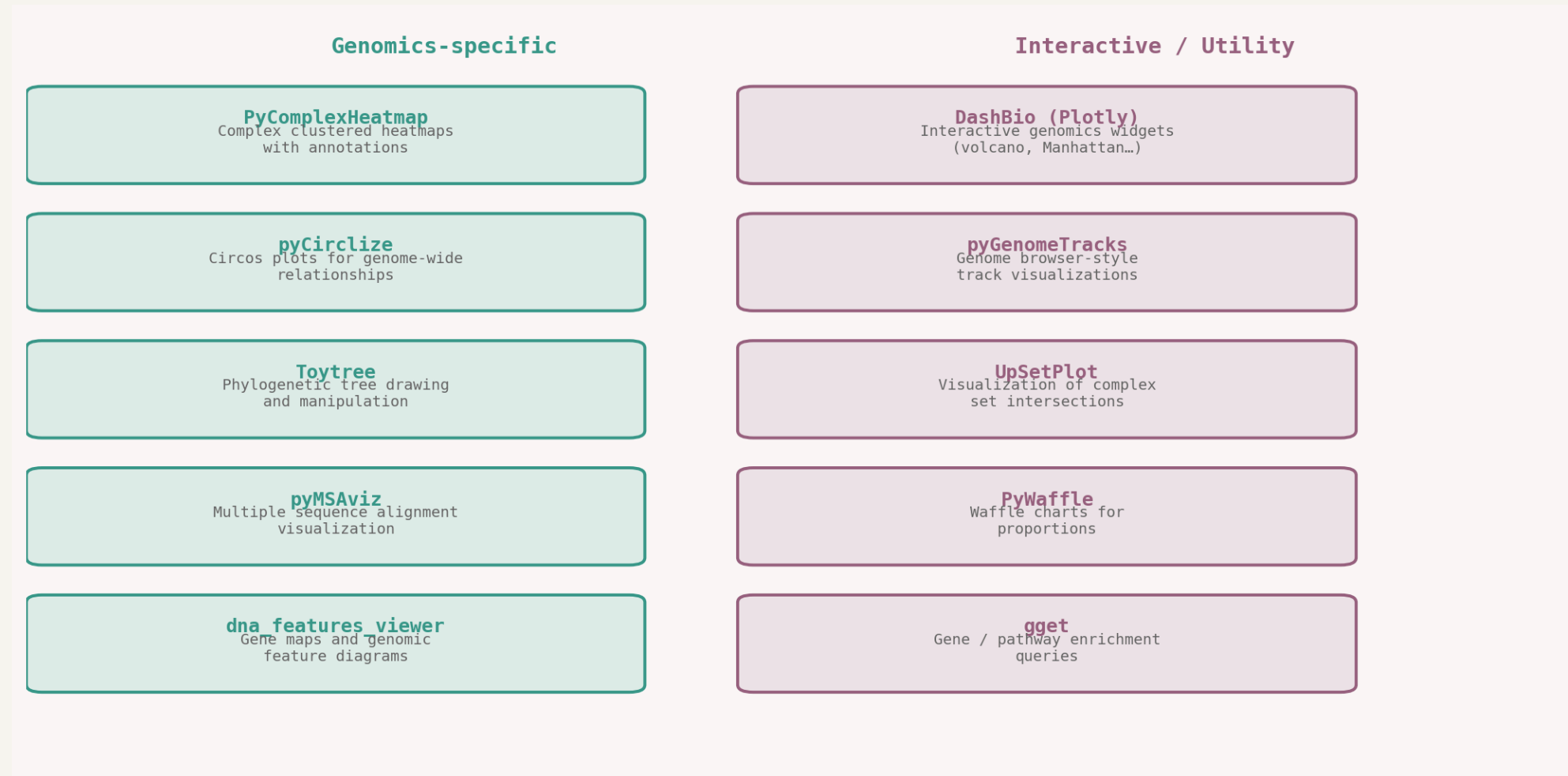
Matplotlib • Seaborn • Plotly



Genomics-Specific Libraries

THE PYTHON VIZ STACK

Two pillars of the genomics ecosystem



Day 1 Recap

DAY 01 • WRAP

What We Covered Today

DAY 01 • WRAP

Today’s tour, in two cards

Python visualization ecosystem

- matplotlib (figure anatomy, building plots step by step)
- seaborn (statistical visualizations, clustermap)
- plotly (interactive counterparts)
- Setting up a reproducible environment

Genomics plot types

Plot	Library	Use case
Volcano	matplotlib / DashBio	Differential expression
PCA / t-SNE / UMAP	scikit-learn	Dimensionality reduction
Heatmap	seaborn / PyComplexHeatmap	Expression matrices
Waffle	PyWaffle	Proportions
Waterfall	matplotlib	Mutation burden
Peaks	pyGenomeTracks / matplotlib	ChIP-seq / ATAC-seq
Pathway	gget / gseapy	Enrichment results

Looking Ahead

DAY 01 • WRAP

Days 2 and 3 preview

Day 2 (Tuesday 13:00 – 18:00):

- Stacked bar plots
- Time series plots
- Multiple sequence alignment visualization (`pyMSAviz`)
- Manhattan plots
- Genome statistics plots (coverage, gene density)

Day 3 (Wednesday 13:00 – 18:00):

- Phylogenetic trees (`Toytree` , `Bio.Phylo`)
- Circos plots (`pyCircLize`)
- Venn diagrams & UpSet plots (`matplotlib-venn` , `UpSetPlot`)
- Synteny plots (`JCVI` / `MCscan`)
- Ideograms

Download today's notebooks:

GENOMICS VIZ WITH PYTHON

```
1 # All code from today is available in:  
2 notebooks/day1_python_genomics_viz.ipynb
```

Resources

DAY 01 • WRAP

Where to go next

Essential reading:

Documentation:

- [Matplotlib docs](#)
- [Seaborn docs](#)
- [Plotly Python](#)
- [Python Graph Gallery](#)

Tutorials:

- [Scientific Visualization \(Rougier\)](#)
- [From Data to Viz](#)
- [Matplotlib cheatsheet](#)

Genomics-specific:

- [PyComplexHeatmap docs](#)
- [pyCircize docs](#)
- [pyGenomeTracks docs](#)
- [gget docs](#)
- [UpSetPlot docs](#)

Awesome lists:

- [Awesome Genome Visualization](#)
- [Awesome Bioinformatics](#)